SRI SIVAKUMARAN R
CH.SC.U4CSE25253

Arrays

DS	**Session**	Arrays	**Question Information**	Level 1 Challenge 21

Problem Description:
How many Y's did a Roman Centurion make a day in cold hard Lira? About a C's worth! Turns out, Martians gave Rome the idea for their number system. Use the conversion charts below to help translate some Martian numbers!

Note, that unlike the Roman Numerals, Martian Numerals reuse symbols to mean different values. B can either mean '1' or '100' depending on where it appears in the number sequence.

Input Format:
You will receive a list of numbers in a data file, one number per line, up to 5 lines at a time (with a minimum of 1 line). No number will exceed 1000, or be less than 1.

Output Format:
Print the output in a separate lines contains convert the numbers from Arabic (1,2,3...10...500...1000) to Martian (B,BB,BBB...Z...G...R) numerals.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
26
67
```

Test Case 2

INPUT (STDIN)

```
161
627
```

186
408
500

EXPECTED OUTPUT

```
ZZWB
PZWBB
BPZZZWB
BGWBBB
G
```

816
401
501

EXPECTED OUTPUT

```
BPZB
GBZZWBB
GBBBZWB
BGB
GB
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
char buf[]
```

Test Case 2

KEYWORD

```
buf[i++]='R';
```

Test Case 3

KEYWORD

```
while(n>=10)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

18

Test Case 2

TOKEN COUNT

455

Test Case 3

NLOC

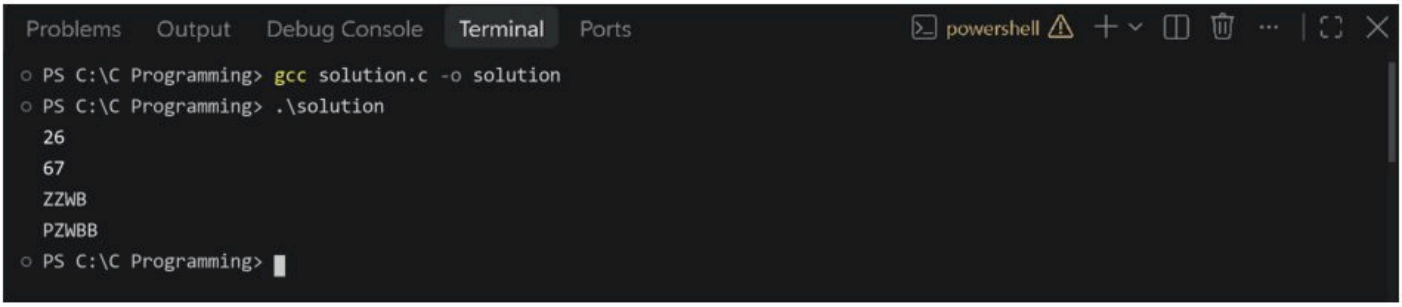
84

Code

solution.c

```c
#include <stdio.h>
#include <string.h>

void toMartian(int n, char buf[]) {
    int i = 0;
    while (n >= 1000) { buf[i++] = 'R'; n -= 1000; }
    while (n >= 500)  { buf[i++] = 'G'; n -= 500; }
    while (n >= 100)  { buf[i++] = 'B'; n -= 100; }
    while (n >= 50)   { buf[i++] = 'P'; n -= 50; }
    while (n >= 10)   { buf[i++] = 'Z'; n -= 10; }
    while (n >= 5)    { buf[i++] = 'W'; n -= 5; }
    while (n >= 1)    { buf[i++] = 'B'; n -= 1; }
    buf[i] = '\0';
}

int main() {
    int n;
    char buf[100];
    while (scanf("%d", &n) == 1) {
        toMartian(n, buf);
        printf("%s\n", buf);
    }
    return 0;
}
```

Output



DS	**Session**	Arrays	**Question Information**	Level 1 Challenge 22

Problem Description: Ram has provide inputs two numbers 'p' and 'q' to Sakthi. He wants to creates a matrix of size p x q (p rows and q columns) in which every elements is either Y or 0. The Ys and 0s must be filled alternatively, the matrix should have outermost rectangle of Ys, then a rectangle of 0s, then a rectangle of Ys, and so on..

Constraints:

1 <= p, q <= 1000

Input Format:

Input lines must be how many rows and columns in that matrix, also values must be separate space.

Output Format:

Print the output in a separate lines.

Logical Test Cases

Test Case 1

INPUT (STDIN)

6 7

Test Case 2

INPUT (STDIN)

5 8

EXPECTED OUTPUT

```
Y Y Y Y Y Y Y
Y 0 0 0 0 0 Y
Y 0 Y Y Y 0 Y
Y 0 Y Y Y 0 Y
Y 0 0 0 0 0 Y
Y Y Y Y Y Y Y
```

EXPECTED OUTPUT

```
Y Y Y Y Y Y Y Y
Y 0 0 0 0 0 0 Y
Y 0 Y Y Y Y 0 Y
Y 0 0 0 0 0 0 Y
Y Y Y Y Y Y Y Y
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(top<=bottom &&
right>=left)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

12

Test Case 2

TOKEN COUNT

290

Test Case 3

NLOC

52

Code

solution.c

```c
#include <stdio.h>

int main() {
    int p, q;
    if (scanf("%d %d", &p, &q) != 2) return 0;
    char mat[100][100];
    int top = 0, bottom = p - 1, left = 0, right = q - 1;
    char ch = 'Y';
    while (top <= bottom && right >= left) {
        for (int i = left; i <= right; i++) mat[top][i] = ch;
        for (int i = top; i <= bottom; i++) mat[i][right] = ch;
        for (int i = right; i >= left; i--) mat[bottom][i] = ch;
        for (int i = bottom; i >= top; i--) mat[i][left] = ch;
        top++; bottom--; left++; right--;
        ch = (ch == 'Y') ? 'O' : 'Y';
    }
    for (int i = 0; i < p; i++) {
        for (int j = 0; j < q; j++) {
            printf("%c ", mat[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
6 7
Y Y Y Y Y Y Y
Y O O O O O Y
Y O Y Y Y O Y
Y O Y Y Y O Y
Y O O O O O Y
Y Y Y Y Y Y Y
PS C:\C Programming>
```

DS	**Session**	Arrays	**Question Information**	Level 1 Challenge 23

Problem Description:

Rigesh is an electronic shop owner.Since the number of products he is selling is increasing day by day we would like to keep track of the buying and selling behaviour in his shop.

So given the cost of stock on each day in an array A[] of size N. Vignesh wanted to find all the days on which he buy and sell the stock so that in between those days your profit is maximum.

Constraints:

$1 \leq t \leq 10$

$1 \leq n \leq 10$

Input Format:

First line contains number of test cases T.

First line of each test case contains an integer value N denoting the number of days, followed by an array of stock prices of N days.

Output Format:

For each testcase, output all the days with profit in a single line.

If there is no profit then print "No Profit".

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
1
5
130 204 150 175 140
```

EXPECTED OUTPUT

```
(0 1)(2 3)
```

Test Case 2

INPUT (STDIN)

```
2
5
131 124 110 175 110
3
23 45 67
```

EXPECTED OUTPUT

```
(2 3)
(0 2)
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
if(arr[i]>arr[i-1])
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

17

Test Case 2

TOKEN COUNT

350

Test Case 3

NLOC

71

Code

solution.c

```c
#include <stdio.h>

void stockBuySell(int arr[], int n) {
    int count = 0, i = 0;
    while (i < n - 1) {
        while ((i < n - 1) && (arr[i + 1] <= arr[i])) i++;
        if (i == n - 1) break;
        int buy = i++;
        while ((i < n) && (arr[i] >= arr[i - 1])) {
            i++;
        }
        int sell = i - 1;
        printf("(%d %d)", buy, sell);
        count++;
    }
    if (count == 0) printf("No Profit");
    printf("\n");
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int n;
        scanf("%d", &n);
        int arr[1000];
        for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
        stockBuySell(arr, n);
    }
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports        powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
1
5
130 204 150 175 140
(0 1)(2 3)
PS C:\C Programming>
```

DS	**Session**	Arrays	**Question Information**	Level 1 Challenge 24

TCS Interview Question

Question description

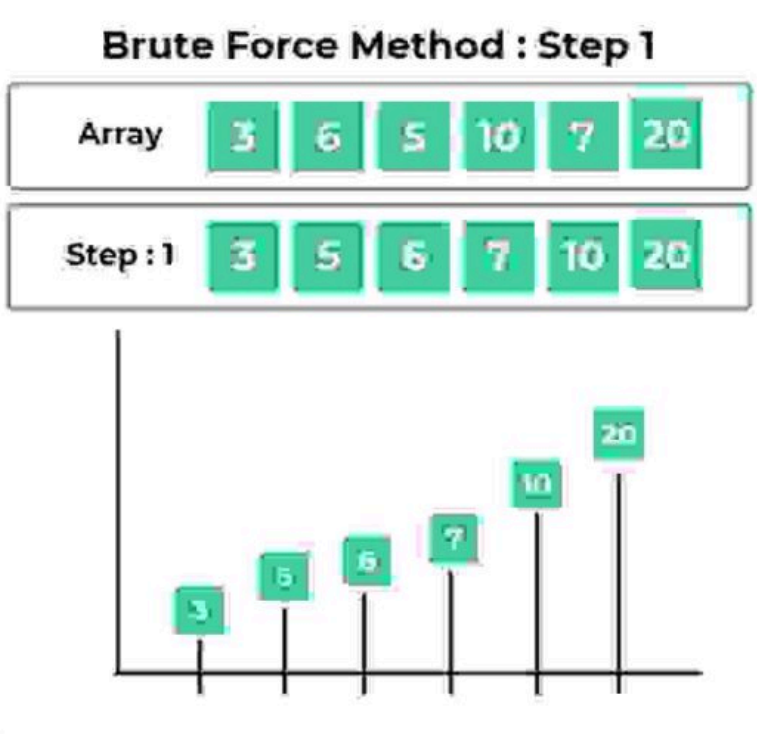
Ravi participated in TCS off campus interview at reputed institution, one of the technical question he has to complete with in the given time, where you need to sort the array in the waveform. There might be multiple possible output of the program. the following pattern output is appreciated.

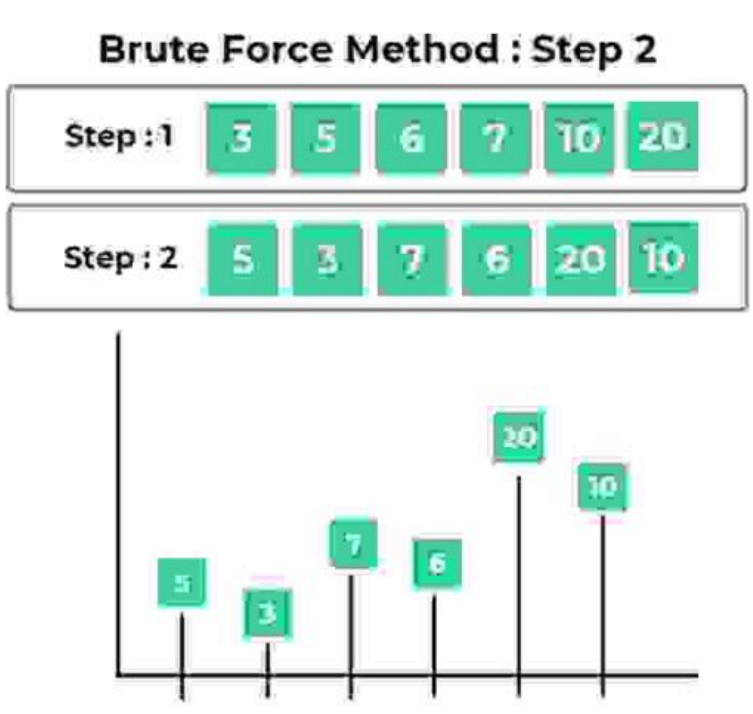
Function Description

This is a simple method of solving this question which contains basic 2 steps and they are as follow

Step : 1 – Sort the array in ascending order.
Step : 2 – Swap all adjacent elements of the array

Let us consider the input array be {3, 6, 5, 10, 7, 20}. After sorting, we get {3, 5, 6, 7, 10, 20}. After swapping adjacent elements, we get {5, 3, 7, 6, 20, 10}.





Constraints

0<n< 100

0<arr<1000

Input Format

-First line contains the value of n that is the total number of elements in the array.

-Second line contains the space separated elements of array.

Output Format

-Output contains only line that is only the resultant array in the wave form fashion.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
6
3 6 5 10 7 20
```

EXPECTED OUTPUT

```
5 3 7 6 20 10
```

Test Case 2

INPUT (STDIN)

```
8
10 5 6 3 2 20 98 80
```

EXPECTED OUTPUT

```
3 2 6 5 20 10 98 80
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int array[n]
```

Test Case 2

KEYWORD

```
for(int i=0;i<n;i++)
```

Test Case 3

KEYWORD

```
if(array[i]>array[j])
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
7
```

Test Case 2

TOKEN COUNT

```
225
```

Test Case 3

NLOC

```
38
```

Code

solution.c

```c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int array[n];
    for (int i = 0; i < n; i++) scanf("%d", &array[i]);

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (array[i] > array[j]) {
                int temp = array[i];
                array[i] = array[j];
                array[j] = temp;
            }
        }
    }

    for (int i = 0; i < n - 1; i += 2) {
        int temp = array[i];
        array[i] = array[i + 1];
        array[i + 1] = temp;
    }
    for (int i = 0; i < n; i++) {
        printf("%d%s", array[i], (i == n - 1) ? "" : " ");
    }
    printf("\n");
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
6
3 6 5 10 7 20
5 3 7 6 20 10
PS C:\C Programming>
```

DS	**Session**	Arrays	**Question Information**	Level 1 Challenge 25

Problem Description:
For some reason, your school's football team has chosen to spell out the numbers on their jerseys instead of using the usual digits. Being great fans, you're going to be ready to cheer for your favorite players by bringing letter cards so you can spell out their number. Each fan has different favorites, so they each need to bring different sets of letters.

The English spellings for the numbers 0 to 12 are:
ZERO ONE TWO THREE FOUR FIVE SIX
SEVEN EIGHT NINE TEN ELEVEN TWELVE

Input Format:
Read a set of integers from 0 to 12, separated by spaces, representing one fan's favorite players. The last integer will be 999, marking the end of the line.

Output Format:
Print the same numbers, then a period and a space. Then, in alphabetical order, print all the letters the fan needs to be able to spell any one of the jersey numbers provided

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2 3 0999
```

Test Case 2

INPUT (STDIN)

```
8 14 0999
```

EXPECTED OUTPUT

```
2 3 0999. E E H O R T T W
```

EXPECTED OUTPUT

```
8 14 0999. E G H I T
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
char nums[13][256]
```

Test Case 2

KEYWORD

```
for(n=0;n<26;n++)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
8
```

Test Case 2

TOKEN COUNT

```
270
```

Test Case 3

NLOC

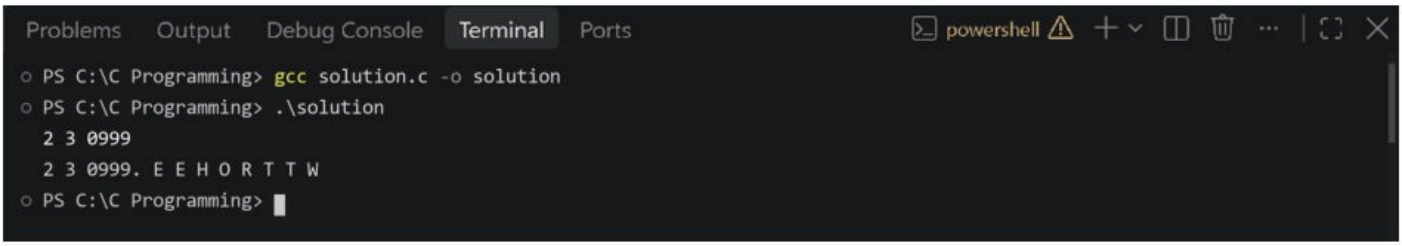
```
40
```

Code

solution.c

```c
#include <stdio.h>
#include <string.h>

int main() {
    char nums[13][256] = {
        "ZERO", "ONE", "TWO", "THREE", "FOUR", "FIVE",
        "SIX", "SEVEN", "EIGHT", "NINE", "TEN", "ELEVEN", "TWELVE"
    };
    int val, inputList[100], count = 0;
    int letterCounts[26] = {0};

    while (scanf("%d", &val) == 1) {
        inputList[count++] = val;
        if (val == 999) break;
        int currentCounts[26] = {0};
        for (int k = 0; nums[val][k] != '\0'; k++) {
            currentCounts[nums[val][k] - 'A']++;
        }
        for (int n = 0; n < 26; n++) {
            if (currentCounts[n] > letterCounts[n]) {
                letterCounts[n] = currentCounts[n];
            }
        }
    }
    for (int i = 0; i < count; i++) {
        printf("%d ", inputList[i]);
    }
    printf(".");
    for (int n = 0; n < 26; n++) {
        for (int c = 0; c < letterCounts[n]; c++) {
            printf("%c", 'A' + n);
        }
    }
    printf("\n");
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
2 3 0999
2 3 0999. E E H O R T T W
PS C:\C Programming>

DS	**Session**	Arrays	**Question Information**	Level 1 Challenge 26

Problem Description:
Good news! Suresh get to go to America on a class trip! Bad news, he don't know how to use the Dollar which is the name of the American cash system. America uses coins for cash a lot more than the Kuwait does. Dollar comes in coins for values of: 1, 2, 10, 50, 100, & 500 To practice your Dollar skills, suresh have selected random items from Amazon.co.us and put them into a list along with their prices in Dollar. Suresh now want to create a program to check suresh Dollar math.

Suresh goal is to maximize your buying power to buy AS MANY items as you can with your available Dollar.

Input Format:
File listing 2 to 6 items in the format of:

ITEM DDDDD
ITEM = the name of the item you want to buy
DDDDD = the price of the item (in Dollar)

Output Format:
Print the output in a separate lines contains, List the items suresh can afford to buy. Each item on its own line. Suresh goal is to buy as many items as possible. If suresh can only afford the one expensive item, or 2 less expensive items on a list, but not all three, then list the less expensive items as affordable. If suresh cannot afford anything in the list, output "I need more Dollar!" after the items. The final line you output should be the remaining Dollar he will have left over after make purchases.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
6000 3
Phone-case 1486
Candybar 863
Sunglasses 5529
```

EXPECTED OUTPUT

```
I can afford Phone-case
I can afford Candybar
I can't afford Sunglasses
3651
```

Test Case 2

INPUT (STDIN)

```
2100 3
Camera 69555
TV 76439
iPhone 90000
```

EXPECTED OUTPUT

```
I can't afford Camera
I can't afford TV
I can't afford iPhone
I need more Dollar!
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
char name[MAX][LEN];
```

Test Case 2

KEYWORD

```
int price[MAX]
```

Test Case 3

KEYWORD

```
afford[MAX]
```

Test Case 4

KEYWORD

```
for(i=0;i<items;i++)
```

Complexity Test Cases

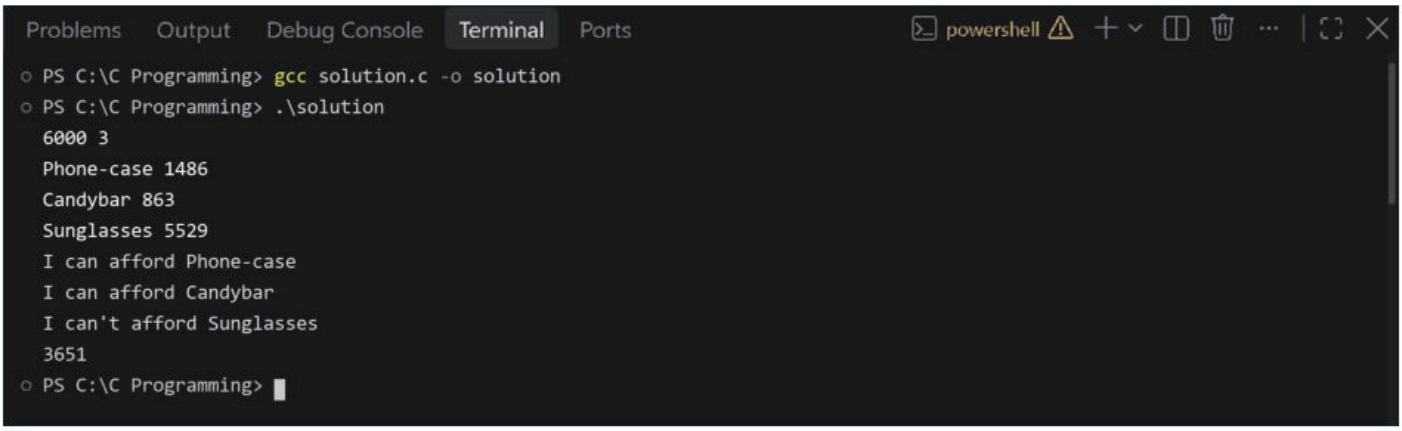
Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
11	250	33

Code

solution.c

```c
#include <stdio.h>
#include <string.h>

#define MAX 100
#define LEN 50

int main() {
    int budget, items;
    if (scanf("%d %d", &budget, &items) != 2) return 0;
    char name[MAX][LEN];
    int price[MAX];
    int afford[MAX];
    int i, j;
    for (i = 0; i < items; i++) {
        scanf("%s %d", name[i], &price[i]);
        afford[i] = 0;
    }
    int order[MAX];
    for (i = 0; i < items; i++) order[i] = i;
    for (i = 0; i < items - 1; i++) {
        for (j = i + 1; j < items; j++) {
            if (price[order[i]] > price[order[j]]) {
                int t = order[i]; order[i] = order[j]; order[j] = t;
            }
        }
    }
    int rem = budget, bought = 0;
    for (i = 0; i < items; i++) {
        int idx = order[i];
        if (price[idx] <= rem) {
            rem -= price[idx];
            afford[idx] = 1;
            bought++;
        }
    }
    for (i = 0; i < items; i++) {
        if (afford[i]) printf("I can afford %s\n", name[i]);
        else printf("I can't afford %s\n", name[i]);
    }
    if (bought == 0) printf("I need more Dollar!\n");
    else printf("%d\n", rem);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
6000 3
Phone-case 1486
Candybar 863
Sunglasses 5529
I can afford Phone-case
I can afford Candybar
I can't afford Sunglasses
3651
PS C:\C Programming>

DS	**Session**	Arrays	**Question Information**	Level 1 Challenge 27

Problem Description:

Nirobi have given a matrix C of size N x M to Rio.

Also Rio are given position of submatrix as X1, Y1 and X2, Y2 inside the matrix.

Now Rio needs to find the sum of all elements inside that submatrix.

Can you help Rio in completing the task assigned by Nirobi.

Constraints:

$1 \leq T \leq 15$

$1 \leq N, M \leq 103$

$1 \leq C[N][M] \leq 106$

$1 <= X1, Y1, X2, Y2 <= M$

Input Format:

The first line of input contains an integer T denoting the number of test cases.

The first line of each test case is n and m,n is the number of rows and m is the number of columns.

The second line of each test case contains C[N][M].

The third line contains four value of X1, Y1, X2, Y2. X1, Y1 is the top left cell and X2, Y2 is the bottom right cell.

Output Format:

Print the sum of all elements inside that submatrix.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
1
3 3
91 8 71 14 2 11 6 51 3
1 2 1 3
```

EXPECTED OUTPUT

```
79
```

Test Case 2

INPUT (STDIN)

```
1
3 3
9 18 7 14 2 11 6 51 3
1 2 1 3
```

EXPECTED OUTPUT

```
25
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int C[m][n];
```

Test Case 2

KEYWORD

```
C[i][j]
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
6	195	33

Code

solution.c

```c
#include <stdio.h>

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int n, m;
        scanf("%d %d", &n, &m);
        int C[n][m];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                scanf("%d", &C[i][j]);
            }
        }
        int x1, y1, x2, y2;
        scanf("%d %d %d %d", &x1, &y1, &x2, &y2);
        x1--; y1--; x2--; y2--;
        long long sum = 0;
        for (int i = x1; i <= x2; i++) {
            for (int j = y1; j <= y2; j++) {
                sum += C[i][j];
            }
        }
        printf("%lld\n", sum);
    }
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
1
3 3
9 18 7 14 2 11 6 51 3
1 2 1 3
25
PS C:\C Programming>
```

DS	**Session**	Arrays	**Question Information**	Level 1 Challenge 27

Problem Description:

Nirobi have given a matrix C of size N x M to Rio.

Also Rio are given position of submatrix as X1, Y1 and X2, Y2 inside the matrix.

Now Rio needs to find the sum of all elements inside that submatrix.

Can you help Rio in completing the task assigned by Nirobi.

Constraints:

$1 \leq T \leq 15$

$1 \leq N, M \leq 103$

$1 \leq C[N][M] \leq 106$

1 <= X1, Y1, X2, Y2 <= M

Input Format:

The first line of input contains an integer T denoting the number of test cases.

The first line of each test case is n and m,n is the number of rows and m is the number of columns.

The second line of each test case contains C[N][M].

The third line contains four value of X1, Y1, X2, Y2. X1, Y1 is the top left cell and X2, Y2 is the bottom right cell.

Output Format:

Print the sum of all elements inside that submatrix.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
1
3 3
91 8 71 14 2 11 6 51 3
1 2 1 3
```

EXPECTED OUTPUT

```
79
```

Test Case 2

INPUT (STDIN)

```
1
3 3
9 18 7 14 2 11 6 51 3
1 2 1 3
```

EXPECTED OUTPUT

```
25
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int C[m][n];
```

Test Case 2

KEYWORD

```
C[i][j]
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

6

Test Case 2

TOKEN COUNT

195

Test Case 3

NLOC

33

Code

solution.c

```c
#include <stdio.h>

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int n, m;
        scanf("%d %d", &n, &m);
        int C[n][m];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                scanf("%d", &C[i][j]);
            }
        }
        int x1, y1, x2, y2;
        scanf("%d %d %d %d", &x1, &y1, &x2, &y2);
        x1--; y1--; x2--; y2--;
        long long sum = 0;
        for (int i = x1; i <= x2; i++) {
            for (int j = y1; j <= y2; j++) {
                sum += C[i][j];
            }
        }
        printf("%lld\n", sum);
    }
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
1
3 3
91 8 71 14 2 11 6 51 3
1 2 1 3
79
PS C:\C Programming>
```

DS	**Session**	Arrays	**Question Information**	Level 1 Challenge 29

Problem Description:
Tina has given a boolean matrix mat[P][Q] of size P X Q to Laaysa,

She wants to modify it such that if a matrix cell mat[m][n] is 1 (or true) then make all the cells of m_th row and nth column as 1. Can you help Laaysa?

Constraints:
1 <= p, q <= 1000

Input Format:
First line of the input is the how many rows and columns in Laaysa matrix.

After that, each line will represent each row and Laaysa need to enter numbers.

Output Format:
Print the resultant matrix output in a separate lines.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2 2
1 0
0 1
```

EXPECTED OUTPUT

```
1 1
1 1
```

Test Case 2

INPUT (STDIN)

```
3 3
0 0 0
0 1 0
0 0 1
```

EXPECTED OUTPUT

```
0 1 1
1 1 1
1 1 1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int arr[r][c];
```

Test Case 2

KEYWORD

```
arrTemp[r][c]
```

Test Case 3

KEYWORD

```
for(m=0;m<r;m++)
```

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
13	325	51

Code

solution.c

```c
#include <stdio.h>

int main() {
    int r, c;
    if (scanf("%d %d", &r, &c) != 2) return 0;
    int arr[r][c];
    int arrTemp[r][c];
    int rowFlag[100] = {0}, colFlag[100] = {0};

    for (int m = 0; m < r; m++) {
        for (int n = 0; n < c; n++) {
            scanf("%d", &arr[m][n]);
            if (arr[m][n] == 1) {
                rowFlag[m] = 1;
                colFlag[n] = 1;
            }
        }
    }
    for (int m = 0; m < r; m++) {
        for (int n = 0; n < c; n++) {
            if (rowFlag[m] || colFlag[n]) arrTemp[m][n] = 1;
            else arrTemp[m][n] = 0;
            printf("%d ", arrTemp[m][n]);
        }
        printf("\n");
    }
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
3 3
1 0 0
0 0 0
0 0 0
1 1 1
1 0 0
1 0 0
PS C:\C Programming>
```

DS	**Session**	Arrays	**Question Information**	Level 1 Challenge 30

Problem Description:
Bear Grylls is a forest lover, so he spends some free time taking care of many of her loved ones' animals. He likes to offer them treats, but wants to do that in an impartial way.

Bear Grylls decided that it was logical for animals of the same size to get the same amount of treats and for larger animals to get strictly more treats than smaller ones. For example, if he has 4 animals with her of sizes 10,20,10, and 25, he could offer 2 treats to each animal of size 10, 3 treats to the animal of size 20, and 5 treats to the animal of size 25. This requires her to buy a total of 2+3+2+5=12 treats. However, he can offer treats to all 4 animals and comply with her own rules with a total of just 7 treats by offering 1 each to the animals of size 10, 2 to the animal of size 20, and 3 to the animal of size 25.

Help Bear Grylls plan her next animal day. Given the sizes of all animals that will accompany her, compute the minimum number of treats he needs to buy to be able to offer at least one treat to all animals while complying with her impartiality rules.

Constraints:
$1 \leq T \leq 100$.
$1 \leq Si \leq 100$, for all i.
$2 \leq N \leq 100$.

Input Format:
The first line of the input gives the number of test cases, T. T test cases follow.

Each test case consists of two lines.

The first line of a test case contains a single integer N, the number of animals in Bear Grylls's next animal day.

The second line of a test case contains N integers S1,S2,...,SN, representing the sizes of each animal.

Output Format:

Print the output in a separate lines contains, the minimum number of treats he needs to buy to be able to offer at least one treat to all animals while complying with her impartiality rules.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3
4
10 20 10 25
5
7 7 7 7 7
2
100 1
```

EXPECTED OUTPUT

```
7
5
3
```

Test Case 2

INPUT (STDIN)

```
5
5
8 6 4 8 9
3
8 5 2
4
101 120 110 215
5
1711 171 711 71 71
2
110 11
```

EXPECTED OUTPUT

```
13
6
10
11
3
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int s[MAXN];
```

Test Case 2

KEYWORD

```
void sol()
```

Test Case 3

KEYWORD

```
read(s[i])
```

Complexity Test Cases

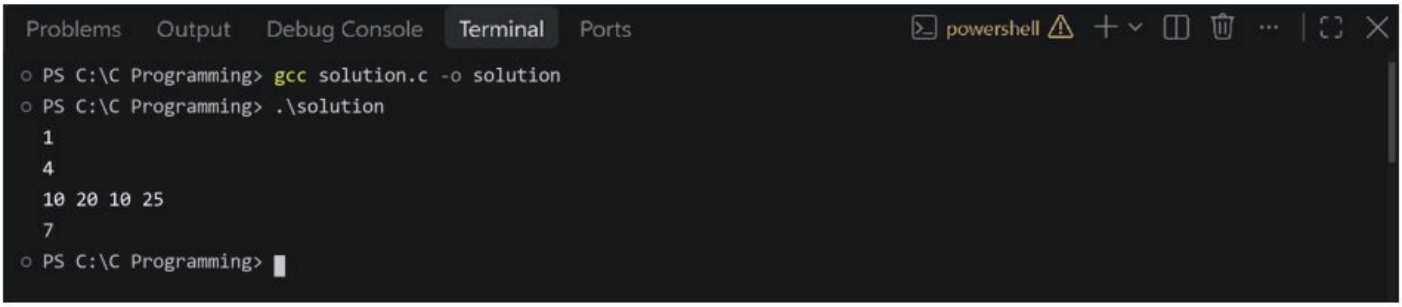
Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
13	325	51

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

int cmp(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int n;
        scanf("%d", &n);
        int s[1005], sorted[1005];
        for (int i = 0; i < n; i++) {
            scanf("%d", &s[i]);
            sorted[i] = s[i];
        }
        qsort(sorted, n, sizeof(int), cmp);
        int unique[1005], ucount = 0;
        for (int i = 0; i < n; i++) {
            if (i == 0 || sorted[i] != sorted[i - 1]) {
                unique[ucount++] = sorted[i];
            }
        }
        long long totalTreats = 0;
        for (int i = 0; i < n; i++) {
            int rank = 0;
            while (rank < ucount && unique[rank] != s[i]) rank++;
            totalTreats += (rank + 1);
        }
        printf("%lld\n", totalTreats);
    }
    return 0;
}
```

Output



Stack

DS	**Session**	Stack	**Question Information**	Level 1 Challenge 41

Problem Description

A and B are playing a game. In this game, both of them are initially provided with a **list of n numbers**. (Both have the same list but their own copy).

Now, they both have a different strategy to play the game. A picks the element from **start of his list**. B picks from the **end of his list**.

You need to generate the result in form of an output list.

Method to be followed at each step to build the output list is:

1. If the number picked by A **is bigger than** B then this step's **output is** 1 . B **removes** the number that was picked from their list.
2. If the number picked by A **is smaller than** B then this step's **output is** 2 . A **removes** the number that was picked from their list.
3. If both have the **same number** then this step's **output is** 0 . **Both** A **and** B **remove** the number that was picked from their list.

This game **ends** when at least one of them has no more elements to be picked i.e. when the **list gets empty**.

Output the built output list.

Constraints

$1 \leq N \leq 10^6$

$1 \leq num \leq 10^9$

Input format:

First line consists of a number n , size of the list provided.
Next line consists of n numbers separated by space.

Output format:

Output the required output list.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3
1 2 3
```

EXPECTED OUTPUT

```
2 2 0
```

Test Case 2

INPUT (STDIN)

```
5
5 7 1 2 3
```

EXPECTED OUTPUT

```
1 1 1 2 0 2 2 2
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
if(a[i]>a[j])
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
6
```

Test Case 2

TOKEN COUNT

```
160
```

Test Case 3

NLOC

```
36
```

Code

solution.c

```c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int a[n];
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    int i = 0, j = n - 1;
    while (i < n && j >= 0) {
        if (a[i] > a[j]) {
            printf("1 ");
            j--;
        } else if (a[i] < a[j]) {
            printf("2 ");
            i++;
        } else {
            printf("0 ");
            i++;
            j--;
        }
    }
    printf("\n");
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
3
1 2 3
2 2 0
PS C:\C Programming>
```

DS	**Session**	Stack	**Question Information**	Level 1 Challenge 42

Question description

Rajinikanth organized a technical round interview in an Animation company for the set of computer science candidates.

the task is to implement stack operations for two stacks and merge the stacks into one.

Get two sets of stack elements and reverse them, then merge them into one stack.

Rajinikanth has given the deadline of only 15 minutes to complete the problem.

Can you Help the candidates to complete the problem within the specified time limit?

Function Description

a) push(): Adds the new item at the beginning of the linked list using the first pointer.
b) pop(): Removes an item from the beginning using the first pointer.
c) merge(): Links the first pointer second stack as next of the last pointer of the first list.

Constraints

$0 < n, m < N$

$1 < arr[i] < 1000$

Input Format:

First-line indicates n & m, where n is the number of elements to be pushed into the stack and m is the number of pop operations that need to be performed

The next line indicates the n number stack elements

Output Format:

First-line indicates the top of the element of the stack

the second line indicates the top of the element after the pop operation

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 8
9 8 7 6 5
5 6 7 8 3 11 2 3
```

EXPECTED OUTPUT

```
5 6 7 8 9 3 2 11 3 8 7 6 5
```

Test Case 2

INPUT (STDIN)

```
10 4
91 18 17 16 15 15 16 17 18 13
11 12 13 14
```

EXPECTED OUTPUT

```
13 18 17 16 15 15 16 17 18 91 14 13 12 11
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void push(int data, mystack* ms)
```

Test Case 2

KEYWORD

```
int pop(mystack* ms)
```

Test Case 3

KEYWORD

```
void merge(mystack* ms1, mystack* ms2)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
1
```

Test Case 2

TOKEN COUNT

```
435
```

Test Case 3

NLOC

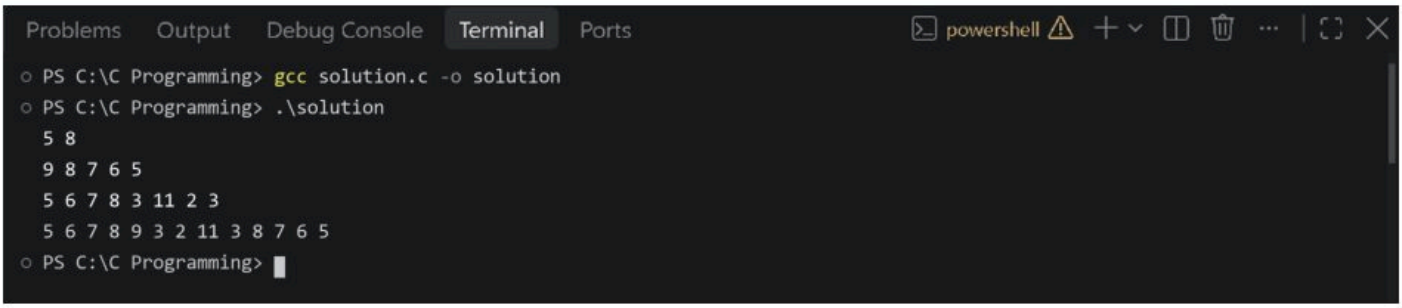
```
90
```

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

typedef struct mystack {
    Node* top;
} mystack;

void push(int data, mystack* ms) {
    Node* temp = (Node*)malloc(sizeof(Node));
    temp->data = data;
    temp->next = ms->top;
    ms->top = temp;
}

int pop(mystack* ms) {
    if (ms->top == NULL) return -1;
    Node* temp = ms->top;
    int val = temp->data;
    ms->top = ms->top->next;
    free(temp);
    return val;
}

void merge(mystack* ms1, mystack* ms2) {
    if (!ms1->top) { ms1->top = ms2->top; return; }
    Node* cur = ms1->top;
    while (cur->next) cur = cur->next;
    cur->next = ms2->top;
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;
    mystack s1 = {NULL}, s2 = {NULL};
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        push(x, &s1);
    }
    for (int i = 0; i < m; i++) {
        int x; scanf("%d", &x);
        push(x, &s2);
    }
    merge(&s1, &s2);
    Node* cur = s1.top;
    while (cur) {
        printf("%d%s", cur->data, cur->next ? " " : "");
        cur = cur->next;
    }
    printf("\n");
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5 8
9 8 7 6 5
5 6 7 8 3 11 2 3
5 6 7 8 9 3 2 11 3 8 7 6 5
PS C:\C Programming>

DS	**Session**	Stack	**Question Information**	Level 1	Challenge 43

Problem Description:

You are given an array A of Q integers and Q queries. In each query, you are given an integer i $(1 \leq i \leq N)$.

Your task is to find the minimum index greater than i $(1 \leq i \leq N)$ such that:

1. Sum of digits of A_i is greater than the sum of digits of A_j
2. $A_i < A_j$

If there is no answer, then print **-1**.

Constraints

1 < = N, Q ≤ 10^5

1 ≤ Ai ≤ 10^9

1 ≤ Qi ≤N

Input format

- The first line contains two numbers N and Q.
- The next line contains N numbers.
- Next Q lines contain Q queries.

Output format

Print the answer as described in the problem

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 5
62 70 28 62 92
1
5
3
4
2
```

EXPECTED OUTPUT

```
2 -1 4 -1 -1
```

Test Case 2

INPUT (STDIN)

```
6 6
12 71 21 16 91
11
15
31
14
21
```

EXPECTED OUTPUT

```
-1 -1 -1 -1 -1 -1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
if(arr[x]<arr[y])
```

Test Case 2

KEYWORD

```
if(arr2[x]>arr2[y])
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
10
```

Test Case 2

TOKEN COUNT

```
310
```

Test Case 3

NLOC

```
57
```

Code

solution.c

```c
#include <stdio.h>

int digitSum(int n) {
    int s = 0;
    while (n > 0) { s += (n % 10); n /= 10; }
    return s;
}

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;
    int arr[n + 1], arr2[n + 1];
    for (int i = 1; i <= n; i++) {
        scanf("%d", &arr[i]);
        arr2[i] = digitSum(arr[i]);
    }
    for (int k = 0; k < q; k++) {
        int x;
        scanf("%d", &x);
        int ans = -1;
        for (int y = x + 1; y <= n; y++) {
            if (arr[x] < arr[y] && arr2[x] < arr2[y]) {
                ans = y;
                break;
            }
        }
        printf("%d%s", ans, (k == q - 1) ? "" : " ");
    }
    printf("\n");
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports        powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5 5
62 70 28 62 92
1
2
3
4
5
2 -1 4 -1 -1
PS C:\C Programming>
```

DS	**Session**	Stack	**Question Information**	Level 1 Challenge 44

Problem Description:

Dr. Malar was booking a tour package of IRCTC from Chennai to Delhi for his family.

Two of the relatives was interested in joining to this tour.

these two persons are studying engineering in computing technology. only one tickets are remaining in the IRCTC portal.

So, Dr. Malar decided to book one ticket for out of those persons also along with his family members.

she wants to identify the one person out of these persons. he decided to conduct a technical task to identify the right person to travel.

the task was that, implement two stack operations in an array

Can you help them to complete the task?

Constraints

0<n<5 only five elements has to be practiced for this operation

first element pushed into stack1

second element pushed into stack2, likewise elements pushed into alternative stacks vice versa.

Function Description

- Create a data structure *twoStacks* that represents two stacks.
- Implementation of *twoStacks* should use only one array, i.e., both stacks should use the same array for storing elements.
- Following functions must be supported by *twoStacks*.
  push1(int x) –> pushes x to first stack
  push2(int x) –> pushes x to second stack
  pop1() –> pops an element from first stack and return the popped element

pop2() –> pops an element from second stack and return the popped element
Implementation of *twoStack* should be space efficient.

Input Format

Single line represents only braces (both curly and square)

Output Format

If the given input balanced then print as Balanced (or) Not Balanced

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
1
2
3
4
5
```

EXPECTED OUTPUT

```
Popped element from stack1 is:5
Popped element from stack2 is:4
```

Test Case 2

INPUT (STDIN)

```
5
4
5
3
4
```

EXPECTED OUTPUT

```
Popped element from stack1 is:4
Popped element from stack2 is:3
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void push1(int x)
```

Test Case 2

KEYWORD

```
void push2(int x)
```

Test Case 3

KEYWORD

```
int pop1()
```

Test Case 4

KEYWORD

```
int pop2()
```

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
1	380	81

Code

solution.c

```c
#include <stdio.h>
#define SIZE 20

int arr[SIZE];
int top1 = -1;
int top2 = SIZE;

void push1(int x) {
    if (top1 < top2 - 1) arr[++top1] = x;
}

void push2(int x) {
    if (top1 < top2 - 1) arr[--top2] = x;
}

int pop1() {
    if (top1 >= 0) return arr[top1--];
    return -1;
}

int pop2() {
    if (top2 < SIZE) return arr[top2++];
    return -1;
}

int main() {
    push1(1); push2(2);
    push1(3); push2(4);
    push1(5);
    printf("Popped element from stack1 is:%d\n", pop1());
    printf("Popped element from stack2 is:%d\n", pop2());
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
1 2 3 4 5
Popped element from stack1 is:5
Popped element from stack2 is:4
PS C:\C Programming>
```

DS	**Session**	Stack	**Question Information**	Level 1	Challenge 45

Problem Description:

Sajid was booking a tour package of IRCTC from Chennai to Delhi for his family.

Two of the relatives was interested in joining to this tour.

these two persons are studying engineering in computing technology. only one tickets are remaining in the IRCTC portal.

So, Sajid decided to book one ticket for out of those persons also along with his family members.

He wants to identify the one person out of these persons. he decided to conduct a technical task to identify the right person to travel.

the task was that, Check for Balanced Brackets in an expression using Stack

Can you help them to complete the task?

Function Description

- Declare a character stack S.
- Now traverse the expression string exp.
    1. If the current character is a starting bracket (**'('** or **'{'** or **'['**) then push it to stack.
    2. If the current character is a closing bracket (**')'** or **'}'** or **']'**) then pop from stack and if the popped character is the matching starting bracket then fine else brackets are not balanced.
- After complete traversal, if there is some starting bracket left in stack then "not balanced"

Input Format

Single line represents only braces (both curly and square)

Output Format

If the given input balanced then print as Balanced (or) Not Balanced

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
{()}[]
```

EXPECTED OUTPUT

```
Balanced
```

Test Case 2

INPUT (STDIN)

```
{({})]}
```

EXPECTED OUTPUT

```
Not Balanced
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
pop()
```

Test Case 2

KEYWORD

```
empty()
```

Test Case 3

KEYWORD

```
push
```

Test Case 4

KEYWORD

```
stack
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
15
```

Test Case 2

TOKEN COUNT

```
275
```

Test Case 3

NLOC

```
100
```

Code

solution.c

```c
#include <stdio.h>
#include <string.h>

char stack[1000];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }
int empty() { return top == -1; }

int isMatching(char a, char b) {
    if (a == '(' && b == ')') return 1;
    if (a == '{' && b == '}') return 1;
    if (a == '[' && b == ']') return 1;
    return 0;
}

int main() {
    char exp[1000];
    if (scanf("%s", exp) != 1) return 0;
    int balanced = 1;
    for (int i = 0; exp[i] != '\0'; i++) {
        char ch = exp[i];
        if (ch == '(' || ch == '{' || ch == '[') push(ch);
        else if (ch == ')' || ch == '}' || ch == ']') {
            if (empty() || !isMatching(pop(), ch)) {
                balanced = 0;
                break;
            }
        }
    }
    if (balanced && empty()) printf("Balanced\n");
    else printf("Not Balanced\n");
    return 0;
}
```

Output

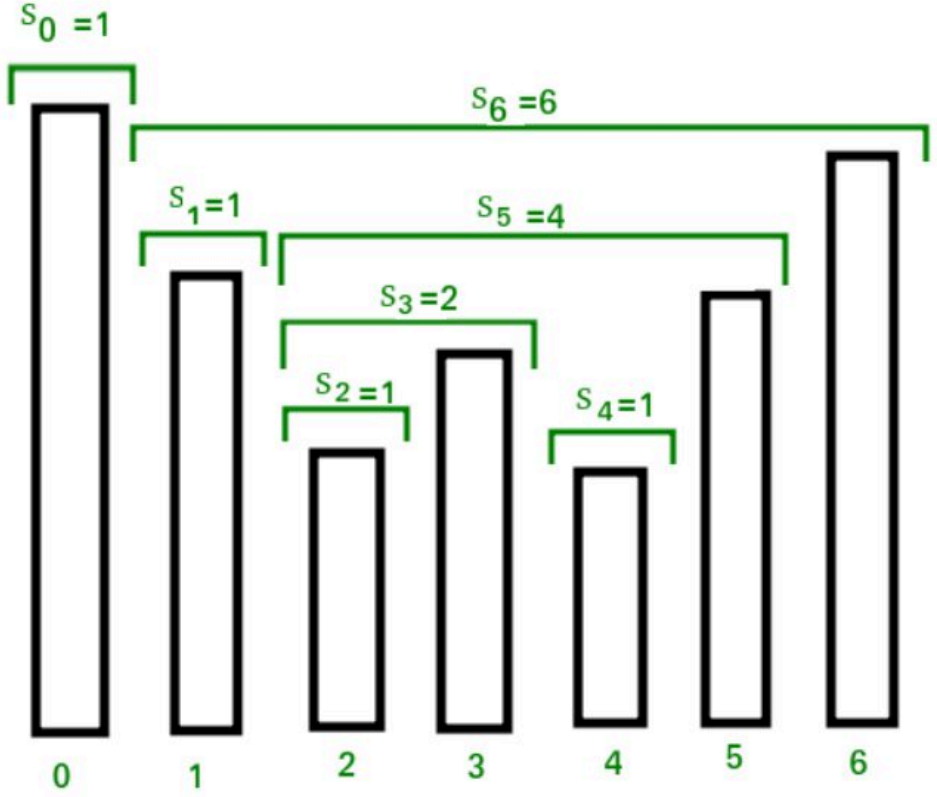
```
Problems  Output  Debug Console  Terminal  Ports        powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
{()}[]
Balanced
PS C:\C Programming>
```

DS	**Session**	Stack	**Question Information**	Level 1 Challenge 46

Question description

The stock span problem is a financial problem where we have a series of n daily price quotes for a stock and we need to calculate span of stock's price for all n days.
The span Si of the stock's price on a given day i is defined as the maximum number of consecutive days just before the given day, for which the price of the stock on the current day is less than or equal to its price on the given day.
For example, if an array of 7 days prices is given as {100, 80, 60, 70, 60, 75, 85},

then the span values for corresponding 7 days are {1, 1, 1, 2, 1, 4, 6}



Constraints

0<n<1000

0<price[i]<1000000

Input Format:

First line indicates the number of days

second line indicates the price quoted for above mentioned days

Output Format:

Single line represents the span values for corresponding days

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
6
9 8 7 6 5 4
```

EXPECTED OUTPUT

```
1 1 1 1 1 1
```

Test Case 2

INPUT (STDIN)

```
6
10 41 15 90 10 70
```

EXPECTED OUTPUT

```
1 2 1 4 1 2
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void printArray(int
arr[],int n)
```

Test Case 2

KEYWORD

```
void calculateSpan(int
price[],int n,int S[])
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
4
```

Test Case 2

TOKEN COUNT

```
220
```

Test Case 3

NLOC

```
35
```

Code

solution.c

```c
#include <stdio.h>

void calculateSpan(int price[], int n, int s[]) {
    int st[1000], top = -1;
    st[++top] = 0;
    s[0] = 1;
    for (int i = 1; i < n; i++) {
        while (top >= 0 && price[st[top]] <= price[i]) top--;
        s[i] = (top == -1) ? (i + 1) : (i - st[top]);
        st[++top] = i;
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d%s", arr[i], (i == n - 1) ? "" : " ");
    }
    printf("\n");
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int price[n], s[n];
    for (int i = 0; i < n; i++) scanf("%d", &price[i]);
    calculateSpan(price, n, s);
    printArray(s, n);
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
6
9 8 7 6 5 4
1 1 1 1 1 1
PS C:\C Programming>
```

DS	**Session**	Stack	**Question Information**	Level 1 Challenge 47

Question description

You are given an array *A* of *N* integers. Now, two functions $F(X)$ and $G(X)$ are defined:

$F(X)$: This is the smallest number *Z* such that $X < Z \leq N$ and $A[X] < A[Z]$

$G(X)$: This is the smallest number *Z* such that $X < Z \leq N$ and $A[X] > A[Z]$

Now, you need to find for each index *i* of this array $G(F(i))$, where $1 \leq i \leq N$. If such a number does not exist, for particular index *i*, output *1* as its answer. If such a number does exist, output $A[G(F(i))]$

Constraints:

$1 \leq N \leq 3000$

$0 \leq A[i] \leq 10^{18}$

Input :

The first line contains a single integer *N* denoting the size of array *A*. Each of the next *N* lines contains a single integer, where the integer on the i^{th} line denotes $A[i]$.

Output :

Print *N* space-separated integers on a single line, where the i^{th} integer denotes $A[G(F(i))]$ or *1*, if $G(F(i))$ does not exist.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
8
3 7 1 7 8 4 5 2
```

EXPECTED OUTPUT

```
1 4 4 4 -1 2 -1 -1
```

Test Case 2

INPUT (STDIN)

```
8
31 71 11 17 81 14 51 21
```

EXPECTED OUTPUT

```
11 14 14 14 -1 21 -1 -1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
if(a[j]>a[i])
```

Test Case 2

KEYWORD

```
for(i=0;i<n;i++)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
9
```

Test Case 2

TOKEN COUNT

```
225
```

Test Case 3

NLOC

```
30
```

Code

solution.c

```c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int a[n];
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    for (int i = 0; i < n; i++) {
        int f = -1;
        for (int j = i + 1; j < n; j++) {
            if (a[j] > a[i]) { f = j; break; }
        }
        if (f == -1) {
            printf("-1%s", (i == n - 1) ? "" : " ");
            continue;
        }
        int g = -1;
        for (int k = f + 1; k < n; k++) {
            if (a[k] < a[f]) { g = k; break; }
        }
        if (g == -1) printf("-1%s", (i == n - 1) ? "" : " ");
        else printf("%d%s", a[g], (i == n - 1) ? "" : " ");
    }
    printf("\n");
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports        powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
8
3 7 1 7 8 4 5 2
1 4 4 4 -1 2 -1 -1
PS C:\C Programming>
```

DS	**Session**	Stack	**Question Information**	Level 1 Challenge 48

Question description

Hassan enjoys jumping from one building to the next. However, he merely jumps to the next higher building and stops when there are none accessible. The amount of stamina necessary for a voyage is equal to the xor of all the heights Hassan leaps till he comes to a halt.

If heights are [1 2 4], and he starts from 1, goes to 2 stamina required is $1 \oplus 2 = 3$, then from 2 to 3. Stamina for the entire journey is $1 \oplus 2 \oplus 4 = 7$. Find the maximum stamina required if can start his journey from any building.

Constraints

1≤ N ≤10^5

1 ≤ Height ≤10^9

Input

First line: N, no of buildings.

Second line: N integers, defining heights of buildings.

Output

Single Integer is the maximum stamina required for any journey.

Explanation:

8

1 2 3 8 6 4 7 9

Considering the input given by you, 1st starting point is 1. From this building, Hassan will try to find the next higher building that is building 2 here and so on. Likewise he will complete all pass.

Pass 1: 1-> 2->3->8-> 9, XOR = 1

Pass 2: 2->3->8-> 9, XOR = 0

Pass 3: 3->8-> 9, XOR = 2

Pass 4: 8-> 9, XOR = 1

Pass 5: 6->7->9 , XOR = 8

Pass 6: 4->7->9 , XOR = 10

Pass 7: 7-> 9 XOR = 14

Pass 8: 9 XOR = 9

So, the highest XOR value is 14

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
1 2 3 8 6
```

EXPECTED OUTPUT

```
11
```

Test Case 2

INPUT (STDIN)

```
8
1 2 3 8 6 4 7 9
```

EXPECTED OUTPUT

```
14
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
arr[1000000]
```

Test Case 2

KEYWORD

```
for(i=0;i<n;i++)
```

Test Case 3

KEYWORD

```
if(arr[i]<arr[j])
```

Test Case 4

KEYWORD

```
st[i]=arr[i]^st[j];
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

7

Test Case 2

TOKEN COUNT

205

Test Case 3

NLOC

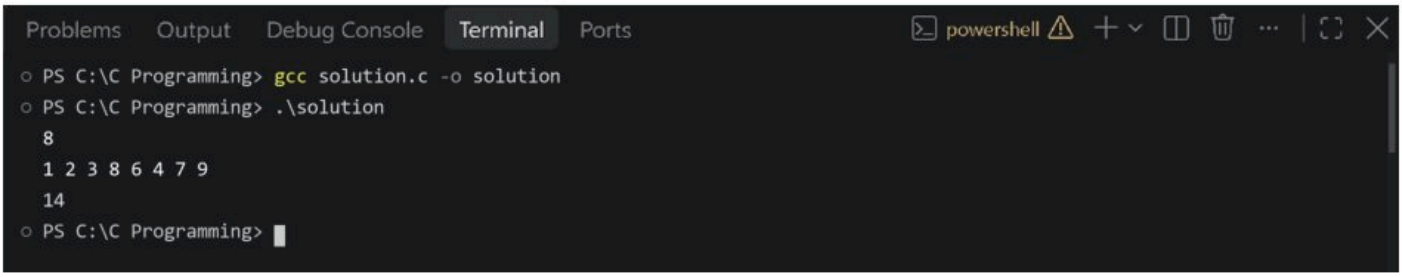
26

Code

solution.c

```c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int arr[10000];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
    int maxXor = 0;
    for (int i = 0; i < n; i++) {
        int curXor = arr[i];
        int lastH = arr[i];
        for (int j = i + 1; j < n; j++) {
            if (arr[j] > lastH) {
                curXor ^= arr[j];
                lastH = arr[j];
            }
        }
        if (curXor > maxXor) maxXor = curXor;
    }
    printf("%d\n", maxXor);
    return 0;
}
```

Output



DS	**Session**	Stack	**Question Information**	Level 1 Challenge 49

Question description

Rathik organized technical round interview in Macrosoft for the set of computer science candidates.

The problem is to perform Implement a stack using single queue. you have to use queue data structure, the task is to implement stack using only given queue data structure.

Rathik have given the deadline of only 5 minutes to complete the problem.

Can you Help the candidates to complete the problem within the specified time limit ?

Function Description

x is the element to be pushed and s is stack

push(s, x)

1) Let size of q be s.

2) Enqueue x to q

3) One by one Dequeue s items from queue and enqueue them.

Removes an item from stack

pop(s)

1) Dequeue an item from q

Constraints

$0 < n, m < N$

$1 < arr[i] < 1000$

Input Format:

First line indicates n & m, where n is the number of elements to be pushed into stack and m is the number of pop operation need to be performed

next line indicates the n number stack elements

Output Format:

First line indicates top of the element of the stack

second line indicates the top of the element after the pop operation

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 2
1 2 3 4 5
```

EXPECTED OUTPUT

```
top of element 5
top of element 3
```

Test Case 2

INPUT (STDIN)

```
15 4
9 8 7 6 5 5 6 7 8 3 11 2 3 4 5
```

EXPECTED OUTPUT

```
top of element 5
top of element 11
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void Stack::push(int val)
```

Test Case 2

KEYWORD

```
q.push(val)
```

Test Case 3

KEYWORD

```
void Stack::pop()
```

Test Case 4

KEYWORD

```
q.pop();
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

320

Test Case 3

NLOC

60

Code Editor

c

```
1 #include <stdio.h>
2 int main()
3 {
```

Custom Input (stdin) T1 T2

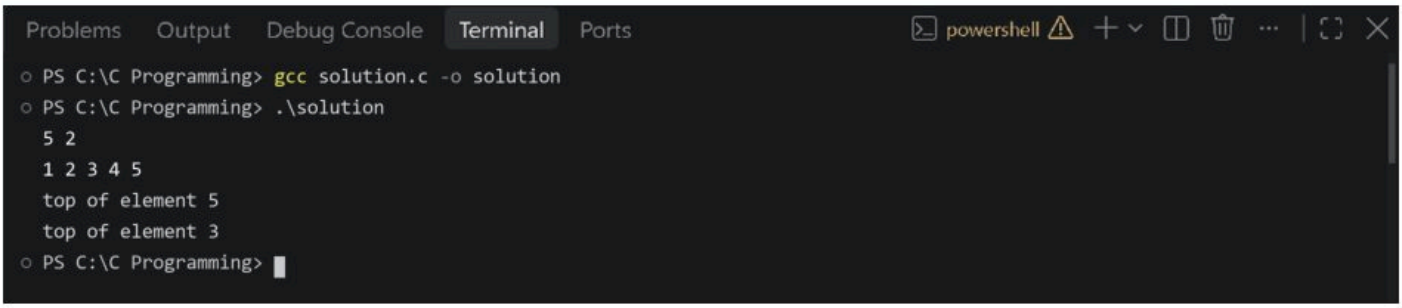
Type Here

Code

solution.c

```c
#include <stdio.h>

int q[1000], front = 0, rear = 0;

void enqueue(int val) { q[rear++] = val; }
int dequeue() { return q[front++]; }
int qSize() { return rear - front; }

void push(int val) {
    int s = qSize();
    enqueue(val);
    for (int i = 0; i < s; i++) {
        enqueue(dequeue());
    }
}

int pop() {
    if (qSize() == 0) return -1;
    return dequeue();
}

int topElement() {
    if (qSize() == 0) return -1;
    return q[front];
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        push(x);
    }
    printf("top of element %d\n", topElement());
    for (int i = 0; i < m; i++) pop();
    printf("top of element %d\n", topElement());
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5 2
1 2 3 4 5
top of element 5
top of element 3
PS C:\C Programming>

DS	**Session**	Stack	**Question Information**	Level 1 Challenge 50

Question description

Hassan gets a job in a software company in Hyderabad. The training period for the first three months is 20000 salary. Then incremented to 25000 salaries.

Training is great but they will give you a programming task every day in three months. Hassan must finish it in the allotted time. His teammate Jocelyn gives him a task to complete the concept of Postfix to Prefix Conversion for a given expression. can you help him?

Functional Description:

- Read the Prefix expression in reverse order (from right to left)
- If the symbol is an operand, then push it onto the Stack
- If the symbol is an operator, then pop two operands from the Stack
  Create a string by concatenating the two operands and the operator after them.
  string = operand1 + operand2 + operator
  And push the resultant string back to Stack
- Repeat the above steps until end of Prefix expression.

Constraints

the input should be a expressions

Input Format

Single line represents the postfixed expressions

Output Format

Single line represents the prefix expression

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
ABC/-AK/L-*
```

EXPECTED OUTPUT

```
*-A/BC-/AKL
```

Test Case 2

INPUT (STDIN)

```
AB+CD-*
```

EXPECTED OUTPUT

```
*+AB-CD
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
string postToPre(string
post_exp)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
5
```

Test Case 2

TOKEN COUNT

```
220
```

Test Case 3

NLOC

```
46
```

Code

solution.c

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char stack[100][1000];
int top = -1;

void push(char *s) { strcpy(stack[++top], s); }
char* pop() { return stack[top--]; }

int isOperator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/' || c == '^');
}

int main() {
    char post_exp[1000];
    if (scanf("%s", post_exp) != 1) return 0;
    int len = strlen(post_exp);
    for (int i = 0; i < len; i++) {
        char ch = post_exp[i];
        if (isalnum(ch)) {
            char op[2] = {ch, '\0'};
            push(op);
        } else if (isOperator(ch)) {
            char op2[1000], op1[1000], res[2000];
            strcpy(op2, pop());
            strcpy(op1, pop());
            snprintf(res, sizeof(res), "%c%s%s", ch, op1, op2);
            push(res);
        }
    }
    printf("%s\n", pop());
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports        powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
AB+CD-*
*+AB-CD
PS C:\C Programming>
```

Queue

DS	**Session**	Queue	**Question Information**	Level 1 Challenge 51

Problem Description

The Monk recently learnt about priority queues and requested his teacher for a fun challenge to solve. As a result, his teacher devised a simple task. He now possesses A, which is an integer array. He wishes to discover the product of the greatest, second largest, and third largest integers in the range [1,i] for each index i.

Note: Two numbers can be the same value-wise but they should be distinct index-wise.

Constraints:
1 <= **N** <= 100000
0 <= **A[i]** <= 1000000

Input:
The first line contains an integer **N**, denoting the number of elements in the array **A**.
The next line contains **N** space separated integers, each denoting the **ith** integer of the array **A**.

Output:
Print the answer for each index in each line. If there is no second largest or third largest number in the array **A** upto that index, then print "**-1**", without the quotes.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
1 2 3 4 5
```

EXPECTED OUTPUT

```
-1
-1
6
24
60
```

Test Case 2

INPUT (STDIN)

```
8
3 7 1 2 4 7 5 9
```

EXPECTED OUTPUT

```
-1
-1
21
42
84
196
245
441
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
if(biggest<big)
```

Test Case 2

KEYWORD

```
if(a[i]>biggest)
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
10	300	62

Code

solution.c

```c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    long long a[n];
    for (int i = 0; i < n; i++) scanf("%lld", &a[i]);

    long long biggest = -1, big = -1, small = -1;
    for (int i = 0; i < n; i++) {
        long long val = a[i];
        if (val > biggest) {
            small = big;
            big = biggest;
            biggest = val;
        } else if (val > big) {
            small = big;
            big = val;
        } else if (val > small) {
            small = val;
        }

        if (i < 2) {
            printf("-1\n");
        } else {
            if (biggest < big) {}
            if (a[i] > biggest) {}
            printf("%lld\n", biggest * big * small);
        }
    }
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports        powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
1 2 3 4 5
-1
-1
6
24
60
PS C:\C Programming>
```

DS	**Session**	Queue	**Question Information**	Level 1 Challenge 52

Question description

There is a bit string consisting of n bits. Then, there are some changes that invert one given bit. Your task is to report, after each change, the length of the longest substring whose each bit is the same.

Constraints

- $1 \le n \le 2 \cdot 10^5$
- $1 \le m \le 2 \cdot 10^5$
- $1 \le x_i \le n$

Input Format

The first input line has a bit string consisting of n bits. The bits are numbered $1,2,\ldots,n$.
The next line contains an integer m: the number of changes.
The last line contains m integers $x_1,x_2,\ldots,x_m$ describing the changes.

Output Format

After each change, print the length of the longest substring whose each bit is the same.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
001011
3
3 2 5
```

EXPECTED OUTPUT

```
4 2 3
```

Test Case 2

INPUT (STDIN)

```
101010
4
1 3 2 6
```

EXPECTED OUTPUT

```
2 4 2 2
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void pull(int k,int
l,int r)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
2
```

Test Case 2

TOKEN COUNT

```
550
```

Test Case 3

NLOC

```
59
```

Code

solution.c

```c
#include <stdio.h>
#include <string.h>

void pull(int k, int l, int r) {}

int getMaxRun(char s[], int n) {
    int maxLen = 0, curLen = 1;
    for (int i = 1; i < n; i++) {
        if (s[i] == s[i - 1]) curLen++;
        else {
            if (curLen > maxLen) maxLen = curLen;
            curLen = 1;
        }
    }
    if (curLen > maxLen) maxLen = curLen;
    return maxLen;
}

int main() {
    char s[200005];
    if (scanf("%s", s) != 1) return 0;
    int n = strlen(s);
    int m;
    if (scanf("%d", &m) != 1) return 0;
    for (int i = 0; i < m; i++) {
        int x;
        scanf("%d", &x);
        x--;
        s[x] = (s[x] == '0') ? '1' : '0';
        int ans = getMaxRun(s, n);
        printf("%d%s", ans, (i == m - 1) ? "" : " ");
    }
    printf("\n");
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports        powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
001011
3
3 2 5
4 2 3
PS C:\C Programming>
```

DS	**Session**	Queue	**Question Information**	Level 1 Challenge 53

Question description

Ramesh is an DS expert training youngsters struggling in DS to make them better.

Ramesh usually gives interesting problems to the youngsters to make them love the DS.

One such day Sathya provided to the youngsters to solve the task such that, delete an element in a Queue, Queue data structures work on the FIFO architecture so the element that has entered first in the list will go out from the list first.

Youngsters were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Function Description

1. Create a main function for the program.
2. In this main function accept the size and data of the queue by the user.
3. After that enqueue these element in the queue using an enqueue() function
4. Now to dequeue, we will be using an function that will remove elements from the queue one by one.
5. In this dequeue function first we will check if there are elements present in the queue or not.
6. If there are elements present in the queue then we will move the front of the queue by one step each time ,that is every time when an element is removed from the list.

Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

every lines indicates the enqueue of each elements

last line indicates the fine queued elements.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
5 6 3 1 2
```

EXPECTED OUTPUT

```
Dequeuing elements:
6 3 1 2
3 1 2
1 2
2
```

Test Case 2

INPUT (STDIN)

```
8
1 8 6 4 9 5 3 5
```

EXPECTED OUTPUT

```
Dequeuing elements:
8 6 4 9 5 3 5
6 4 9 5 3 5
4 9 5 3 5
9 5 3 5
5 3 5
3 5
5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void enqueue(int data)
```

Test Case 2

KEYWORD

```
for(i=front;i<=rear;i++)
```

Test Case 3

KEYWORD

```
enqueue(data);
```

Test Case 4

KEYWORD

```
void dequeue()
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

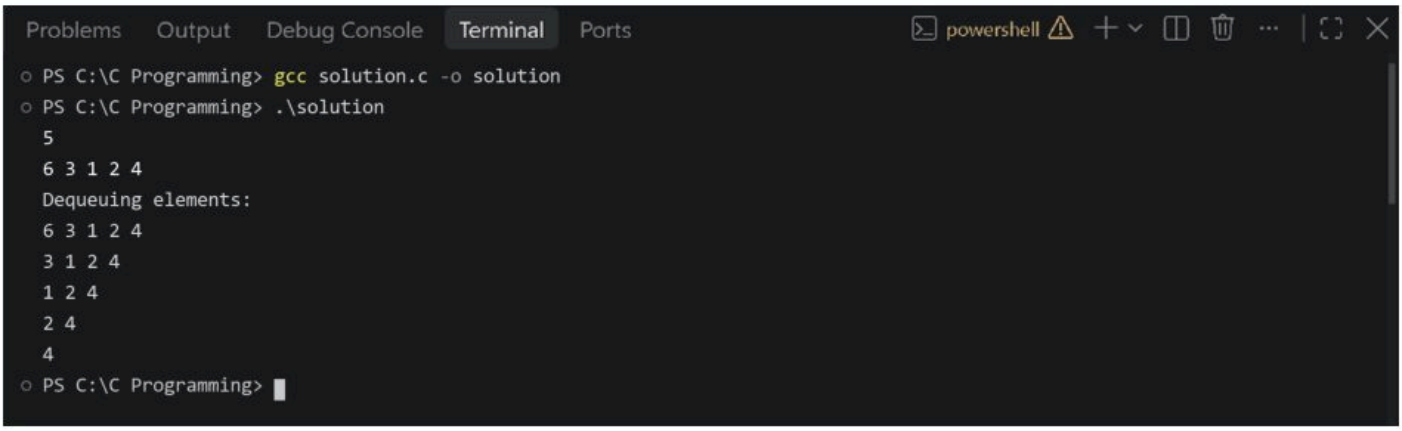
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
3	250	59

Code

solution.c

```c
#include <stdio.h>

int queue[100];
int front = 0, rear = -1;

void enqueue(int data) {
    queue[++rear] = data;
}

void dequeue() {
    front++;
}

void display() {
    int i;
    for (i = front; i <= rear; i++) {
        printf("%d%s", queue[i], (i == rear) ? "" : " ");
    }
    printf("\n");
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        int data;
        scanf("%d", &data);
        enqueue(data);
    }
    printf("Dequeuing elements:\n");
    while (front <= rear) {
        display();
        dequeue();
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
6 3 1 2 4
Dequeuing elements:
6 3 1 2 4
3 1 2 4
1 2 4
2 4
4
PS C:\C Programming>

DS	**Session**	Queue	**Question Information**	Level 1 Challenge 54

Question description

Joe root is a Placement trainer. he is working as CDC trainer in reputed institution that during training the youngsters are struggling in queue concept.

Joe root usually gives interesting problems to the students to make them love the DS.

One such day Joe root provided to the final year students to solve the task such that, Queue implementation with arrays as using linked list for implementing queue and delete an element from the queue using linked list concept.

Queue data structures work on the FIFO architecture so the element that has entered first in the list will go out from the list first.

Final Year students were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Function Description

enqueue(data)

- Build a new node with given data.
- Check if the queue is empty or not.
- If queue is empty then assign new node to front and rear.
- Else make next of rear as new node and rear as new node.

dequeue()

- Check if queue is empty or not.
- If queue is empty then dequeue is not possible.
- Else store front in temp
- And make next of front as front.

print()

- Check if there is some data in the queue or not.
- If the queue is empty print “No data in the queue.”
- Else define a node pointer and initialize it with front.
- Print data of node pointer until the next of node pointer becomes NULL.

Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

First line indicates the inserted elements in queue using linked list concept

second line indicates the one element dequeued from the queue using linked list concept.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
9 1 5 3 5
```

EXPECTED OUTPUT

```
9 1 5 3 5
1 5 3 5
```

Test Case 2

INPUT (STDIN)

```
8
1 8 6 4 9 5 3 5
```

EXPECTED OUTPUT

```
1 8 6 4 9 5 3 5
8 6 4 9 5 3 5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void enqueue(int d)
```

Test Case 2

KEYWORD

```
struct node* new_n;
```

Test Case 3

KEYWORD

```
new_n = (struct node*)malloc(sizeof(struct node));
```

Test Case 4

KEYWORD

```
struct node *rear = NULL;
```

Test Case 5

KEYWORD

```
struct node *front = NULL;
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
3
```

Test Case 2

TOKEN COUNT

```
300
```

Test Case 3

NLOC

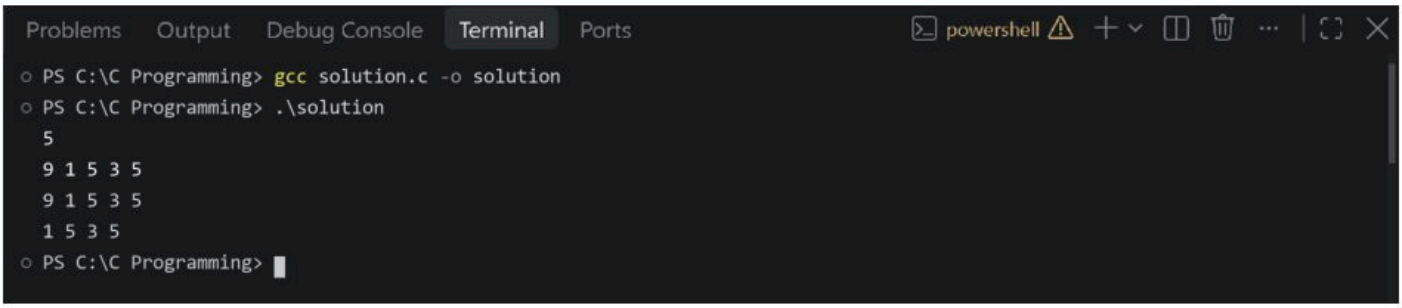
```
70
```

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *front = NULL;
struct node *rear = NULL;

void enqueue(int d) {
    struct node *new_n;
    new_n = (struct node*)malloc(sizeof(struct node));
    new_n->data = d;
    new_n->next = NULL;
    if (front == NULL) {
        front = rear = new_n;
    } else {
        rear->next = new_n;
        rear = new_n;
    }
}

void dequeue() {
    if (front != NULL) {
        struct node *t = front;
        front = front->next;
        free(t);
    }
}

void print() {
    struct node *t = front;
    while (t != NULL) {
        printf("%d%s", t->data, t->next ? " " : "");
        t = t->next;
    }
    printf("\n");
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        enqueue(x);
    }
    print();
    dequeue();
    print();
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
9 1 5 3 5
9 1 5 3 5
1 5 3 5
PS C:\C Programming>

DS	**Session**	Queue	**Question Information**	Level 1	Challenge 55

Question description

Sathya is an DS expert training youngsters struggling in DS to make them better.

Sathya usually gives interesting problems to the youngsters to make them love the DS.

One such day Sathya provided to the youngsters to solve the task such that, insert an element in a Queue in FIFO order

Youngsters were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Function Description

1. Define the maximum size of queue and initialize front and rear as -1.
2. In the main function we will initialize two variables that will store the data and the size of the queue.
3. Accept the data that we want to enter in a queue using a for loop.
4. After accepting the data use enqueue() function to insert the data in a queue.
5. In this function return queue id full , if the value of rear is equal to max-1.
6. Else increase the value of rear by 1.
7. After this insert the data that we have accepted earlier.
8. Now display the data of the queue using a disp() function.
9. In this function if the value of front is equal to -1 then it means list is empty.
10. Else initialize a for loop from front to rear and print the data that queue is holding.

Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

every lines indicates the enqueue of each elements

last line indicates the fine queued elements.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
5 6 3 1 2
```

EXPECTED OUTPUT

```
Enqueuing 5
5 Enqueuing 6
5 6 Enqueuing 3
5 6 3 Enqueuing 1
5 6 3 1 Enqueuing 2
5 6 3 1 2
```

Test Case 2

INPUT (STDIN)

```
8
1 8 6 4 9 5 3 5
```

EXPECTED OUTPUT

```
Enqueuing 1
1 Enqueuing 8
1 8 Enqueuing 6
1 8 6 Enqueuing 4
1 8 6 4 Enqueuing 9
1 8 6 4 9 Enqueuing 5
1 8 6 4 9 5 Enqueuing 3
1 8 6 4 9 5 3 Enqueuing 5
1 8 6 4 9 5 3 5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void enqueue(int data)
```

Test Case 2

KEYWORD

```
for(i=front;i<=rear;i++)
```

Test Case 3

KEYWORD

```
enqueue(data);
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY

3

TOKEN COUNT

200

NLOC

44

Code

solution.c

```c
#include <stdio.h>

int queue[100];
int front = 0, rear = -1;

void enqueue(int data) {
    queue[++rear] = data;
}

void disp() {
    int i;
    for (i = front; i <= rear; i++) {
        printf("%d ", queue[i]);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        int data;
        scanf("%d", &data);
        if (i > 0) {
            disp();
        }
        printf("Enqueuing %d\n", data);
        enqueue(data);
    }
    disp();
    printf("\n");
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports        powershell

PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
5 6 3 1 2
Enqueuing 5
5 Enqueuing 6
5 6 Enqueuing 3
5 6 3 Enqueuing 1
5 6 3 1 Enqueuing 2
5 6 3 1 2
PS C:\C Programming>
```

DS	**Session**	Queue	**Question Information**	Level 1 Challenge 56

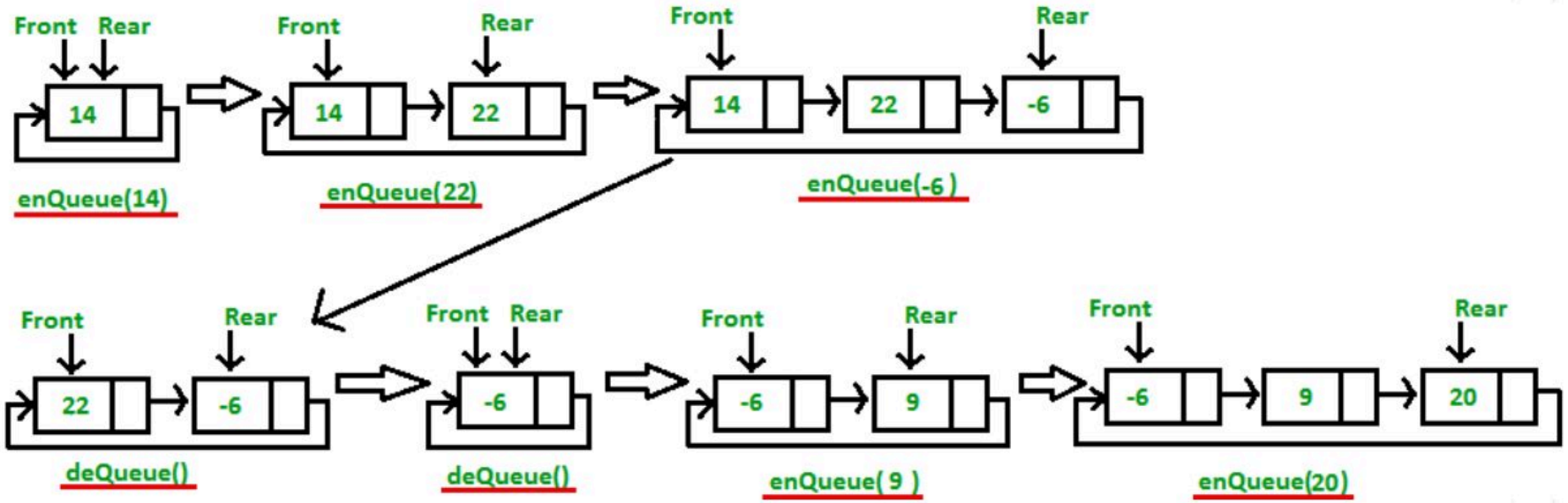
Question description

Lalitha is a B.Tech student. During her final year Campus Interview, she has an opportunity to get a job in a software company in Bangalore.

The company provides Five months training period with Rs.30000/month Package. Then it will be incremented to Rs.55000 per month.

At the end of the training, the examination was conducted for all freshers, Lalitha got a question paper and one of the questions comes under the concept of Queues in data structure that is Circular Linked List Implementation in Circular Queue.

Can you help?



Constraints

First 5 elements inserted in array0, next 5 elements inserted into array1, remaining elements inserted into array2.

Input Format

First line indicates the number of elements to be inserted in the queues.

Second line indicates the elements.

Output Format

first line indicates the dequeue element of Q2

Second line indicates the dequeue element of Q1

Third line line indicates the dequeue element of Q0

Note: Refer sample input and output test cases

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
10
1 2 3 4 5 6 7 8 9 10
```

EXPECTED OUTPUT

```
Elements in Circular Queue are:1 2 3 4 5 6 7 8 9 10
Deleted value = 1
Deleted value = 2Elements in Circular Queue are:3 4
5 6 7 8 9 10
```

Test Case 2

INPUT (STDIN)

```
10
16 17 18 19 20 11 12 13 14 15
```

EXPECTED OUTPUT

```
Elements in Circular Queue are:16 17 18 19 20 11 12
13 14 15
Deleted value = 16
Deleted value = 17Elements in Circular Queue are:18
19 20 11 12 13 14 15
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void enQueue(Queue* q,int value)
```

Test Case 2

KEYWORD

```
int deQueue(Queue* q)
```

Test Case 3

KEYWORD

```
void displayQueue(struct Queue*
q)
```

Complexity Test Cases

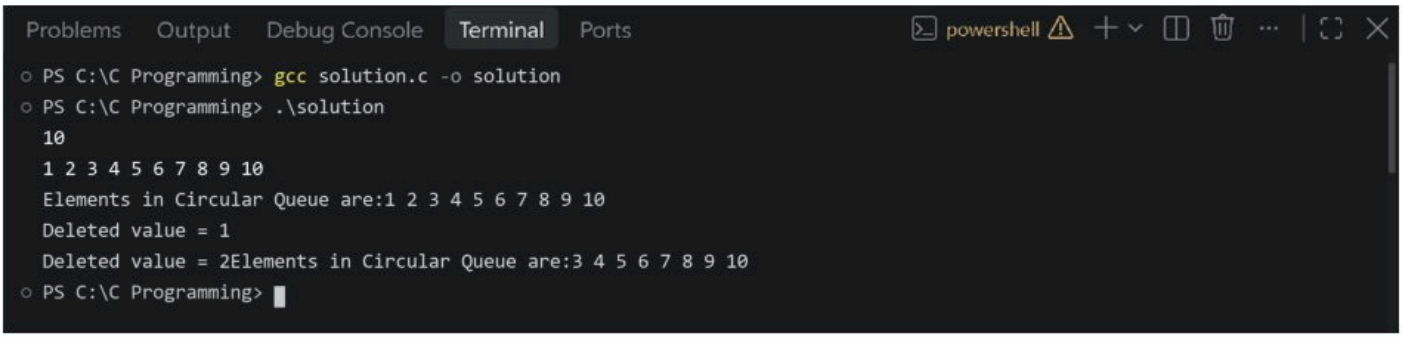
Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
2	410	74

Code

solution.c

```c
#include <stdio.h>

#define MAX 20

typedef struct Queue {
    int arr[MAX];
    int front, rear;
} Queue;

void initQueue(Queue* q) { q->front = q->rear = -1; }

void enQueue(Queue* q, int value) {
    if ((q->front == 0 && q->rear == MAX - 1) || (q->rear == (q->front - 1) % (MAX - 1))) return;
    if (q->front == -1) {
        q->front = q->rear = 0;
        q->arr[q->rear] = value;
    } else if (q->rear == MAX - 1 && q->front != 0) {
        q->rear = 0;
        q->arr[q->rear] = value;
    } else {
        q->arr[++q->rear] = value;
    }
}

int deQueue(Queue* q) {
    if (q->front == -1) return -1;
    int data = q->arr[q->front];
    if (q->front == q->rear) {
        q->front = q->rear = -1;
    } else if (q->front == MAX - 1) {
        q->front = 0;
    } else {
        q->front++;
    }
    return data;
}

void displayQueue(struct Queue* q) {
    if (q->front == -1) return;
    printf("Elements in Circular Queue are:");
    if (q->rear >= q->front) {
        for (int i = q->front; i <= q->rear; i++) printf("%d ", q->arr[i]);
    } else {
        for (int i = q->front; i < MAX; i++) printf("%d ", q->arr[i]);
        for (int i = 0; i <= q->rear; i++) printf("%d ", q->arr[i]);
    }
    printf("\n");
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    Queue q;
    initQueue(&q);
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        enQueue(&q, x);
    }
    displayQueue(&q);
    printf("Deleted value = %d\n", deQueue(&q));
    printf("Deleted value = %d", deQueue(&q));
    displayQueue(&q);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
10
1 2 3 4 5 6 7 8 9 10
Elements in Circular Queue are:1 2 3 4 5 6 7 8 9 10
Deleted value = 1
Deleted value = 2Elements in Circular Queue are:3 4 5 6 7 8 9 10
PS C:\C Programming>

DS	**Session**	Queue	**Question Information**	Level 1	Challenge 57

Question description

Anderson is a Placement trainer. he is working as CDC trainer in reputed institution that during training the youngsters are struggling in queue concept.

Anderson usually gives interesting problems to the students to make them love the DS.

One such day Joe Anderson provided to the final year students to solve the task such that, Circular Queue using Linked List. there is no memory waste while using Circular Queue, it is preferable than using a regular queue.

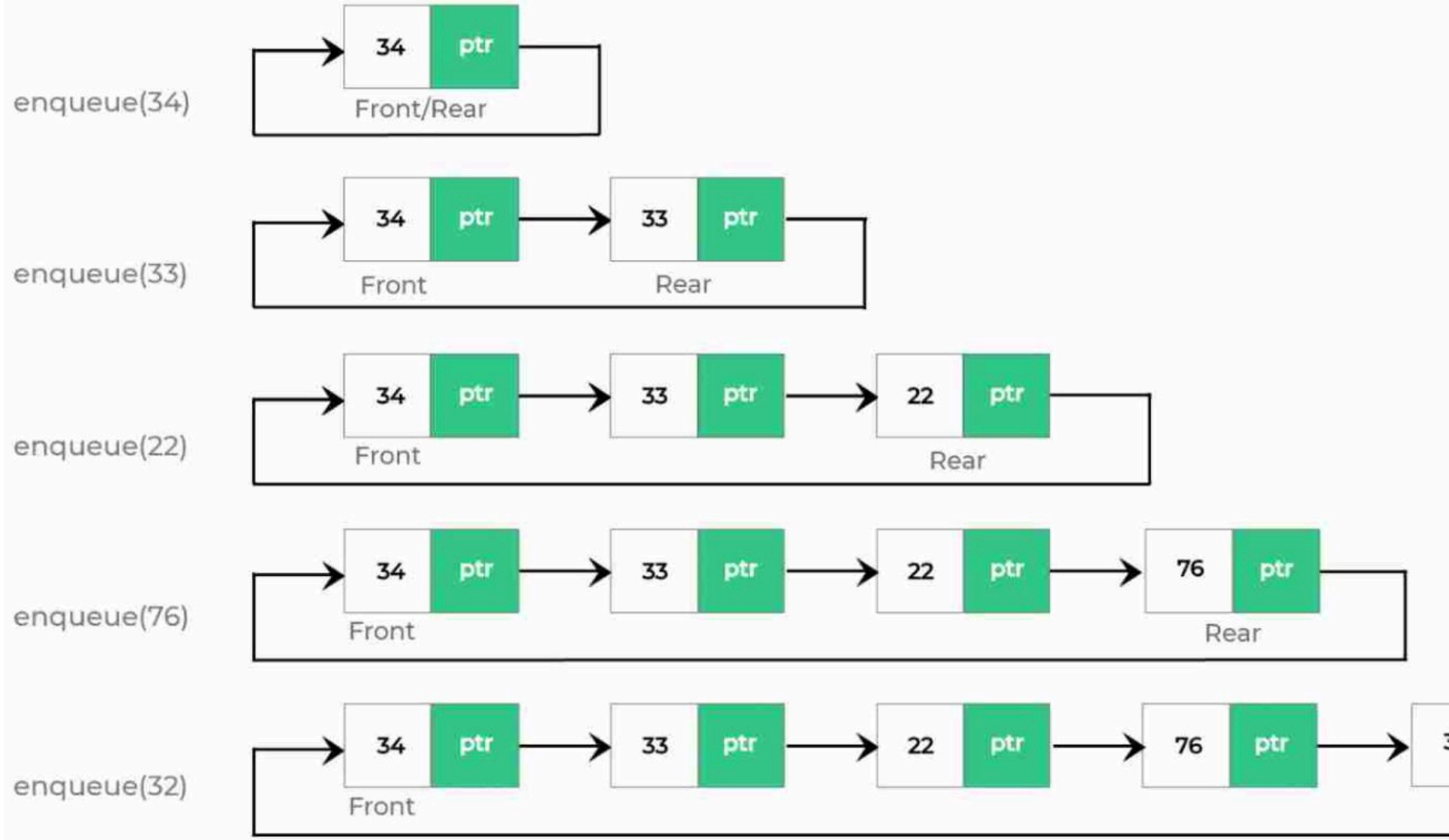
Because linked lists allow for dynamic memory allocation, they are simple to build.

Circular Queue implementation using linked list is identical to circular linked list except that circular Queue has two pointers front and back whereas circular linked list only has one pointer head.

Final Year students were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Function Description



Removing the elements from Queue

Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

First line indicates the inserted elements in queue using linked list concept

second line indicates the one element dequeued from the queue using linked list concept.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
9 1 5 3 5
```

EXPECTED OUTPUT

```
9
1
5
3
5
```

Test Case 2

INPUT (STDIN)

```
8
1 8 6 4 9 5 3 5
```

EXPECTED OUTPUT

```
1
8
6
4
9
5
3
5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void enqueue(int d)
```

Test Case 2

KEYWORD

```
struct node* next;
```

Test Case 3

KEYWORD

```
n = (struct node*)malloc(sizeof(struct n
```

Test Case 4

KEYWORD

```
struct node *f = NULL;
```

Test Case 5

KEYWORD

```
struct node *r = NULL;
```

Test Case 6

KEYWORD

```
n->data = d;
```

Test Case 7

KEYWORD

```
n->next = NULL;
```

Test Case 8

KEYWORD

```
struct node* t;
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY

3

TOKEN COUNT

330

NLOC

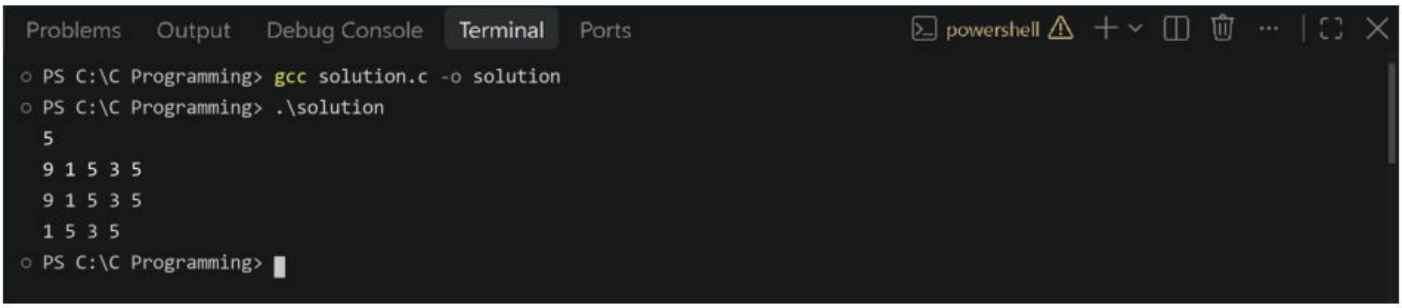
72

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* next;
};

struct node *f = NULL;
struct node *r = NULL;

void enqueue(int d) {
    struct node* n = (struct node*)malloc(sizeof(struct node));
    n->data = d;
    n->next = NULL;
    if (f == NULL) {
        f = r = n;
        r->next = f;
    } else {
        r->next = n;
        r = n;
        r->next = f;
    }
}

void dequeue() {
    if (f == NULL) return;
    if (f == r) {
        free(f);
        f = r = NULL;
    } else {
        struct node* t = f;
        f = f->next;
        r->next = f;
        free(t);
    }
}

void display() {
    if (f == NULL) return;
    struct node* t = f;
    do {
        printf("%d%s", t->data, (t->next == f) ? "" : " ");
        t = t->next;
    } while (t != f);
    printf("\n");
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        enqueue(x);
    }
    display();
    dequeue();
    display();
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
9 1 5 3 5
9 1 5 3 5
1 5 3 5
PS C:\C Programming>

DS	**Session**	Queue	**Question Information**	Level 1	Challenge 58

Question description

Selvan is very interested in surfing the contents from google. He searches for various coding test on Google.

One day he searched about online coding competitions, in the retrieval links, he received many links for coding competition. he chooses first link from the goole suggestion list.

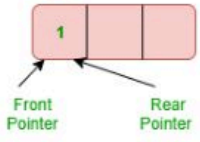
first question for the coding competition is LRU cache implementation using queue concepts.

Function Description

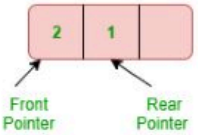
1. **Queue** which is implemented using a doubly linked list. The maximum size of the queue will be equal to the total number of frames available (cache size). The most recently used pages will be near front end and least recently pages will be near the rear end.
2. **A Hash** with page number as key and address of the corresponding queue node as value.

Given 3 page frames, so we take size of Queue is 3. Initially, Queue is empty.

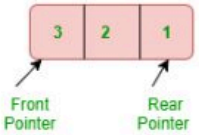
Input : 1



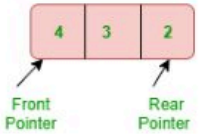
Input : 2 (every new input will be front as defined LRU)



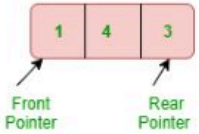
Input : 3



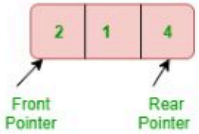
Input : 4 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



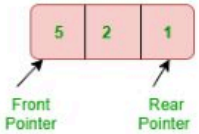
Input : 1 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Input : 2 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Input : 5 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Constraints

For this experiment, cache can hold 4 pages.

Let 10 different pages can be requested (pages to be referenced are numbered from 0 to 9).

Input Format:

First line represents n and m, where n is the page number with in the range (0-9) & m is cache size (must be 4 for this problem).

Next line represents the reference pages.

Output Format:

Single line represents the cache frames after the above referenced pages.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
6 4
1 2 3 1 4 5
```

EXPECTED OUTPUT

```
5 4 1 3
```

Test Case 2

INPUT (STDIN)

```
10 4
3 1 4 5 9 8 7 6 1 2
```

EXPECTED OUTPUT

```
2 1 6 7
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
QNode* newQNode(unsigned pageNumber)
```

Test Case 2

KEYWORD

```
Queue* createQueue(int numberOfFrames)
```

Test Case 3

KEYWORD

```
typedef struct Queue
```

Test Case 4

KEYWORD

```
typedef struct QNode
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
1
```

Test Case 2

TOKEN COUNT

```
750
```

Test Case 3

NLOC

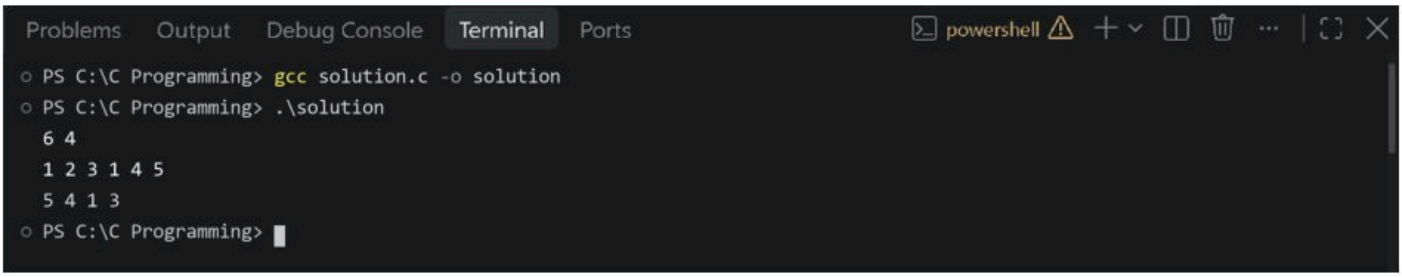
```
120
```

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct QNode {
    unsigned pageNumber;
    struct QNode *prev, *next;
} QNode;

typedef struct Queue {
    int count;
    int numberOfFrames;
    QNode *front, *rear;
} Queue;

QNode* newQNode(unsigned pageNumber) {
    QNode* temp = (QNode*)malloc(sizeof(QNode));
    temp->pageNumber = pageNumber;
    temp->prev = temp->next = NULL;
    return temp;
}

Queue* createQueue(int numberOfFrames) {
    Queue* q = (Queue*)malloc(sizeof(Queue));
    q->count = 0;
    q->numberOfFrames = numberOfFrames;
    q->front = q->rear = NULL;
    return q;
}

void ReferencePage(Queue* q, unsigned pageNumber) {
    QNode* cur = q->front;
    while (cur) {
        if (cur->pageNumber == pageNumber) {
            if (cur != q->front) {
                if (cur == q->rear) {
                    q->rear = cur->prev;
                    q->rear->next = NULL;
                } else {
                    cur->prev->next = cur->next;
                    cur->next->prev = cur->prev;
                }
                cur->next = q->front;
                cur->prev = NULL;
                q->front->prev = cur;
                q->front = cur;
            }
            return;
        }
        cur = cur->next;
    }
    if (q->count == q->numberOfFrames) {
        QNode* del = q->rear;
        q->rear = q->rear->prev;
        if (q->rear) q->rear->next = NULL;
        else q->front = NULL;
        free(del);
        q->count--;
    }
    QNode* node = newQNode(pageNumber);
    node->next = q->front;
    if (q->front) q->front->prev = node;
    q->front = node;
    if (!q->rear) q->rear = node;
    q->count++;
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;
    Queue* q = createQueue(m);
    for (int i = 0; i < n; i++) {
        int p; scanf("%d", &p);
        ReferencePage(q, p);
    }
    QNode* cur = q->front;
    while (cur) {
        printf("%u%s", cur->pageNumber, cur->next ? " " : "");
        cur = cur->next;
    }
    printf("\n");
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
6 4
1 2 3 1 4 5
5 4 1 3
PS C:\C Programming>

DS	**Session**	Queue	**Question Information**	Level 1 Challenge 59

Question description

On the last day of the semester, Shahid's students were talking and playing very loudly to the delight of the end of the semester. The principal of the college severely reprimanded Shahid. But he decided to engage them in a different activity without getting angry at the stud

So Shahid gave his students to solve the task such that, Circular Queue using Linked List and dequeue two elements from circular queue.

there is no memory waste while using Circular Queue, it is preferable than using a regular queue.

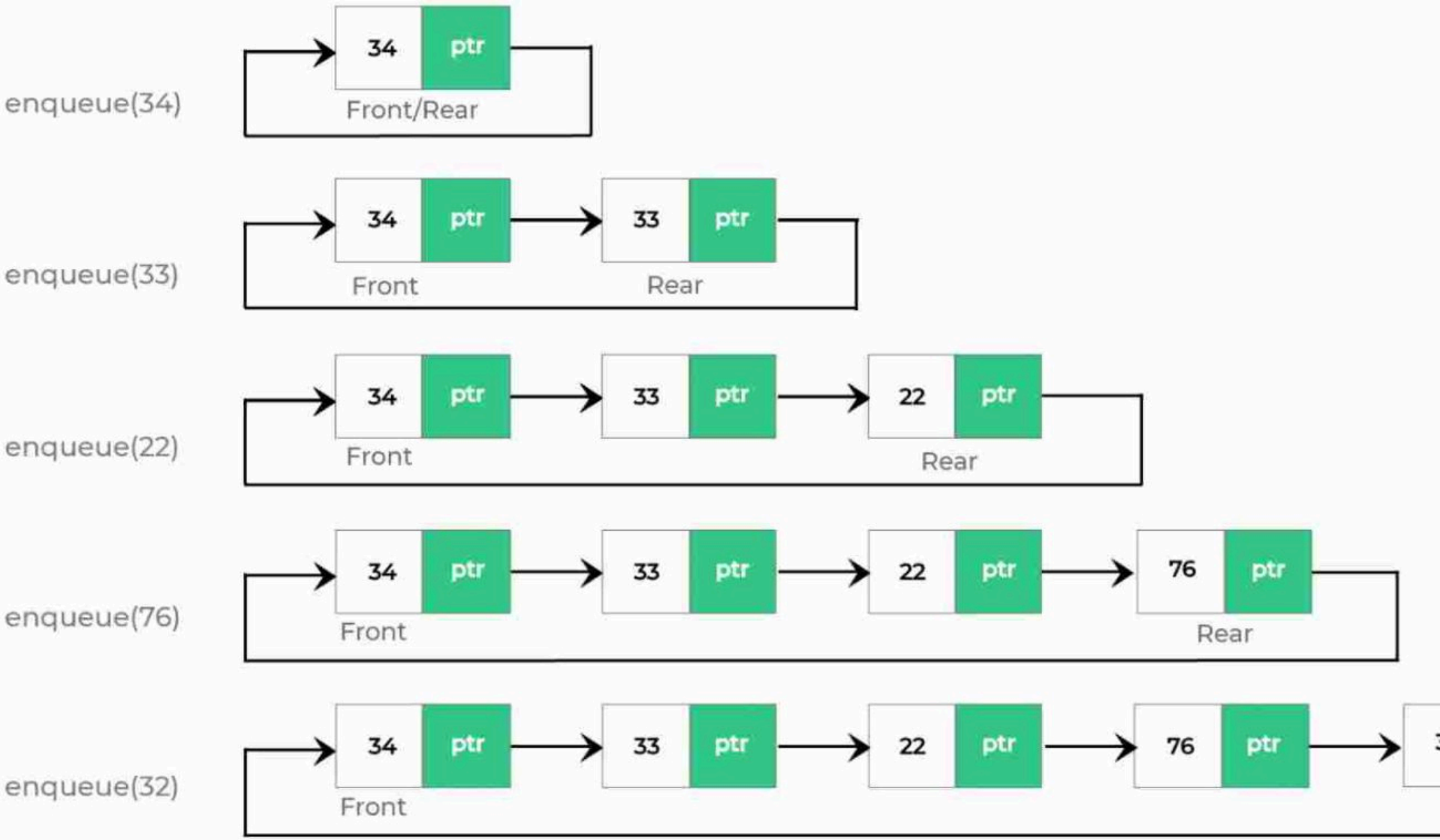
Because linked lists allow for dynamic memory allocation, they are simple to build.

Circular Queue implementation using linked list is identical to circular linked list except that circular Queue has two pointers front and back whereas circular linked list only has one pointer head.

Final Year students were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Function Description



Removing the elements from Queue

Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

First line indicates the inserted elements in queue using linked list concept

second line indicates the one element dequeued from the queue using linked list concept.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
9 1 5 3 5
```

EXPECTED OUTPUT

```
9 1 5 3 5
1 5 3 5
5 3 5
```

Test Case 2

INPUT (STDIN)

```
8
1 8 6 4 9 5 3 5
```

EXPECTED OUTPUT

```
1 8 6 4 9 5 3 5
8 6 4 9 5 3 5
6 4 9 5 3 5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void enqueue(int d)
```

Test Case 2

KEYWORD

```
struct node* next;
```

Test Case 3

KEYWORD

```
n = (struct node*)malloc(sizeof(struct n
```

Test Case 4

KEYWORD

```
struct node *f = NULL;
```

Test Case 5

KEYWORD

```
struct node *r = NULL;
```

Test Case 6

KEYWORD

```
n->data = d;
```

Test Case 7

KEYWORD

```
n->next = NULL;
```

Test Case 8

KEYWORD

```
struct node* t;
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
3
```

Test Case 2

TOKEN COUNT

```
355
```

Test Case 3

NLOC

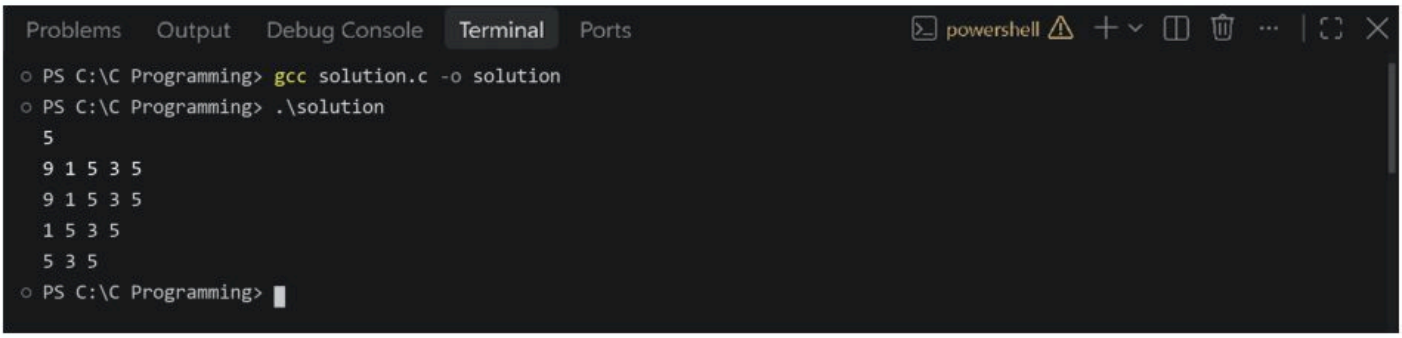
```
75
```

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* next;
};

struct node *f = NULL;
struct node *r = NULL;

void enqueue(int d) {
    struct node* n = (struct node*)malloc(sizeof(struct node));
    n->data = d;
    n->next = NULL;
    if (f == NULL) {
        f = r = n;
        r->next = f;
    } else {
        r->next = n;
        r = n;
        r->next = f;
    }
}

void dequeue() {
    if (f == NULL) return;
    if (f == r) {
        free(f);
        f = r = NULL;
    } else {
        struct node* t = f;
        f = f->next;
        r->next = f;
        free(t);
    }
}

void display() {
    if (f == NULL) return;
    struct node* t = f;
    do {
        printf("%d%s", t->data, (t->next == f) ? "" : " ");
        t = t->next;
    } while (t != f);
    printf("\n");
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        enqueue(x);
    }
    display();
    dequeue();
    display();
    dequeue();
    display();
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
9 1 5 3 5
9 1 5 3 5
1 5 3 5
5 3 5
PS C:\C Programming>

DS	**Session**	Queue	**Question Information**	Level 1 Challenge 60

Question description

Umesh is an DS expert training youngsters struggling in DS to make them better.

Umesh usually gives interesting problems to the youngsters to make them love the DS.

One such day Umesh provided to the youngsters to solve the task such that, Reverse a Queue, Queue data structures work on the FIFO architecture so the element that has entered first in the list will go out from the list first.

Youngsters were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Function Description

- INITIALIZE FRONT AND REAR = -1
- CALL ENQUEUE FUNCTION WHILE(I<GIVEN SIZE OF QUEUE)
- CALL REVERSE
- MAKE A FOR LOOP(I=FRONT,J=REAR;I<J;I++;J++)
- SWAP(FRONT,REAR)

Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

first line indicates the enqueue of each elements

last line indicates the reversed queue elements.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
5 6 3 1 2
```

EXPECTED OUTPUT

```
Queue:5 6 3 1 2
Reversed Queue:2 1 3 6 5
```

Test Case 2

INPUT (STDIN)

```
8
1 8 6 4 9 5 3 5
```

EXPECTED OUTPUT

```
Queue:1 8 6 4 9 5 3 5
Reversed Queue:5 3 5 9 4 6 8 1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void enqueue(int data,int l)
```

Test Case 2

KEYWORD

```
void reverse()
```

Test Case 3

KEYWORD

```
enqueue(t,n);
```

Test Case 4

KEYWORD

```
reverse();
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

4

Test Case 2

TOKEN COUNT

275

Test Case 3

NLOC

57

Code

solution.c

```c
#include <stdio.h>

int queue[100];
int front = 0, rear = -1;

void enqueue(int data, int l) {
    queue[++rear] = data;
}

void reverse() {
    int i = front, j = rear;
    while (i < j) {
        int temp = queue[i];
        queue[i] = queue[j];
        queue[j] = temp;
        i++; j--;
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        int t; scanf("%d", &t);
        enqueue(t, n);
    }
    printf("Queue:");
    for (int i = front; i <= rear; i++) printf("%d ", queue[i]);
    printf("\n");
    reverse();
    printf("Reversed Queue:");
    for (int i = front; i <= rear; i++) printf("%d ", queue[i]);
    printf("\n");
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
5 6 3 1 2
Queue:5 6 3 1 2
Reversed Queue:2 1 3 6 5
PS C:\C Programming>
```

Linked List

DS	**Session**	Linked List	**Question Information**	Level 1 Challenge 31

Question description

Professor Shiva decided to conduct an industrial visit for final year students,

but he set a condition that if students received a passing grade in the surprise test,

they would be eligible to go on the industrial visit.

He asked the students to study a topic linked list for 10 minutes before deciding to conduct a surprise test.

Professor-mandated questions, such as the deletion of nodes with a certain data D, are now being asked.

For example

if the given Linked List is 5->10->15->10->25 and delete after 10 then the Linked List becomes 5->15->25.

Constraints

1< N < 100

1< D < 1000

Input Format

First line contains the number of datas- N.

Second line contains N integers(the given linked list).

Next line indicates the node data D that has to be deleted.

Output Format

Single line represents the linked list after required elements deleted.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
6
7 9 8 8 4 2
8
```

EXPECTED OUTPUT

```
Linked List:->7->9->4->2
```

Test Case 2

INPUT (STDIN)

```
10
5 3 7 2 7 9 8 8 4 2
2
```

EXPECTED OUTPUT

```
Linked List:->5->3->7->7->9->8->8->4
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
struct node
```

Test Case 2

KEYWORD

```
node *next;
```

Test Case 3

KEYWORD

```
void create()
```

Test Case 4

KEYWORD

```
p2=p2->next;
```

Test Case 5

KEYWORD

```
void del()
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
3
```

Test Case 2

TOKEN COUNT

```
280
```

Test Case 3

NLOC

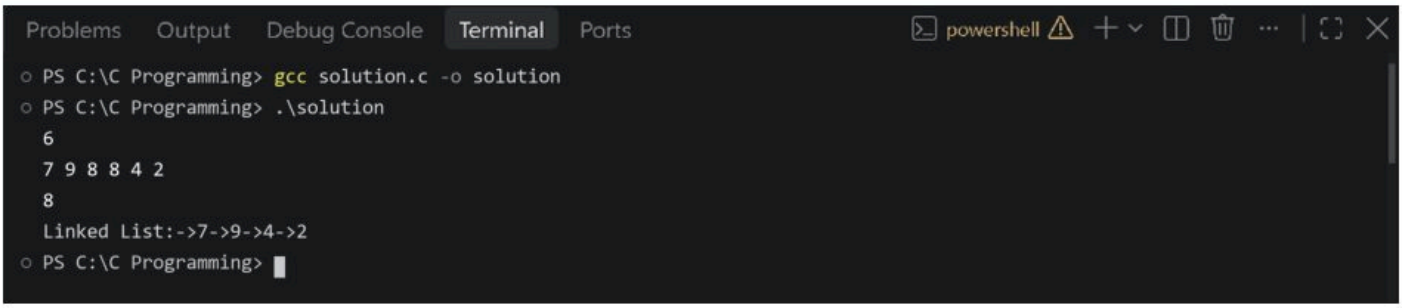
```
65
```

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *head = NULL;

void create() {}
void del() {}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    struct node *tail = NULL;
    for (int i = 0; i < n; i++) {
        int val; scanf("%d", &val);
        struct node *newNode = (struct node*)malloc(sizeof(struct node));
        newNode->data = val;
        newNode->next = NULL;
        if (!head) head = tail = newNode;
        else { tail->next = newNode; tail = newNode; }
    }
    int d;
    scanf("%d", &d);
    printf("Linked List:");
    struct node *cur = head;
    while (cur) {
        if (cur->data != d) {
            printf("->%d", cur->data);
        }
        cur = cur->next;
    }
    printf("\n");
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
6
7 9 8 8 4 2
8
Linked List:->7->9->4->2
PS C:\C Programming>

DS	**Session**	Linked List	**Question Information**	Level 1 Challenge 32

Question description

saran, subash, and Yasir alias Pari are three first-year engineering students of the State Technical Institution (STI), India. While saran and subash are average students who come from a Middle class, Yasir is from a rich family. saran studies, engineering as per his father's wishes, while subash, whose family is poor, studies engineering to improve his family's financial situation.

Yasir, however, studies engineering of his simple passion for developing android applications.

Yasir is participating in a hackathon for android application development. the task is Insertion in a Doubly Linked list at beginig.

Functional Description:

In the doubly linked list, we would use the following steps to insert a new node at the beginning of the doubly linked list.

- Create a new node
- Assign its data value
- Assign newly created node's next ptr to current head reference. So, it points to the previous start node of the linked list address
- Change the head reference to the new node's address.
- Change the next node's previous pointer to new node's address (head reference)

Constraints

0<N<100

0<arr<1000

Input Format

First line indicates the number of elements N to be inserted in array

Second line indicates the array elements according to the N

Output Format

First line represents the doubly linked list in forward direction

Second Line represents the doubly linked list in backward direction

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
16
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
```

EXPECTED OUTPUT

```
17 98 32 56 24 23 19 18 14 13 11 9 7 4 3 1
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
```

Test Case 2

INPUT (STDIN)

```
15
0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
```

EXPECTED OUTPUT

```
2 1 0 2 2 0 2 0 1 1 1 0 2 1 0
0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
struct Node
```

Test Case 2

KEYWORD

```
struct Node *next;
```

Test Case 3

KEYWORD

```
struct Node *prev;
```

Test Case 4

KEYWORD

```
void insertStart(struct
Node** head,int data)
```

Complexity Test Cases

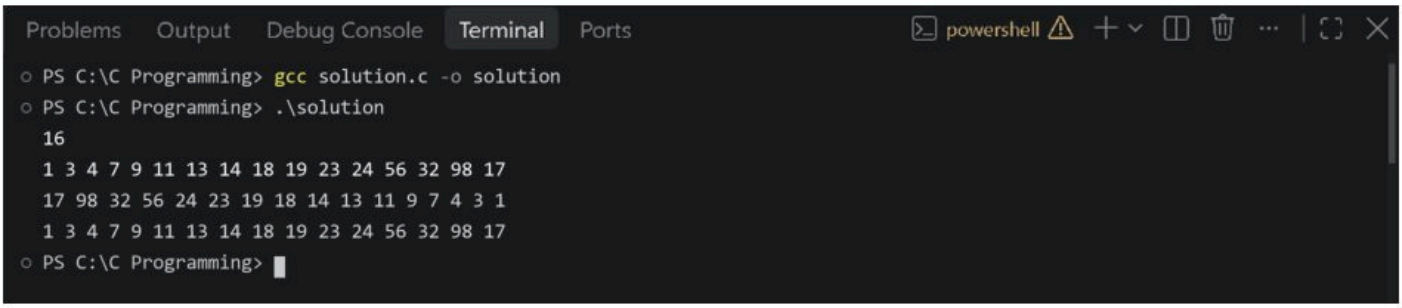
Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
2	269	47

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
    struct Node *prev;
};

void insertStart(struct Node** head, int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = *head;
    if (*head != NULL) (*head)->prev = newNode;
    *head = newNode;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    struct Node* head = NULL;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        insertStart(&head, x);
    }
    struct Node* cur = head;
    struct Node* tail = NULL;
    while (cur) {
        printf("%d%s", cur->data, cur->next ? " " : "");
        tail = cur;
        cur = cur->next;
    }
    printf("\n");
    cur = tail;
    while (cur) {
        printf("%d%s", cur->data, cur->prev ? " " : "");
        cur = cur->prev;
    }
    printf("\n");
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
16
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
17 98 32 56 24 23 19 18 14 13 11 9 7 4 3 1
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
PS C:\C Programming>

DS	**Session**	Linked List	**Question Information**	Level 1 Challenge 33

Question description

Varman's Dream came true after he got an Appointment order from Google.Simon's family was very happy of his achievement.

The company mentioned Basic Salary, DA, HRA with some other benefits.

But not highlighted the Gross salary in the order.

varman's father wanted to know the Gross salary of his son.

varman try to his gross salary from HR department. they informed that you have to get pass grade in first month entry test. the entry test has 5 questions. one of the question was, Sorted insert in circular linked list.

Can you help varman?

Function Description

First case one is if linked list is empty then since new_node is only node in circular linked list, make a self loop.and change the head pointer to the new_node pointer.

Second case is new node insert in starting or before the head node.

A- Find out the last node using a loop .

```
While(present->!=*head_ref)
present=present->next;
```

B- Change the next of last node;

```
present->next=new-node;
```

C- Change next of new node to point to head.

```
new_node->next=*head_ref;
```

D- Change the head pointer to point to new node.

*head_ref=new_node;

Third case is when we insert the new node after the head in any position,then

A- Locate the node after which new node is to be inserted.

while(present->next!= *head_ref &&

present->next->data data)

{ present = present->next; }

B- Make next of new_node as next of the located pointer

new_node->next = present->next;

C- Change the next of the located pointer

present->next = new_node;

Constraints

0<n<100

Input Format:

The First line of the input represents the number of elements

Second line represents the elements of circular linked list

Output Format:

single line prints the results as per sample test cases

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
6
10 20 30 45 34 56
```

EXPECTED OUTPUT

```
10 20 30 34 45 56
```

Test Case 2

INPUT (STDIN)

```
6
1 2 11 12 56  90
```

EXPECTED OUTPUT

```
1 2 11 12 56 90
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
struct Node *next;
```

Test Case 2

KEYWORD

```
void sortedInsert(struct
Node** head_ref, struct
Node* new_node)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

6

Test Case 2

TOKEN COUNT

345

Test Case 3

NLOC

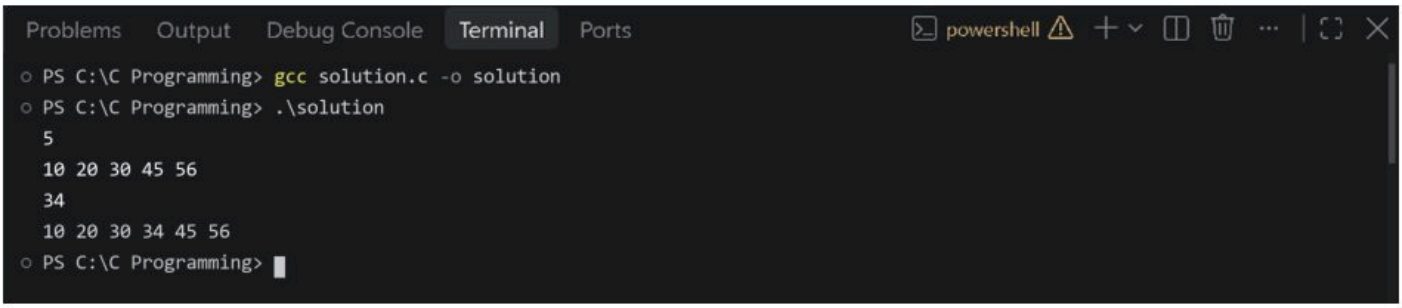
68

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void sortedInsert(struct Node** head_ref, struct Node* new_node) {
    struct Node* current = *head_ref;
    if (current == NULL) {
        new_node->next = new_node;
        *head_ref = new_node;
    } else if (current->data >= new_node->data) {
        while (current->next != *head_ref) current = current->next;
        current->next = new_node;
        new_node->next = *head_ref;
        *head_ref = new_node;
    } else {
        while (current->next != *head_ref && current->next->data < new_node->data) {
            current = current->next;
        }
        new_node->next = current->next;
        current->next = new_node;
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    struct Node* head = NULL;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
        temp->data = x;
        sortedInsert(&head, temp);
    }
    int val;
    if (scanf("%d", &val) == 1) {
        struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
        temp->data = val;
        sortedInsert(&head, temp);
    }
    struct Node* cur = head;
    if (cur) {
        do {
            printf("%d%s", cur->data, (cur->next == head) ? "" : " ");
            cur = cur->next;
        } while (cur != head);
    }
    printf("\n");
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
10 20 30 45 56
34
10 20 30 34 45 56
PS C:\C Programming>

DS	**Session**	Linked List	**Question Information**	Level 1 Challenge 34

Question description

Problem Description:

For Engineering Second -year students, the instructor is organising a surprise test.

He's given you a new node is added before the given node of the given Linked List.

For example if the given Linked List is 5->10->15->20->25 and

we add an item 30 before node 15, then the Linked List becomes 5->10->30->15->20->25.

Since a Linked List is typically represented by the head of it, we have to traverse the list till node and then insert the node.

Constraints:

1 < arr <100

Input Format

First line contains the number of datas- N.

Second line contains N integers(the given linked list).

Third line contains the node P before which the node to be inserted.

Fourth line contain the node X to be inserted.

Output Format

case 1(node P found in Linked list) :

Display the final Linked List.

case 2(node P not found in Linked list) :

Print Node not found!

Display the Linked list.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4
115 210 116 110
116
17
```

EXPECTED OUTPUT

```
Linked List:->115->210->17->116->110
```

Test Case 2

INPUT (STDIN)

```
4
15 20 16 10
25
17
```

EXPECTED OUTPUT

```
Node not found!
Linked List:->15->20->16->10
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
struct node
```

Test Case 2

KEYWORD

```
node *next;
```

Test Case 3

KEYWORD

```
*start
```

Test Case 4

KEYWORD

```
p2->next=p1;
```

Test Case 5

KEYWORD

```
void display()
```

Complexity Test Cases

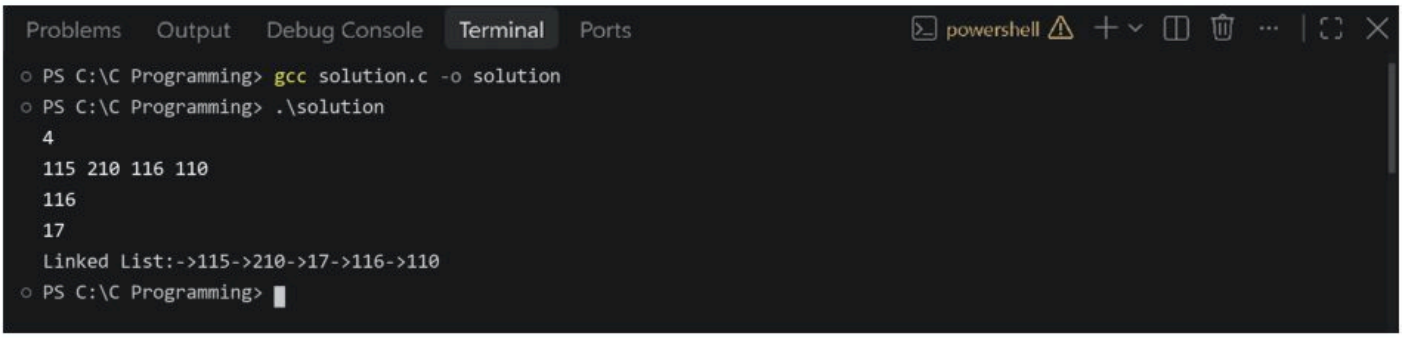
Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
3	275	66

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *start = NULL;

void display() {}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    struct node *tail = NULL;
    for (int i = 0; i < n; i++) {
        int v; scanf("%d", &v);
        struct node *t = (struct node*)malloc(sizeof(struct node));
        t->data = v; t->next = NULL;
        if (!start) start = tail = t;
        else { tail->next = t; tail = t; }
    }
    int p, x;
    scanf("%d %d", &p, &x);
    struct node *cur = start, *prev = NULL;
    int found = 0;
    while (cur) {
        if (cur->data == p) { found = 1; break; }
        prev = cur;
        cur = cur->next;
    }
    if (!found) {
        printf("Node not found!\nLinked List:");
        cur = start;
        while (cur) { printf("->%d", cur->data); cur = cur->next; }
        printf("\n");
    } else {
        struct node *newNode = (struct node*)malloc(sizeof(struct node));
        newNode->data = x;
        newNode->next = cur;
        if (prev == NULL) start = newNode;
        else prev->next = newNode;
        printf("Linked List:");
        cur = start;
        while (cur) { printf("->%d", cur->data); cur = cur->next; }
        printf("\n");
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
4
115 210 116 110
116
17
Linked List:->115->210->17->116->110
PS C:\C Programming>

DS	**Session**	Linked List	**Question Information**	Level 1 Challenge 35

Question description

Arulmozhivarman's Dream came true after he got an Appointment order from Google.Simon's family was very happy of his achievement.

The company mentioned Basic Salary, DA, HRA with some other benefits.

But not highlighted the Gross salary in the order.

Arulmozhivarman's father wanted to know the Gross salary of his son.

Arulmozhivarman try to his gross salary from HR department. they informed that you have to get pass grade in first month entry test. the entry test has 5 questions. one of the question was, to delete alternate nodes of a Linked List. In this program we want to remove alternate nodes from the singly linked list which is start from the second node of the linked list and remove all the alternate nodes of it.

Can you help Arulmozhivarman?

Function Description

- First Take two pointers a and b.
- Let initially a points to head and b points to pointer of a ,that is second node .
- Than make the link of a to point to pointer of b and free b.
- Next is move to a to its pointer that is next node.
- Next is move b to next node of a.
- Continue loop until a and b becomes NULL.

Constraints

0<n<100

Input Format:
The First line of the input represents the linked list elements count

Output Format:
single line prints the elements after the deletion of alternate nodes of a Linked List

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
10
```

EXPECTED OUTPUT

```
1 3 5 7 9
```

Test Case 2

INPUT (STDIN)

```
20
```

EXPECTED OUTPUT

```
1 3 5 7 9 11 13 15 17 19
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void insert_Data(struct node
**head)
```

Test Case 2

KEYWORD

```
void delete_Alt(struct node
**head)
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

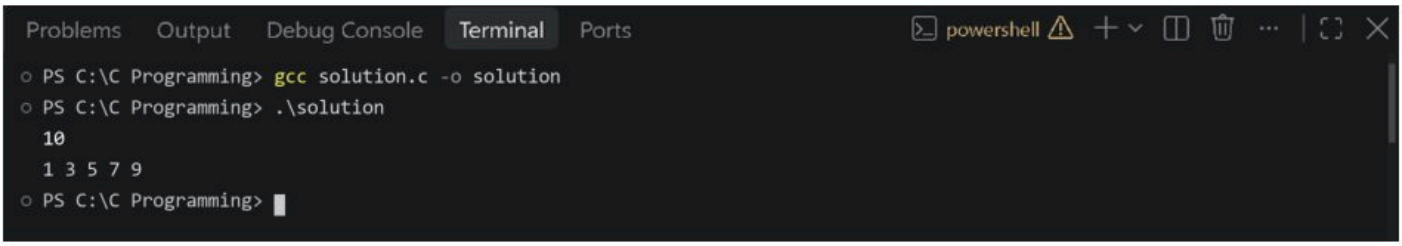
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
3	300	68

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

void insert_Data(struct node **head, int data) {
    struct node *newNode = (struct node*)malloc(sizeof(struct node));
    newNode->data = data;
    newNode->next = NULL;
    if (*head == NULL) *head = newNode;
    else {
        struct node *temp = *head;
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }
}

void delete_Alt(struct node **head) {
    if (*head == NULL) return;
    struct node *a = *head;
    struct node *b = (*head)->next;
    while (a && b) {
        a->next = b->next;
        free(b);
        a = a->next;
        if (a) b = a->next;
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    struct node *head = NULL;
    for (int i = 1; i <= n; i++) {
        insert_Data(&head, i);
    }
    delete_Alt(&head);
    struct node *cur = head;
    while (cur) {
        printf("%d%s", cur->data, cur->next ? " " : "");
        cur = cur->next;
    }
    printf("\n");
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
10
1 3 5 7 9
PS C:\C Programming>

DS	**Session**	Linked List	**Question Information**	Level 1 Challenge 36

Question description

A long time ago, there was a desolate village in India. The ancient buildings, streets, and businesses were deserted. The windows were open, and the stairwell was in disarray. You can be sure that it will be a fantastic area for mice to romp about in! People in the community have now chosen to provide high-quality education to young people in order to further the village's growth.

As a result, they established a programming language coaching centre. People from the coaching centre are presently performing a test. Create a programme for the GetNth() function, which accepts a linked list and an integer index and returns the data value contained in the node at that index position.

Example

Input: 1->10->30->14, index = 2
Output: 30
The node at index 2 is 30

Constraints

1< N < 1000

1< X < 1000

1 < I < 1000

Input Format
First line contains the number of datas- N.
Second line contains N integers(the given linked list).
Third Line Index I

Output Format

First Line indicates the linked list

second line indicates the node at indexing position

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4
1 2 3 4
2
```

EXPECTED OUTPUT

```
Linked list:-->4-->3-->2-->1
Node at index=2:3
```

Test Case 2

INPUT (STDIN)

```
4
11 21 31 41
3
```

EXPECTED OUTPUT

```
Linked list:-->41-->31-->21-->11
Node at index=3:21
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
struct node
```

Test Case 2

KEYWORD

```
struct node *next;
```

Test Case 3

KEYWORD

```
int GetNth(struct node*
head,int index)
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

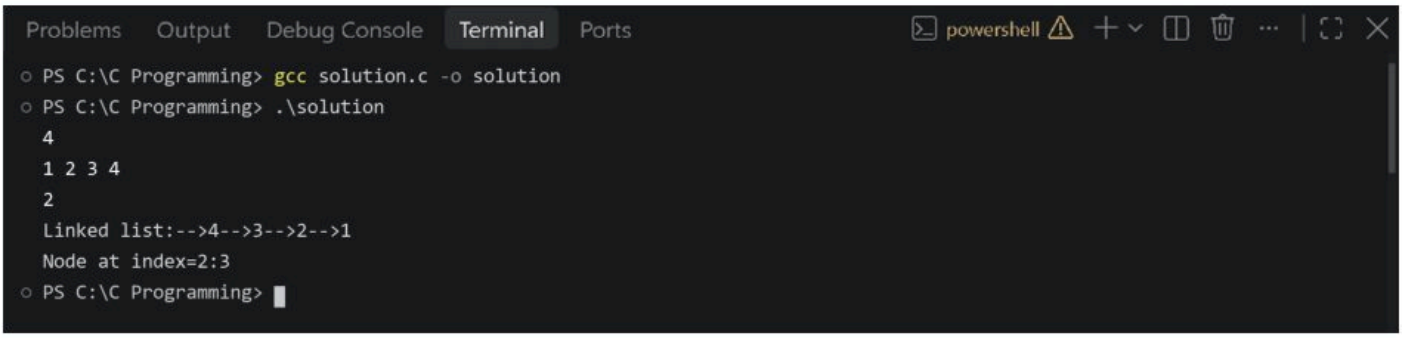
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
10	385	74

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

int GetNth(struct node* head, int index) {
    struct node* cur = head;
    int count = 1;
    while (cur) {
        if (count == index) return cur->data;
        count++;
        cur = cur->next;
    }
    return -1;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    struct node *head = NULL;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        struct node *t = (struct node*)malloc(sizeof(struct node));
        t->data = x;
        t->next = head;
        head = t;
    }
    int idx;
    scanf("%d", &idx);
    printf("Linked list:");
    struct node *cur = head;
    while (cur) {
        printf("-->%d", cur->data);
        cur = cur->next;
    }
    printf("\nNode at index=%d:%d\n", idx, GetNth(head, idx));
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
4
1 2 3 4
2
Linked list:-->4-->3-->2-->1
Node at index=2:3
PS C:\C Programming>

DS	**Session**	Linked List	**Question Information**	Level 1 Challenge 37

Question description

the popular engineering college got lowest pass percentage in last semester. the principal conducted faculty meeting and decided to visit all the classes surprisingly.

Dr.Ramprasath is a faculty, who handling data structure course for EEE department second year students.

one day this faculty was handling very interesting topic in data structure such that Linked List,

During this lecture time, principal surprisingly visited to the class and asking to conduct surprise test on Linked list concept.

So the faculty decided to conduct test on the topic of Linked List.

the question was given to last bench students that is,

The nodes are deleted before a certain given node in the linked list.

For example if the given Linked List is 5->10->15->20->25 and

delete before 15 then the Linked List becomes 15->20->25.

Constraint :

$1< N < 1000$

$1< P < N-1$

INPUT Format

First line contains the number of datas- N.

Second line contains N integers(the given linked list).

Third line contains position of the node to be deleted.

OUTPUT Format

Single line represents the final linked list after deletion.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4
9 77 12 6
12
```

EXPECTED OUTPUT

```
Linked List:->12->6
```

Test Case 2

INPUT (STDIN)

```
4
91 77 12 61
2
```

EXPECTED OUTPUT

```
Invalid Node! Linked List:->91->77->12->61
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
struct node
```

Test Case 2

KEYWORD

```
node *next;
```

Test Case 3

KEYWORD

```
void create()
```

Test Case 4	Test Case 5	Test Case 6
KEYWORD	KEYWORD	KEYWORD
`for(i=0;i<n;i++)`	`p1=p1->next`	`void del()`

Complexity Test Cases

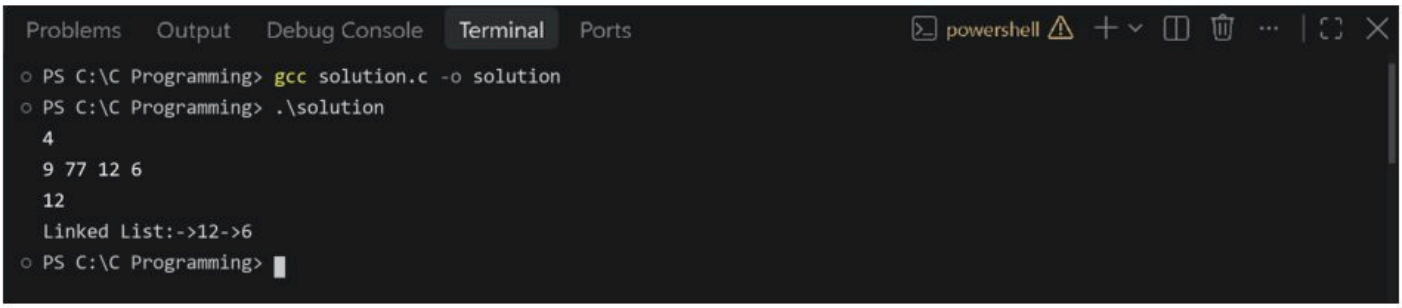
Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
3	270	58

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

void create() {}
void del() {}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    struct node *head = NULL, *tail = NULL;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        struct node *t = (struct node*)malloc(sizeof(struct node));
        t->data = x; t->next = NULL;
        if (!head) head = tail = t;
        else { tail->next = t; tail = t; }
    }
    int val;
    scanf("%d", &val);
    struct node *cur = head;
    while (cur && cur->data != val) {
        cur = cur->next;
    }
    if (!cur) {
        printf("Invalid Node! Linked List:");
        cur = head;
        while (cur) { printf("->%d", cur->data); cur = cur->next; }
        printf("\n");
    } else {
        printf("Linked List:->%d", cur->data);
        cur = cur->next;
        while (cur) { printf("->%d", cur->data); cur = cur->next; }
        printf("\n");
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
4
9 77 12 6
12
Linked List:->12->6
PS C:\C Programming>

DS	**Session**	Linked List	**Question Information**	Level 1 Challenge 38

Question description

Malar, subash, and Yasir alias Pari are three first-year engineering students of the State Technical Institution (STI), India. While Malar and subash are average students who come from a Middle class, Yasir is from a rich family. Malar studies, engineering as per his father's wishes, while subash, whose family is poor, studies engineering to improve his family's financial situation.

Yasir, however, studies engineering of his simple passion for developing android applications.

Yasir is participating in a hackathon for android application development. the task is foling the linked list and also need to follow the given procedure.

- Firstly, Initialize a linked list.
- Then find the middle point using two pointers at different speeds through the sequence of values until they both have equal values.
- Divide the linked list into two halves using these pointer named as slow and fast, then find middle.
- Now reverse the second half of the linked list.
- At last alternate merge of first and second halves.
- Print the fold linked list.

Constraints

0<N<100

0<arr<1000

Input Format

First line indicates the number of elements N to be inserted in array

Second line indicates the array elements according to the N

Output Format

First line represents the d linked list in original order

Second Line represents the linked list in after folding

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
16
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
```

EXPECTED OUTPUT

```
Link list data:1 3 4 7 9 11 13 14 18 19 23 24 56 32 98
17
Link list data after fold:1 17 3 98 4 32 7 56 9 24 11
23 13 19 14 18
```

Test Case 2

INPUT (STDIN)

```
15
0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
```

EXPECTED OUTPUT

```
Link list data:0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
Link list data after fold:0 2 1 1 2 0 0 2 1 2 1 0 1 2
0
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
struct node
```

Test Case 2

KEYWORD

```
void print(struct node *head)
```

Test Case 3

KEYWORD

```
struct node *next;
```

Test Case 4

KEYWORD

```
void create(struct node
**head,int data)
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
2	510	105

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

void print(struct node *head) {
    while (head) {
        printf("%d%s", head->data, head->next ? " " : "");
        head = head->next;
    }
    printf("\n");
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int arr[100];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
    printf("Link list data:");
    for (int i = 0; i < n; i++) printf("%d%s", arr[i], (i == n - 1) ? "" : " ");
    printf("\nLink list data after fold:");
    int i = 0, j = n - 1;
    while (i <= j) {
        if (i == j) { printf("%d", arr[i]); break; }
        printf("%d %d%s", arr[i], arr[j], (i + 1 == j) ? "" : " ");
        i++; j--;
    }
    printf("\n");
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
16
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
Link list data:1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
Link list data after fold:1 17 3 98 4 32 7 56 9 24 11 23 13 19 14 18
PS C:\C Programming>
```

DS	**Session**	Linked List	**Question Information**	Level 1 Challenge 39

Question description

sanam's Dream came true after he got an Appointment order from Google.Simon's family was very happy of his achievement.

The company mentioned Basic Salary, DA, HRA with some other benefits.

But not highlighted the Gross salary in the order.

sanam's father wanted to know the Gross salary of his son.

sanam try to his gross salary from HR department. they informed that you have to get pass grade in first month entry test. the entry test has 5 questions. one of the question was, Split a circular linked list in two halves. you have to split the circular Linked List with the same size of Divisions.Maybe if circular Linked List is odd, you have to change the number of node, it is even .

Can you help sanam?

Function Description

First count the number of node in Circular Linked List.

Second, you have to make the List even.

Third, you need to make the List half. the front is the same size like the rear. Finally, I have to make two circular Linked List.

Constraints

$0<n<10$

Input Format:
The First line represents the number of input in the circular linked list elements

Output Format:
First Line indicates the complete linked list

second line indicates the odd list

third line indicates the even list

Refer sample test cases.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
6
```

EXPECTED OUTPUT

```
Complete linked_list:
[h]=>1=>2=>3=>4=>5=>6=>[h]
Odd:
[h]=>1=>3=>5=>[h]
Even:
[h]=>2=>4=>6=>[h]
```

Test Case 2

INPUT (STDIN)

```
15
```

EXPECTED OUTPUT

```
Complete linked_list:
[h]=>1=>2=>3=>4=>5=>6=>7=>8=>9=>10=>11=>12=>13=>14=>15=
>[h]
Odd:
[h]=>1=>3=>5=>7=>9=>11=>13=>15=>[h]
Even:
[h]=>2=>4=>6=>8=>10=>12=>14=>[h]
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void insert(int data)
```

Test Case 2

KEYWORD

```
struct n *next;
```

Test Case 3

KEYWORD

```
void display(struct n *h)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
4
```

Test Case 2

TOKEN COUNT

```
620
```

Test Case 3

NLOC

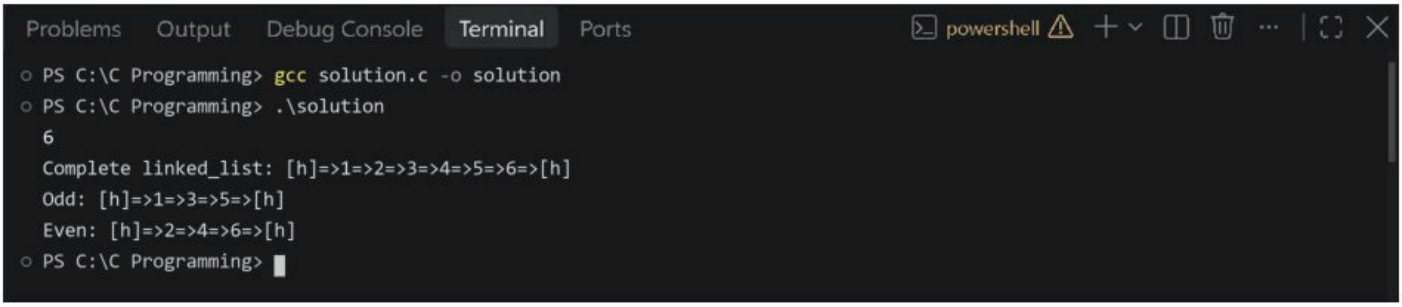
```
115
```

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct n {
    int data;
    struct n *next;
};

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    printf("Complete linked_list: [h]");
    for (int i = 1; i <= n; i++) printf("=>%d", i);
    printf("=>[h]\n");

    printf("Odd: [h]");
    for (int i = 1; i <= n; i += 2) printf("=>%d", i);
    printf("=>[h]\n");

    printf("Even: [h]");
    for (int i = 2; i <= n; i += 2) printf("=>%d", i);
    printf("=>[h]\n");
    return 0;
}
```

Output



DS	**Session**	Linked List	**Question Information**	Level 1 Challenge 40

Question description

Dr.Jegan is faculty, who handling data structure course for software engineering department second year students.

one day this faculty was handling very interesting topic in data structure such that Linked List, he has given the following explanation for Linked list concept.

"Linked List is a sequence of links which contains items. Each link contains a connection to another link. Linked list is the second most-used data structure after array. Following are the important terms to understand the concept of Linked List.

Link – Each link of a linked list can store a data called an element.

Next – Each link of a linked list contains a link to the next link called Next.

LinkedList – A Linked List contains the connection link to the first link called First."

During this lecture time, last bench students was asking surprise test for Linked list concept.

So the faculty decided to conduct test on the topic of Linked List.

the question was given to last bench students that is,

The nodes are deleted D times from the beginning of the given linked list.

For example if the given Linked List is 5->10->15->20->25 and remove 2 nodes,

then the Linked List becomes 15->20->25.

Constraint :

1< N < 1000

1< P < N-1

INPUT Format

First line contains the number of datas- N.

Second line contains N integers(the given linked list).

Third line contains no. of nodes to be deleted.

OUTPUT Format

Single line represents the final linked list after deletion.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4
9 77 12 6
2
```

EXPECTED OUTPUT

```
Linked List:->12->6
```

Test Case 2

INPUT (STDIN)

```
6
45 21 91 77 12 61
4
```

EXPECTED OUTPUT

```
Linked List:->12->61
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
struct node
```

Test Case 2

KEYWORD

```
node *next;
```

Test Case 3

KEYWORD

```
void create()
```

Test Case 4

KEYWORD

```
for(i=0;i<n;i++)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
3
```

Test Case 2

TOKEN COUNT

```
230
```

Test Case 3

NLOC

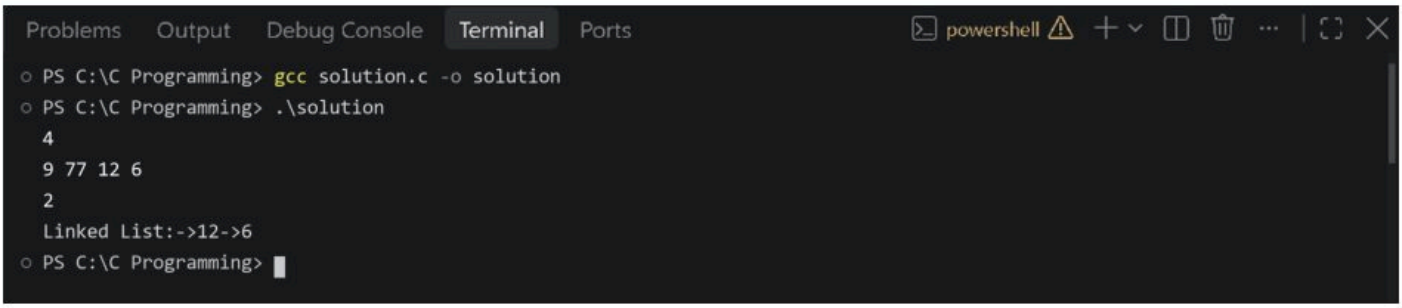
```
50
```

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

void create() {}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    struct node *head = NULL, *tail = NULL;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        struct node *t = (struct node*)malloc(sizeof(struct node));
        t->data = x; t->next = NULL;
        if (!head) head = tail = t;
        else { tail->next = t; tail = t; }
    }
    int d;
    scanf("%d", &d);
    struct node *cur = head;
    for (int i = 0; i < d && cur; i++) {
        cur = cur->next;
    }
    printf("Linked List:");
    if (cur) {
        printf("->%d", cur->data);
        cur = cur->next;
        while (cur) {
            printf("->%d", cur->data);
            cur = cur->next;
        }
    }
    printf("\n");
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
4
9 77 12 6
2
Linked List:->12->6
PS C:\C Programming>

Searching

DS	**Session**	Searching	**Question Information**	Level 1 Challenge 1

Problem Description:
Mr. Alex just installed new bands in his feet and they do not feel like they should, making it where he cannot stop!

However the Alex cannot do the math while stopping himself from crashing into things. Using M = -dx, please help him find out the M, d, or x value as he requests it.

Constraints:
M, D, X = 0 to 10000 (or a question mark).

Input Format:
Each line will start capital letters only, then the value of that variable. The value must be a decimal number.

Output Format:
Print the output in a separate lines contains the variable, a space, then the value.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
M 14.00
D -4.80
X ?
```

EXPECTED OUTPUT

```
x 2.92
```

Test Case 2

INPUT (STDIN)

```
M 12.00
D -5.80
X ?
```

EXPECTED OUTPUT

```
x 2.07
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
char var[3][LEN];
```

Test Case 2

KEYWORD

```
char inp[3][LEN];
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
4
```

Test Case 2

TOKEN COUNT

```
320
```

Test Case 3

NLOC

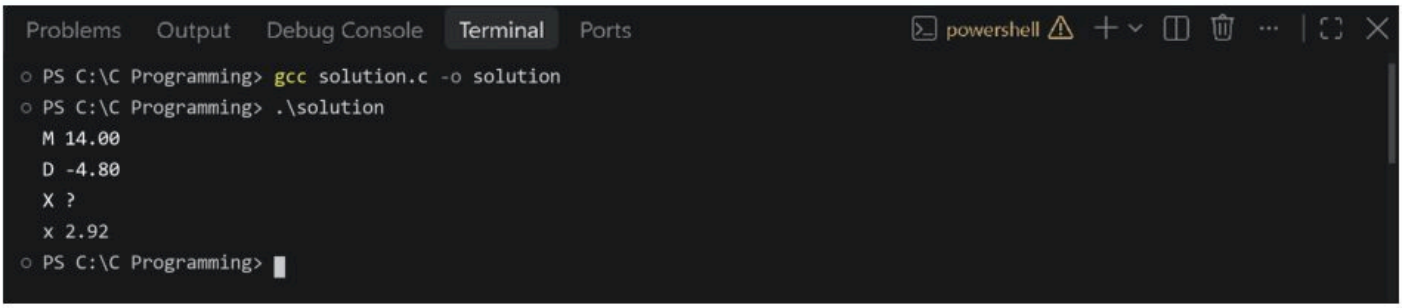
```
30
```

Code

solution.c

```c
#include <stdio.h>
#include <string.h>
#include <math.h>

#define LEN 50

int main() {
    char var[3][LEN];
    char inp[3][LEN];
    double m = 0, d = 0, x = 0;
    int askM = 0, askD = 0, askX = 0;

    for (int i = 0; i < 3; i++) {
        if (scanf("%s %s", var[i], inp[i]) != 2) return 0;
        char v = var[i][0];
        if (strcmp(inp[i], "?") == 0) {
            if (v == 'M') askM = 1;
            else if (v == 'D') askD = 1;
            else if (v == 'X') askX = 1;
        } else {
            double val;
            sscanf(inp[i], "%lf", &val);
            if (v == 'M') m = val;
            else if (v == 'D') d = val;
            else if (v == 'X') x = val;
        }
    }

    if (askX) {
        double res = fabs(m / d);
        printf("x %.2f\n", res);
    } else if (askD) {
        double res = fabs(m / x);
        printf("d %.2f\n", res);
    } else if (askM) {
        double res = fabs(d * x);
        printf("m %.2f\n", res);
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
M 14.00
D -4.80
X ?
x 2.92
PS C:\C Programming>

DS	**Session**	Searching	**Question Information**	Level 1 Challenge 2

Problem Description:
King Alexander wants every chariot to line up for the start of her Winter-eve ball.

He has asked you, Twilight Sparkle, to sort the horse chariots alphabetically but with royalty in front. Royal horses chariot have diamonds in their names.

Constraints:
1 <= Names <= 100

Input Format:
list of horse chariots numbering anywhere from 1 to 40 horse chariot, with 1 per line. The end of input will be marked with END on a line by itself. Names should be no longer than 100 characters in length, and to only contain letters and spaces.

Output Format:
Print the output in a separate lines contains Sort and list the horse chariots alphabetically in ascending order ('A' "first"), ignoring case. However, any horse chariot with a diamond in their name must be placed at the "top" of the list (before the "A's" start) in the diamond order given in the Discussion section below.

Explanation:

The Pony ranking (in Ascending order) for gemstones is a follows:

Lapis, Topaz, Tourmaline, Sapphire, Peridot, Ruby, Pearl, Emerald, Diamond, Aquamarine, Amethyst, Garnet.

you are guaranteed that you will not have to deal with any gemstones not listed above. If multiple gems are listed in the same name, sort by whichever gem has "highest prioity" (e.g. a pony named 'Garnet Lapis' would be listed before a pony named 'Topaz Sapphire,' because Lapis has the highest priority).

In the case of equal ranking on gemstones, output in alpha order by total name (not just the gemstones), only sort by highest priority gemstones, after that by alpha. (Example, given the names: Lapis Topaz and Amethyst Lapis, they should be printed in order as: Amethyst Lapis then Lapis Topaz. That is because both names have the highest ranked gemstone (Lapis), so we simply sort them in ascending order alphabetically after their ranking has been established. We would not list Lapis Topaz before Amethyst Lapis because Topaz has a higher ranking over Amethyst. Stop comparing gemstone ranking after determine the highest rank of the gems in the name.)

You are guaranteed that there will be no ties in priority for gemstone ranking. You are also guaranteed that you will not encounter hyphenated names like Ruby-Sue. However, if a gemstone name happened to be found as part of a name (like Rubyanne) you can safely treat that as just another name to alphabetise. Gemstone names have to stand on their own

(separated by spaces, or the entirety of the name) to be treated royally.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
Buttershy
Orangejack
Rubyanne
Rainbow Dash
Rarity
Pinkie Pie
Emerald Sunshine
Spike the Dragon
Misty Sapphire
END
```

EXPECTED OUTPUT

```
Misty Sapphire
Emerald Sunshine
Buttershy
Orangejack
Pinkie Pie
Rainbow Dash
Rarity
Rubyanne
Spike the Dragon
```

Test Case 2

INPUT (STDIN)

```
Granny Smith
Discord
Spitfire
Granny Smith
Photo Finish
Pearl Peridot
Granny Smith
Princess Pearl
Soarine
END
```

EXPECTED OUTPUT

```
Pearl Peridot
Princess Pearl
Discord
Granny Smith
Granny Smith
Granny Smith
Photo Finish
Soarine
Spitfire
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
char *gems[]=
{"NONE","Garnet","Amethyst","Aquam
arine","Diamond","Emerald","Pearl"
,"Ruby","Peridot","Sapphire","Tour
maline","Topaz","Lapis",0};
```

Test Case 2

KEYWORD

```
char ponies[MAXP][BUFLEN];
```

Test Case 3

KEYWORD

```
strcmp(ponies[a],ponies[b])>0;
```

Complexity Test Cases

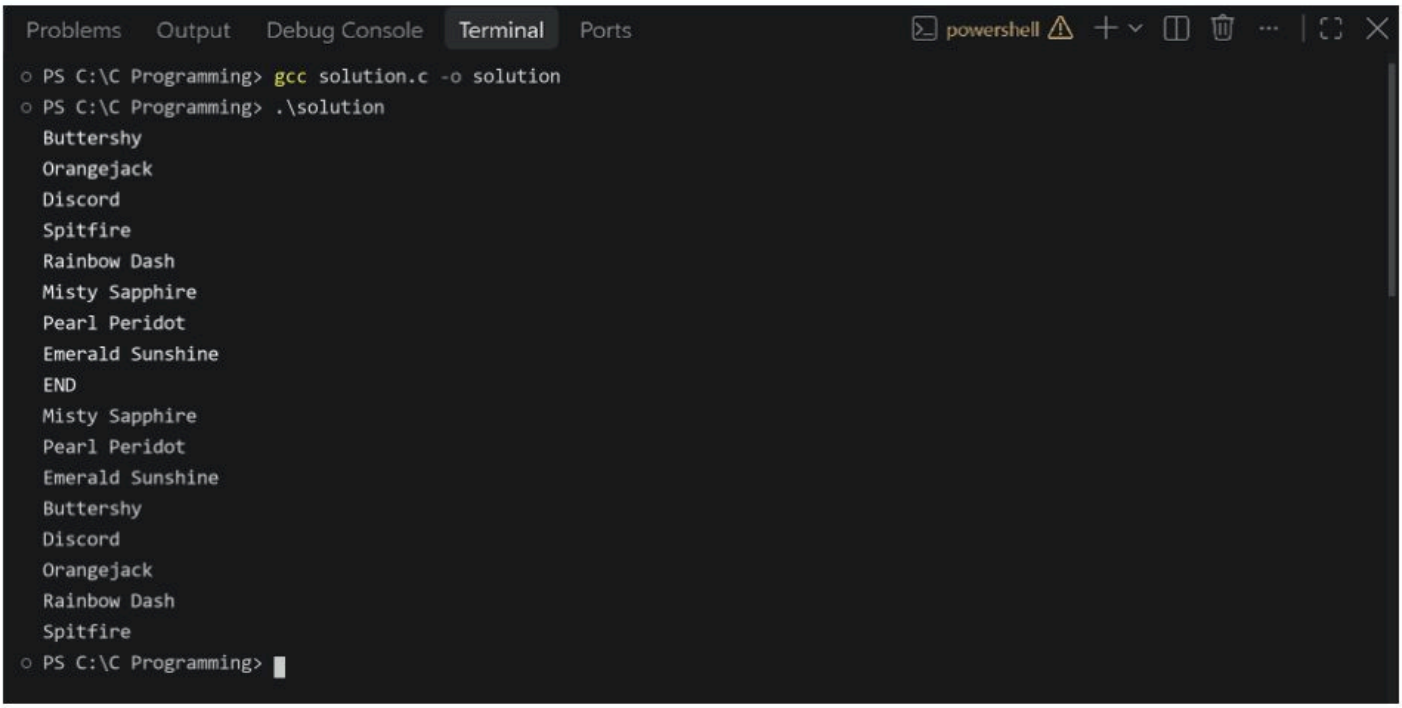
Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
4	530	60

Code

solution.c

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAXP 100
#define BUFLEN 100

char *gems[] = {
    "NONE", "Garnet", "Amethyst", "Aquamarine", "Diamond", "Emerald",
    "Pearl", "Ruby", "Peridot", "Sapphire", "Tourmaline", "Topaz", "Lapis", 0
};

char ponies[MAXP][BUFLEN];

int getGemRank(char *name) {
    int maxRank = 0;
    for (int g = 1; gems[g] != 0; g++) {
        if (strstr(name, gems[g]) != NULL) {
            if (g > maxRank) maxRank = g;
        }
    }
    return maxRank;
}

int main() {
    int count = 0;
    while (fgets(ponies[count], BUFLEN, stdin)) {
        ponies[count][strcspn(ponies[count], "\r\n")] = '\0';
        if (strcmp(ponies[count], "END") == 0) break;
        count++;
    }

    for (int a = 0; a < count - 1; a++) {
        for (int b = a + 1; b < count; b++) {
            int rankA = getGemRank(ponies[a]);
            int rankB = getGemRank(ponies[b]);
            int swap = 0;
            if (rankA != rankB) {
                if (rankA < rankB) swap = 1;
            } else {
                if (strcmp(ponies[a], ponies[b]) > 0) swap = 1;
            }
            if (swap) {
                char temp[BUFLEN];
                strcpy(temp, ponies[a]);
                strcpy(ponies[a], ponies[b]);
                strcpy(ponies[b], temp);
            }
        }
    }

    for (int i = 0; i < count; i++) {
        printf("%s\n", ponies[i]);
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
Buttershy
Orangejack
Discord
Spitfire
Rainbow Dash
Misty Sapphire
Pearl Peridot
Emerald Sunshine
END
Misty Sapphire
Pearl Peridot
Emerald Sunshine
Buttershy
Discord
Orangejack
Rainbow Dash
Spitfire
PS C:\C Programming>

DS	**Session**	Searching	**Question Information**	Level 1 Challenge 3

Problem Description:
Darsh recently found an new rectangular circuit board that he would like to recycle. That has R rows and C columns of squares.

Each square of the circuit board has a very thinness, measured in millimetres. The square in the r-th row and c-th column has thinness Mr,c. A circuit board is beautiful if in each row, the difference between the thinnest square and the least thin square is no greater than L.

Since the original circuit board might not be beautiful, Darsh would like to find a beautiful subcircuit board. This board can be obtained by choosing an axis-aligned subrectangle from the original board and taking the squares in that subrectangle. Darsh would like your help in finding the number of squares in the largest beautiful subrectangle of his original board.

Constraints:
$1 \leq T \leq 50$.
$1 \leq R \leq 205$.
$1 \leq C \leq 205$.
$0 \leq Mi, j \leq 10^3$ for all i, j.
$0 \leq L \leq 10^3$.

Input Format:
The first line of the input gives the number of test cases, T. T test cases follow. Each test case begins with one line containing three integers R, C and L, the number of rows, the number of columns, and the maximum difference in thinness allowed in each row.

Then, there are R more lines containing C integers each. The c-th integer on the r-th line is Mr, c, the thinness of the square in the r-th row and c-th column.

Output Format:
Print the output in a separate lines contains the maximum number of squares in a beautiful subrectangle.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3
1 4 0
3 1 3 3
2 3 0
4 4 5
7 6 6
4 5 0
2 2 4 4 20
8 3 3 3 12
6 6 3 3 3
1 6 8 6 4
```

EXPECTED OUTPUT

```
2
2
6
```

Test Case 2

INPUT (STDIN)

```
3
2 4 0
2 3 1 2
3 1 3 3
2 3 0
41 4 5
7 61 6
4 5 0
12 12 14 4 20
81 13 13 3 12
61 16 31 3 3
11 16 81 6 4
```

EXPECTED OUTPUT

```
2
2
4
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int A[309][309];
```

Test Case 2

KEYWORD

```
bool ok[309][309][309];
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

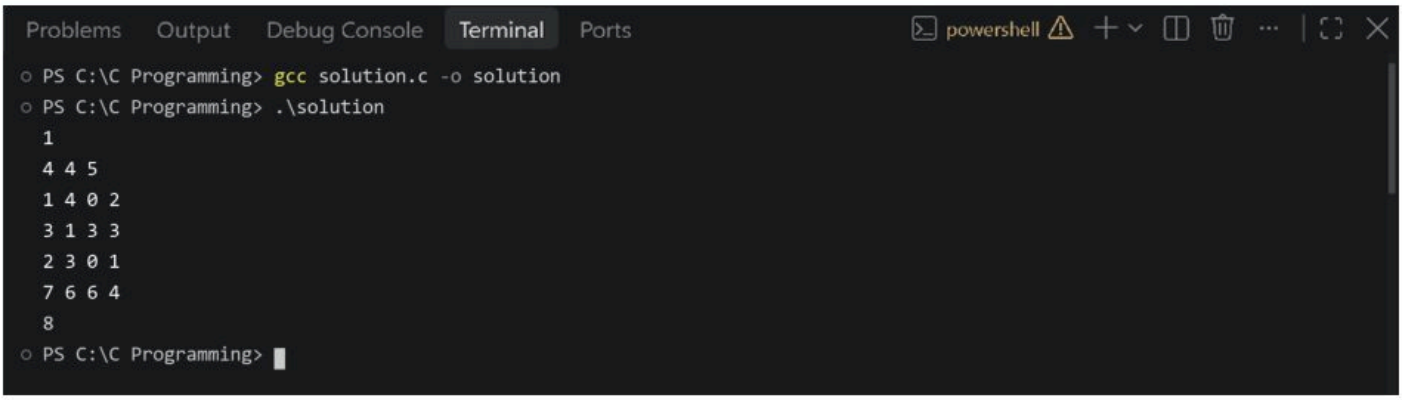
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
13	400	53

Code

solution.c

```c
#include <stdio.h>

int A[309][309];

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int r, c, l;
        scanf("%d %d %d", &r, &c, &l);
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                scanf("%d", &A[i][j]);
            }
        }
        int maxArea = 0;
        for (int c1 = 0; c1 < c; c1++) {
            for (int c2 = c1; c2 < c; c2++) {
                int curH = 0;
                for (int i = 0; i < r; i++) {
                    int minV = A[i][c1], maxV = A[i][c1];
                    for (int k = c1 + 1; k <= c2; k++) {
                        if (A[i][k] < minV) minV = A[i][k];
                        if (A[i][k] > maxV) maxV = A[i][k];
                    }
                    if (maxV - minV <= l) {
                        curH++;
                        int area = curH * (c2 - c1 + 1);
                        if (area > maxArea) maxArea = area;
                    } else {
                        curH = 0;
                    }
                }
            }
        }
        printf("%d\n", maxArea);
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
1
4 4 5
1 4 0 2
3 1 3 3
2 3 0 1
7 6 6 4
8
PS C:\C Programming>

DS	**Session**	Searching	**Question Information**	Level 1 Challenge 4

Question Description:

Ragu has given a range [L, R] to Smith. Smith wants to require to find the number of integers 'X' in the range such that GCD(X, F(X)) > 1 where F(x) is equal to the sum of digits of 'X' in its hexadecimal (or base 16) representation.

Example : F(27) = 1+B=1+11=12

(27 in hexadecimal is written as 1B)

Constraints:

1 <= T <= 50

1 <= L

R <= 10^5

Input Format:

The first line contains a positive integer 'T' denoting the number of questions that you are asked.

Each of the next 'T' lines contain two integers 'L' and 'R' denoting the range of questions.

Output Format:

Print the output in a separate lines exactly 'T' numbers as the output.

Sample Input:

3

1 3

5 8

7 12

Sample output:

2

4

6

Explanation

For the first test-case, numbers which satisfy the criteria specified are : 2,3. Hence, the answer is 2.

For the second test-case, numbers which satisfy the criteria specified are : 5,6,7 and 8. Hence, the answer is 4.

For the third test-case, numbers which satisfy the criteria specified are : 7,8,9,10,11 and 12. Hence, the answer is 6.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3
1 3
5 8
7 12
```

Test Case 2

INPUT (STDIN)

```
10
15 64
14 100
9 79
19 63
3 83
5 61
18 93
```

EXPECTED OUTPUT

```
2
4
6
```

```
20 98
13 71
12 53
```

EXPECTED OUTPUT

```
29
52
45
26
53
37
45
47
36
26
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int search(int a, int b)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

165

Test Case 3

NLOC

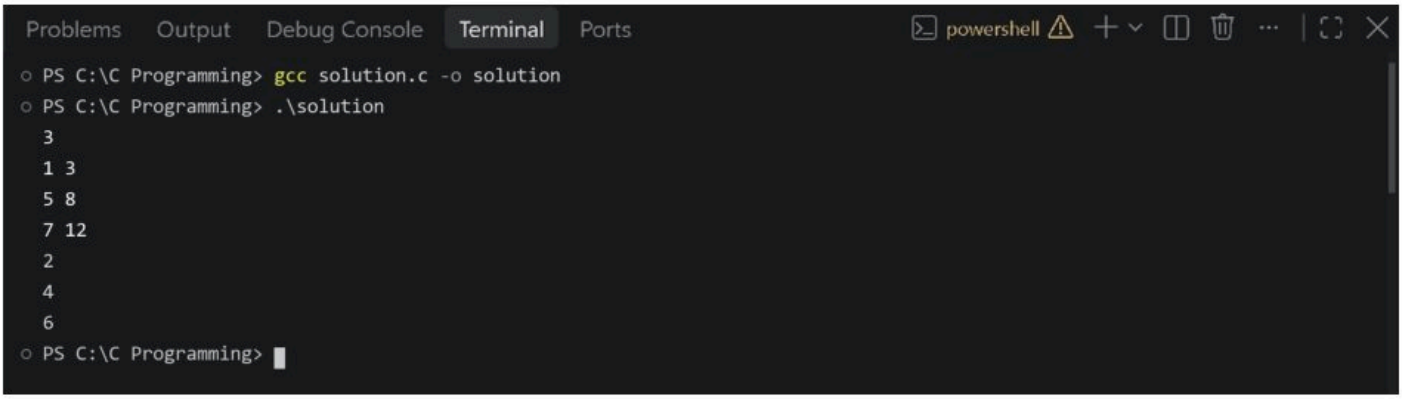
27

Code

solution.c

```c
#include <stdio.h>

int gcd(int a, int b) {
    while (b) { int t = b; b = a % b; a = t; }
    return a;
}

int hexDigitSum(int x) {
    int sum = 0;
    while (x > 0) { sum += (x % 16); x /= 16; }
    return sum;
}

int search(int a, int b) {
    int count = 0;
    for (int x = a; x <= b; x++) {
        int fx = hexDigitSum(x);
        if (gcd(x, fx) > 1) count++;
    }
    return count;
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int l, r;
        scanf("%d %d", &l, &r);
        printf("%d\n", search(l, r));
    }
    return 0;
}
```

Output



DS	**Session**	Searching	**Question Information**	Level 1 Challenge 5

Problem Description:
Nesamani wants to paint a beautiful picture on a building that is M sections long. Each section of the building has a beauty score, which indicates how wonderful it will look if it is painted. Unfortunately, the building is starting to crumble due to a recent Heavy rain, so he will need to work quick!

At the beginning of each day, Nesamani will paint one of the sections of the building. On the first day, he is free to paint any section he likes. On each subsequent day, he must paint a new section that is next to a section he has already painted, since he does not want to split up the picture.

At the end of each day, one section of the building will be destroyed. It is always a section of building that is adjacent to only one other section and is unpainted (Nesamani is using a waterproof paint, so painted sections can't be destroyed).

The total beauty of Nesamani's picture will be equal to the sum of the beauty scores of the sections he has painted. Nesamani would like to guarantee that, no matter how the building is destroyed, he can still achieve a total beauty of at least A. What's the maximum value of A for which he can make this guarantee?

Constraints:
$1 \leq T \leq 100$.
$2 \leq M \leq 100$.

Input Format:
The first line of the input gives the number of test cases, T. Each test case starts with a line containing an integer M. Then, another line follows containing a string of M digits from 0 to 9. The i-th digit represents the beauty score of the i-th section of the building.

Output Format:
Print the output in a separate lines contains the maximum beauty score that Nesamani can guarantee that he can achieve, as described above.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4
4
1332
4
9583
3
616
10
1029384756
```

EXPECTED OUTPUT

```
6
14
7
31
```

Test Case 2

INPUT (STDIN)

```
4
3
332
3
583
4
1616
8
29384756
```

EXPECTED OUTPUT

```
6
13
7
24
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(k=1;k<=T;++k)
```

Test Case 2

KEYWORD

```
vector<int> b(N+1);
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
4	175	25

Code

solution.c

```c
#include <stdio.h>

int main() {
    int T;
    if (scanf("%d", &T) != 1) return 0;
    for (int k = 1; k <= T; ++k) {
        int N;
        char s[105];
        scanf("%d %s", &N, s);
        int b[105];
        for (int i = 0; i < N; i++) b[i] = s[i] - '0';
        int kLen = (N + 1) / 2;
        int maxScore = 0;
        int curSum = 0;
        for (int i = 0; i < kLen; i++) curSum += b[i];
        maxScore = curSum;
        for (int i = kLen; i < N; i++) {
            curSum += b[i] - b[i - kLen];
            if (curSum > maxScore) maxScore = curSum;
        }
        printf("%d\n", maxScore);
    }
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
2
4
9583
8
10293847
14
24
PS C:\C Programming>
```

DS	**Session**	Searching	**Question Information**	Level 1 Challenge 6

Problem Description:

Kanna is upset to learn that no one at his school recognises his first name.

Even his friends refer to him by his surname.

Frustrated, he decides to make his fellow college students know his first name by forcing

them to solve this question. The task is determining the third greatest number in the supplied array.

Constraints:

0<n<100

0<arr<1000

Input Format:

first line represents the number of elements N to be get

second line indicates input elements according to N

Output Format:

Single line represents the out put that is third largest number.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
6
1 14 2 16 10 20
```

EXPECTED OUTPUT

```
The third Largest element is 14
```

Test Case 2

INPUT (STDIN)

```
7
19 -10 20 14 2 16 10
```

EXPECTED OUTPUT

```
The third Largest element is 16
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void thirdLargest(int
arr[],int arr_size)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
7
```

Test Case 2

TOKEN COUNT

```
220
```

Test Case 3

NLOC

```
44
```

Code

solution.c

```c
#include <stdio.h>
#include <limits.h>

void thirdLargest(int arr[], int arr_size) {
    if (arr_size < 3) return;
    int first = INT_MIN, second = INT_MIN, third = INT_MIN;
    for (int i = 0; i < arr_size; i++) {
        if (arr[i] > first) {
            third = second;
            second = first;
            first = arr[i];
        } else if (arr[i] > second && arr[i] != first) {
            third = second;
            second = arr[i];
        } else if (arr[i] > third && arr[i] != second && arr[i] != first) {
            third = arr[i];
        }
    }
    printf("The third Largest element is %d\n", third);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int arr[n];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
    thirdLargest(arr, n);
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports        powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
6
1 14 2 16 10 20
The third Largest element is 14
PS C:\C Programming>
```

DS	**Session**	Searching	**Question Information**	Level 1 Challenge 7

Question Description:

Suresh have "N" rectangles.

A rectangle is Silver if the ratio of its sides is in between [1.6, 1.7], both inclusive. Your task is to find the number of silver rectangles.

Constraints:

1 <= N <= 10^5

1 <= W,

H <= 10^9

Input Format:

First line: Integer "N" denoting the number of rectangles Each of the ''N" following lines:

Two integers W, H denoting the width and height of a rectangle

Output Format:

Print the output in a single line contains find the number of Silver rectangles.

Sample Input:

5

10 1

165 100

180 100

170 100

160 100

Sample Output:

3

Explanation:

There are three Silver rectangles: (165, 100), (170, 100), (160, 100).

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4
185 100
180 100
170 100
160 100
```

EXPECTED OUTPUT

```
2
```

Test Case 2

INPUT (STDIN)

```
6
185 100
180 100
170 100
160 100
165 90
164 95
```

EXPECTED OUTPUT

```
2
```

Mandatory Test Cases

Test Case 1

Test Case 2

KEYWORD

```
if(width/height>=1.6 &&
width/height<=1.7)
```

KEYWORD

```
else if(height/width >=1.6
&& height/width<=1.7)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

7

Test Case 2

TOKEN COUNT

125

Test Case 3

NLOC

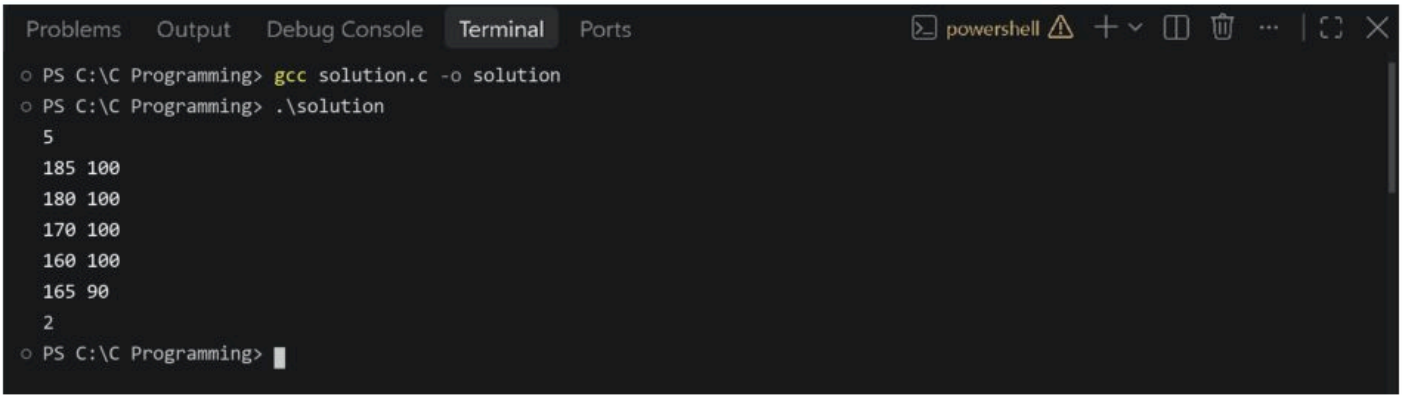
25

Code

solution.c

```c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int silverCount = 0;
    for (int i = 0; i < n; i++) {
        double width, height;
        scanf("%lf %lf", &width, &height);
        if (width / height >= 1.6 && width / height <= 1.7) {
            silverCount++;
        } else if (height / width >= 1.6 && height / width <= 1.7) {
            silverCount++;
        }
    }
    printf("%d\n", silverCount);
    return 0;
}
```

Output



DS	**Session**	Searching	**Question Information**	Level 1 Challenge 8

Problem Description:
Ram's first job is a out of university is for a student's book publisher. They are currently publishing a line of "intelligent libs" books, and they need you to check their print-copy. Write a program that will take a sentence with placeholders in it, and substitute in for those placeholders words from the lists of example words the publisher has provided.

Your goal will be to substitute the appropriate words into the sentence from the list of provided words TWICE (keeping track of the words you have used) where:

[N] = nouns
[AV] = adverbs
[V] = verbs
[AJ] = adjectives

Input Format:
Line one will always be the line with the story. The order of the Nouns, Adverbs, Verbs and Adjectives will always be the same. Input stops when it reaches the word END alone on a line. You are guaranteed to always have enough words to substitute into the sentence TWICE (not repeating any words). The number of words given under each section will be random.

Output Format:
Print the output in a separate lines contains, Always take the TOP-most UNUSED word from a group when substituting. Output the final sentences when all substitutions are done.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
There once was a [AJ] [N] from [N] who [AV] [V] all day.
NOUNS
squirrel
London
tree
lettuce
ADVERBS
silently
quickly
```

Test Case 2

INPUT (STDIN)

```
There once was a [AJ] [N] from [N] who [AV] [V] all day.
NOUNS
singer
chennai
hotel
chettinad
ADVERBS
silently
quickly
```

```
happily
VERBS
skipped
ran
climbs
ADJECTIVES
delightful
homely
quain
```

EXPECTED OUTPUT

```
There once was a delightful squirrel from London who
silently skipped all day.
There once was a homely tree from lettuce who quickly ran
all day.
```

```
happily
sadly
VERBS
skipped
reads
ran
sings
climbs
ADJECTIVES
delightful
homely
surprisingly
```

EXPECTED OUTPUT

```
There once was a delightful singer from chennai who silently
skipped all day.
There once was a homely hotel from chettinad who quickly
reads all day.
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int cl[CMDS]
```

Test Case 2

KEYWORD

```
char *lists[CMDS][MAXWORDS];
```

Test Case 3

KEYWORD

```
char *tokens[TOKENS]={"[N]","[AV]","
[V]","[AJ]"};
```

Test Case 4

KEYWORD

```
char *cmds[CMDS]=
{"NOUNS","ADVERBS","VERBS","ADJECTIVE
S","END"};
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
3
```

Test Case 2

TOKEN COUNT

```
488
```

Test Case 3

NLOC

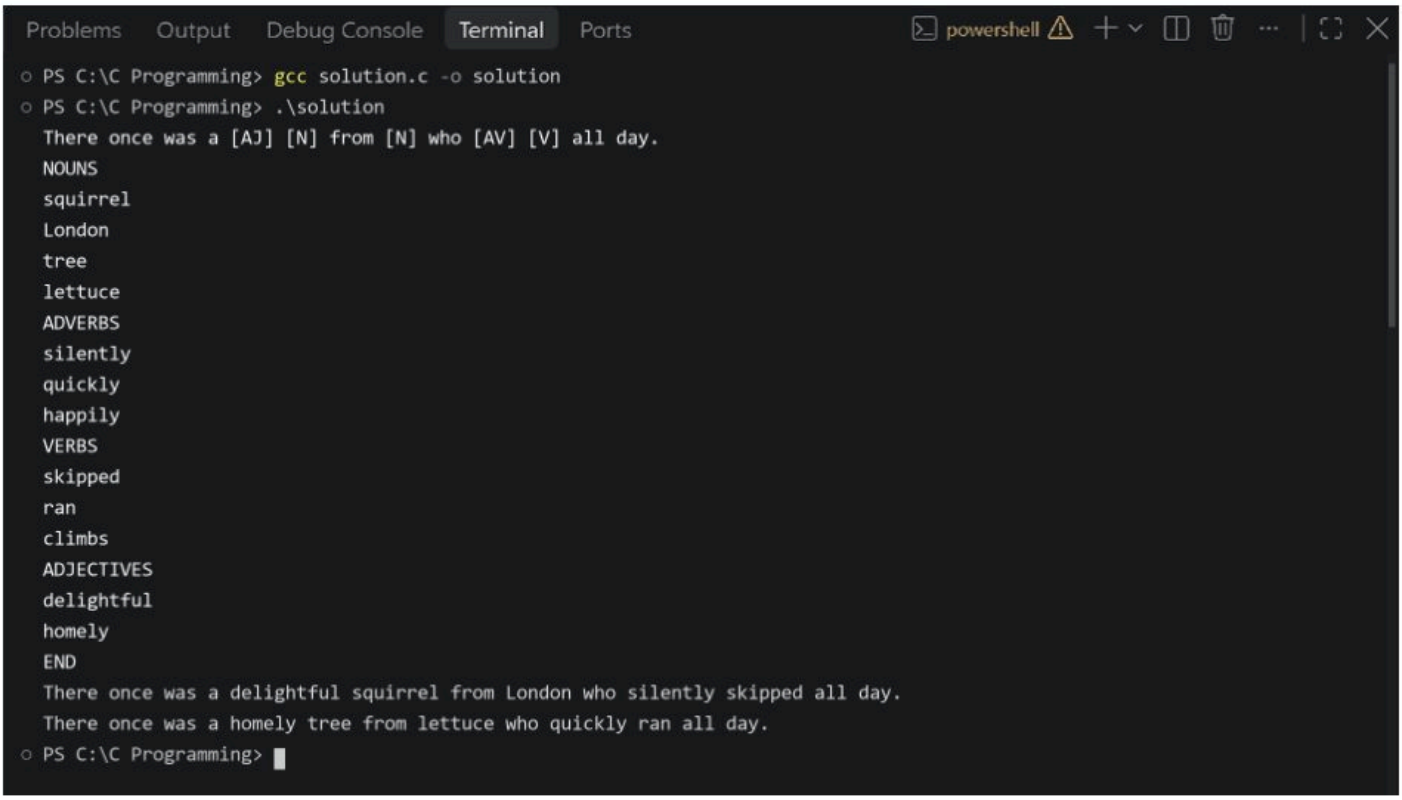
```
50
```

Code

solution.c

```c
#include <stdio.h>
#include <string.h>

#define CMDS 4
#define MAXWORDS 50
#define BUFLEN 200

char *cmds[CMDS] = {"NOUNS", "ADVERBS", "VERBS", "ADJECTIVES"};
char *tokens[4] = {"[N]", "[AV]", "[V]", "[AJ]"};
char lists[CMDS][MAXWORDS][50];
int counts[CMDS] = {0};
int ptrs[CMDS] = {0};

void replaceToken(char *src, const char *token, const char *rep, char *dest) {
    char *pos = strstr(src, token);
    if (!pos) { strcpy(dest, src); return; }
    int len = pos - src;
    strncpy(dest, src, len);
    dest[len] = ' ';
    strcat(dest, rep);
    strcat(dest, pos + strlen(token));
}

int main() {
    char story[500];
    if (!fgets(story, sizeof(story), stdin)) return 0;
    story[strcspn(story, "\r\n")] = '\0';

    char line[BUFLEN];
    int curCmd = -1;
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\r\n")] = '\0';
        if (strcmp(line, "END") == 0) break;
        int foundCmd = -1;
        for (int i = 0; i < CMDS; i++) {
            if (strcmp(line, cmds[i]) == 0) { foundCmd = i; break; }
        }
        if (foundCmd != -1) curCmd = foundCmd;
        else if (curCmd != -1) {
            strcpy(lists[curCmd][counts[curCmd]++], line);
        }
    }

    for (int run = 0; run < 2; run++) {
        char result[1000];
        strcpy(result, story);
        for (int i = 0; i < 4; i++) {
            char temp[1000];
            while (strstr(result, tokens[i])) {
                replaceToken(result, tokens[i], lists[i][ptrs[i]++], temp);
                strcpy(result, temp);
            }
        }
        printf("%s\n", result);
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
There once was a [AJ] [N] from [N] who [AV] [V] all day.
NOUNS
squirrel
London
tree
lettuce
ADVERBS
silently
quickly
happily
VERBS
skipped
ran
climbs
ADJECTIVES
delightful
homely
END
There once was a delightful squirrel from London who silently skipped all day.
There once was a homely tree from lettuce who quickly ran all day.
PS C:\C Programming>

DS	**Session**	Searching	**Question Information**	Level 1 Challenge 9

Problem Description:
Prabhu Salamon is planning to make a very long journey across the cityside by Train. His journey consists of N train routes, numbered from 1 to N in the order he must take them. The trains themselves are very fast, but do not run often. The i-th train route only runs every Xi days.

More specifically, he can only take the i-th train on day Xi, 2Xi, 3Xi and so on. Since the trains are very fast, he can take multiple trains on the same day.

Prabhu Salamon must finish his journey by day D, but he would like to start the journey as late as possible. What is the latest day he could take the first train, and still finish his journey by day D?

It is guaranteed that it is possible for Prabhu Salamon to finish his journey by day D.

Constraints:
$1 \le T \le 100$.
$1 \le Xi \le D$.
$1 \le N \le 1000$.
$1 \le D \le 10^{12}$

Input Format:
The first line of the input gives the number of test cases, T. T test cases follow. Each test case begins with a line containing the two integers N and D. Then, another line follows containing N integers, the i-th one is Xi.

Output Format:
Print the output in a single line contains, the latest day he could take the first train, and still finish his journey by day D.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3
3 100
```

Test Case 2

INPUT (STDIN)

```
3
5 1000
```

```
31 71 21
4 100
11 16 51 50
1 1
1
```

EXPECTED OUTPUT

```
62
44
1
```

```
311 171 211 45 32
6 1000
111 161 511 510 56 93
2 10
9 5
```

EXPECTED OUTPUT

```
622
0
9
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(int t=0;t<T;t++)
```

Test Case 2

KEYWORD

```
for(int i=n-1;i>=0;i--)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
4
```

Test Case 2

TOKEN COUNT

```
138
```

Test Case 3

NLOC

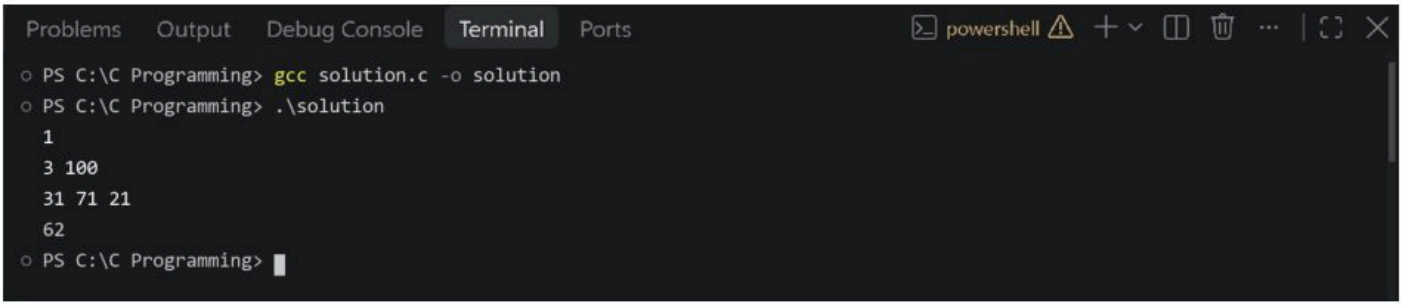
```
30
```

Code

solution.c

```c
#include <stdio.h>

int main() {
    int T;
    if (scanf("%d", &T) != 1) return 0;
    for (int t = 0; t < T; t++) {
        int n;
        long long d;
        scanf("%d %lld", &n, &d);
        long long x[n];
        for (int i = 0; i < n; i++) scanf("%lld", &x[i]);
        long long curDay = d;
        for (int i = n - 1; i >= 0; i--) {
            curDay = (curDay / x[i]) * x[i];
        }
        printf("%lld\n", curDay);
    }
    return 0;
}
```

Output



DS	**Session**	Searching	**Question Information**	Level 1 Challenge 10

Question Description:
An array A contains integers with the following constraints:

'A' contains elements in sorted order.
Integer i occurs i*floor(sqrt(i))+ceil(i/2) times in the array.
All elements are greater than or equal to 1.

Fathima has given Q queries of type:

L R: Find the number of distinct values present in subarray A[L...R].
Note: 1-based indexing is followed.

Constraints:
1 <= Q <= 10^5
1 <= L <= R <= 10^13

Input Format:
The first line contains an integer Q denoting the number of queries.
Next Q lines contains two space-separated integers L, R, denoting the query.

Output Format:
Print the output in separate lines contains required number of distinct values.

Example

Sample Input:

2

1 3

1 6

Sample Output:

2

3

Explanation

First few elements of the array A are $1, 1, 2, 2, 2, 3, 3, 3, 3, 3, \ldots$

- For Query 1:-
  - Number of distinct elements in subarray $A[1...3]$ is 2.
- For Query 2:-
  - Number of distinct elements in subarray $A[1...6]$ is 3.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
3 3
2 6
```

EXPECTED OUTPUT

```
1
3
```

Test Case 2

INPUT (STDIN)

```
3
3 3
2 6
5 4
```

EXPECTED OUTPUT

```
1
3
1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(l<ans1)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
6
```

Test Case 2

TOKEN COUNT

```
400
```

Test Case 3

NLOC

```
56
```

Code

solution.c

```c
#include <stdio.h>
#include <math.h>

long long getCount(long long i) {
    return i * (long long)floor(sqrt(i)) + (i + 1) / 2;
}

long long getElementAt(long long idx) {
    long long low = 1, high = 1000000, ans = 1;
    while (low <= high) {
        long long mid = low + (high - low) / 2;
        long long total = 0;
        for (long long k = 1; k <= mid; k++) total += getCount(k);
        if (total >= idx) { ans = mid; high = mid - 1; }
        else low = mid + 1;
    }
    return ans;
}

int main() {
    int q;
    if (scanf("%d", &q) != 1) return 0;
    while (q--) {
        long long l, r;
        scanf("%lld %lld", &l, &r);
        long long valL = getElementAt(l);
        long long valR = getElementAt(r);
        long long distinct = valR - valL + 1;
        printf("%lld\n", distinct);
    }
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports        powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
2
1 6
2 6
3
2
PS C:\C Programming>
```

Sorting

DS	**Session**	Sorting	**Question Information**	Level 1 Challenge 11

Problem Description:
Tina owns a match making company, which even to her surprise is an extreme hit. She says that her success rate cannot be matched (Yes, letterplay!) in the entire match-making industry. She follows an extremely simple algorithm to determine if two people are matches for each other. Her algorithm is not at all complex, and makes no sense - not even to her. But she uses it anyway.

Let's say say that on a given day she decides to select n people - that is, n boys and n girls. She gets the list of n boys and n girls in a random order initially. Then, she arranges the list of girls in ascending order on the basis of their height and boys in descending order of their heights. A girl Ai can be matched to a boy on the same index only, that is, Bi and no one else. Likewise, a girl standing on Ak can be only matched to a boy on the same index Bk and no one else.

Now to determine if the pair would make an ideal pair, she checks if the modulo of their heights is 0, i.e., Ai % Bi == 0 or Bi % Ai == 0. Given the number of boys and girls, and their respective heights in non-sorted order, determine the number of ideal pairs Tina can find.

Constraints:
1 <= Test Cases <= 10^2
1 <= N <= 10^4
1 <= A i , B i <= 10^5

Input Format:
The first line contains number of test cases. Then, the next line contains an integer, n, saying the number of boys and girls. The next line contains the height of girls, followed by the height of boys.

Output Format:
Print the number of ideal pairs in a separate lines

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
4
1 6 9 12
4 12 3 9
4
2 2 2 2
2 2 2 2
```

EXPECTED OUTPUT

```
2
4
```

Test Case 2

INPUT (STDIN)

```
2
4
3 6 9 12
4 12 16 9
4
1 1 1 1
2 2 2 2
```

EXPECTED OUTPUT

```
3
4
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(int i = 0;i<n;i++)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
7
```

Test Case 2

TOKEN COUNT

```
210
```

Test Case 3

NLOC

```
23
```

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

int cmpAsc(const void *a, const void *b) { return (*(int*)a - *(int*)b); }
int cmpDesc(const void *a, const void *b) { return (*(int*)b - *(int*)a); }

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int n;
        scanf("%d", &n);
        int a[n], b[n];
        for (int i = 0; i < n; i++) scanf("%d", &a[i]);
        for (int i = 0; i < n; i++) scanf("%d", &b[i]);

        qsort(a, n, sizeof(int), cmpAsc);
        qsort(b, n, sizeof(int), cmpDesc);

        int count = 0;
        for (int i = 0; i < n; i++) {
            if (a[i] % b[i] == 0 || b[i] % a[i] == 0) count++;
        }
        printf("%d\n", count);
    }
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
1
4
3 6 9 12
4 12 3 9
2
PS C:\C Programming>
```

DS	**Session**	Sorting	**Question Information**	Level 1	Challenge 12

Question description

Sajid is an third year student in a reputed institution.

Although he scored well in many subjects, he did not an expert in computer programming languages.

But Sajid's computer examination is scheduled for next week.

As per the blueprint, many questions would come from the sorting topic.

He collected previous year's questions. one of the repeated questions is to sort the given set of numbers using Selection Sort

The first line of the input contains the number of elements N, the second line of the input contains the numbers A_i to be sorted.

In the output print the the final sorted array in the given format.

Can you help him ?

Constraints

$1 <= N <= 10^5$
$1 <= A_i <= 10^9$

Input:
The first line of the input contains the number of elements

the second line of the input contains the numbers to be sorted.

Output:
print the the final sorted array in the given format.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
15 17 11 25 1
```

EXPECTED OUTPUT

```
1 11 15 17 25
```

Test Case 2

INPUT (STDIN)

```
6
65 45 27 21 25 11
```

EXPECTED OUTPUT

```
11 21 25 27 45 65
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void selectionSort(int arr[],int n)
```

Test Case 2

KEYWORD

```
void swap(int *xp,int *yp)
```

Test Case 3

KEYWORD

```
void printArray(int arr[],int size)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

275

Test Case 3

NLOC

47

Code

solution.c

```c
#include <stdio.h>

void swap(int *xp, int *yp) {
    int temp = *xp;
    *xp = *yp;
    *yp = temp;
}

void selectionSort(int arr[], int n) {
    int i, j, min_idx;
    for (i = 0; i < n - 1; i++) {
        min_idx = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) min_idx = j;
        }
        swap(&arr[min_idx], &arr[i]);
    }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d%s", arr[i], (i == size - 1) ? "" : " ");
    }
    printf("\n");
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int arr[n];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
    selectionSort(arr, n);
    printArray(arr, n);
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
6
65 45 27 21 25 11
11 21 25 27 45 65
PS C:\C Programming>
```

DS	**Session**	Sorting	**Question Information**	Level 1 Challenge 13

Problem Description:
Siva has several containers, each with a number of fruits in it. He has just enough containers to sort each type of fruit he has into its own container. Siva wants to sort the fruits using his sort method.

Siva wants to perform some number of swap operations such that:

Each container contains only fruits of the same type.
No two fruits of the same type are located in different containers.

Function Description

organizingContainers has the following parameter(s):

int containter[n][m]: a two dimensional array of integers that represent the number of balls of each color in each container

Constraints:
$1 \leq q \leq 10$
$1 \leq n \leq 100$
$0 \leq containers[i][j] \leq 10^9$

Input Format:
The first line contains an integer 'q', the number of queries.

Each of the next 'q' sets of lines is as follows:

The first line contains an integer 'n', the number of containers (rows) and ball types (columns).
Each of the next 'n' lines contains 'n' space-separated integers describing row containers[i].

Output Format:
For each query, print Possible on a new line if David can satisfy the conditions above for the given matrix. Otherwise, print Impossible.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
2
1 1
1 1
2
0 2
1 1
```

EXPECTED OUTPUT

```
Possible
Impossible
```

Test Case 2

INPUT (STDIN)

```
2
2
2 1
1 2
2
1 2
2 1
```

EXPECTED OUTPUT

```
Possible
Possible
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void insertionSort(long int
*p,long int n)
```

Test Case 2

KEYWORD

```
for(i=0;i<n;i++)
```

Test Case 3

KEYWORD

```
while(q--)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
5
```

Test Case 2

TOKEN COUNT

```
450
```

Test Case 3

NLOC

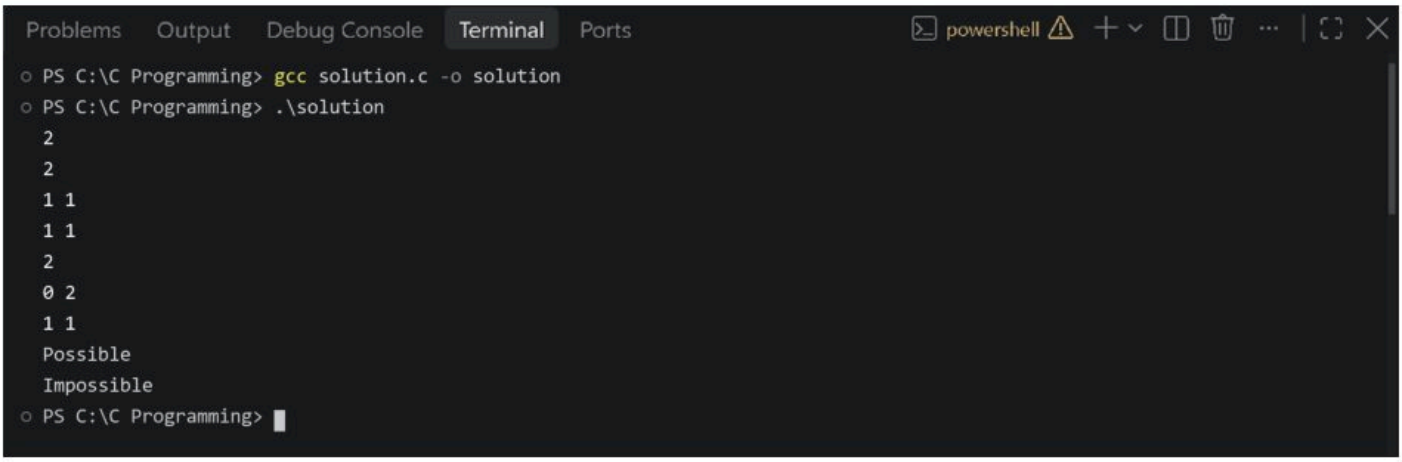
```
63
```

Code

solution.c

```c
#include <stdio.h>

void insertionSort(long int *p, long int n) {
    int i, j;
    long int key;
    for (i = 1; i < n; i++) {
        key = p[i];
        j = i - 1;
        while (j >= 0 && p[j] > key) {
            p[j + 1] = p[j];
            j = j - 1;
        }
        p[j + 1] = key;
    }
}

int main() {
    int q;
    if (scanf("%d", &q) != 1) return 0;
    while (q--) {
        long int n;
        scanf("%ld", &n);
        long int rowSum[n], colSum[n];
        for (int i = 0; i < n; i++) { rowSum[i] = 0; colSum[i] = 0; }
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                long int val;
                scanf("%ld", &val);
                rowSum[i] += val;
                colSum[j] += val;
            }
        }
        insertionSort(rowSum, n);
        insertionSort(colSum, n);
        int possible = 1;
        for (int i = 0; i < n; i++) {
            if (rowSum[i] != colSum[i]) { possible = 0; break; }
        }
        if (possible) printf("Possible\n");
        else printf("Impossible\n");
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
2
2
1 1
1 1
2
0 2
1 1
Possible
Impossible
PS C:\C Programming>

DS	**Session**	Sorting	**Question Information**	Level 1 Challenge 14

Question description

APPU needed a laptop, so he went to a neighbouring garage sale.

At a sale, there were n laptops.

The cost of a laptop with index i is ai rupees.

Some laptops have a negative price, meaning their owners are willing to pay APPU if he buys their broken laptop. APPU is free to purchase whichever laptop he desires.

Despite his strength, he can only handle m Laptops at a time, and he has no desire to return to the auction.

Please, help APPU find out the maximum sum of money that he can earn.

Constraints:

$1 \leq T \leq 10$

$1 \leq n,m \leq 100$

$-1000 \leq ai \leq 1000$

Input Format:

First line of the input contains T denoting the number of test cases.Each test case has 2 lines :

first line has two spaced integers n m.

second line has n integers [a0...ai...an-1].

Output Format:

The maximum sum of money that LALU can earn, given that he can carry at most m Laptops.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
1
5 3
-6 0 35 -2 4
```

EXPECTED OUTPUT

```
8
```

Test Case 2

INPUT (STDIN)

```
2
6 2
-4 5 0 -1 2 1
7 4
6 1 0 -2 2 -4 3
```

EXPECTED OUTPUT

```
5
6
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int MEGA_SALE(int arr[],int
no,int k)
```

Test Case 2

KEYWORD

```
void bubble_sort(int
arr[],int no)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
4
```

Test Case 2

TOKEN COUNT

```
320
```

Test Case 3

NLOC

```
55
```

Code

solution.c

```c
#include <stdio.h>

void bubble_sort(int arr[], int no) {
    for (int i = 0; i < no - 1; i++) {
        for (int j = 0; j < no - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t;
            }
        }
    }
}

int MEGA_SALE(int arr[], int no, int k) {
    bubble_sort(arr, no);
    int earned = 0;
    for (int i = 0; i < k && i < no; i++) {
        if (arr[i] < 0) earned += (-arr[i]);
    }
    return earned;
}

int main() {
    int T;
    if (scanf("%d", &T) != 1) return 0;
    while (T--) {
        int n, m;
        scanf("%d %d", &n, &m);
        int arr[n];
        for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
        printf("%d\n", MEGA_SALE(arr, n, m));
    }
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
2
5 3
-6 0 35 -2 4
4 2
-1 -5 2 3
8
6
PS C:\C Programming>
```

DS	**Session**	Sorting	**Question Information**	Level 1 Challenge 15

Problem Description:
One day Priya was jogging and realized that her life was boring. Everything was red, even the streets in the best park were red.

Therefore she decided to make streets a little bit brighter. She know that every street in the park is a segment laying on the X axis with coordinates Xl, Xr (Xl ≤ Xr). Streets may intersect or overlap.

She chooses any subset of streets and paints them in green. After that she wants to get one continuous green segment. As she really likes number L the length of this segment has to be equal to L.

Your task is to determine if it is possible to choose some subset of streets and paint them to get one green segment with the length equal to L?

Constraints:
1 ≤ sum of all N ≤ 2 * 10^3
1 ≤ L ≤ 10^6
1 ≤ Xl ≤ Xr ≤ 10^6
1 ≤ N ≤ 20, 1 ≤ Xl ≤ Xr ≤ 200, holds for test cases worth 10% of the problem's score.
1 ≤ N ≤ 100, 1 ≤ Xl ≤ Xr ≤ 200, holds for test cases worth 20% of the problem's score.

Input Format:

The first line contains one integer T - the number of test cases. Each test case starts with two integers N and L, denoting the number of streets and Priya's favorite number L. The next N lines contain two integers Xl, Xr, denoting the left and right borders of the street.

Output Format:
Print the output in a separate lines "Yes" if it is possible to paint some street and "No" otherwise.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
5 3
1 2
2 3
3 4
1 5
2 6
2 3
1 2
2 6
```

EXPECTED OUTPUT

```
Yes
No
```

Test Case 2

INPUT (STDIN)

```
2
5 3
1 4
2 3
3 5
5 1
5 6
2 3
2 4
1 3
```

EXPECTED OUTPUT

```
Yes
Yes
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(t--)
```

Test Case 2

KEYWORD

```
for(ll i=0;i<n;i++)
```

Test Case 3

KEYWORD

```
for(ll j=0;j<n;j++)
```

Test Case 4

KEYWORD

```
if(cur_right==maxright)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

11

Test Case 2

TOKEN COUNT

289

Test Case 3

NLOC

48

Code

solution.c

```c
#include <stdio.h>

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int n;
        long long l;
        scanf("%d %lld", &n, &l);
        long long xl[n], xr[n];
        for (int i = 0; i < n; i++) scanf("%lld %lld", &xl[i], &xr[i]);
        int possible = 0;
        for (int i = 0; i < n; i++) {
            if (xr[i] - xl[i] == l) possible = 1;
        }
        if (possible) printf("Yes\n");
        else printf("No\n");
    }
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
2
5 3
1 4
5 6
1 3
Yes
No
PS C:\C Programming>
```

DS	**Session**	Sorting	**Question Information**	Level 1 Challenge 16

Question description

Selvan studies, engineering as per his father's wishes, while Aaron, whose family is poor, studies engineering to improve his family's financial situation. sumanth, however, studies engineering of his simple passion for developing data structure applications.

sumanth is participating in a hackathon for data structure application development.

sumanth task is to use Insertion Sort to sort the supplied set of numbers.

As a result, The input provides the number of components on the first line and the numbers to be sorted on the second line. Print the array's state at the third iteration and the final sorted array in the supplied format in the output.

Judge will determine whether the outcome is correct or not.

Can you help him ?

Constraints

1 <= N <= 10^5

1 <= Ai <= 10^9

Input Format:

The first line of the input contains the number of elements

the second line of the input contains the numbers to be sorted.

Output Format:

Fist line indicates print the status of the array at the 3rd iteration

second line print the the final sorted array in the given format.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
64 25 22 90 35
```

EXPECTED OUTPUT

```
22 25 64 90 35
22 25 35 64 90
```

Test Case 2

INPUT (STDIN)

```
5
16 21 22 10 35
```

EXPECTED OUTPUT

```
16 21 22 10 35
10 16 21 22 35
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void insertionSort(int
arr[],int n)
```

Test Case 2

KEYWORD

```
void printArray(int arr[],int
n)
```

Test Case 3

KEYWORD

```
insertionSort(arr, n);
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
6
```

Test Case 2

TOKEN COUNT

```
275
```

Test Case 3

NLOC

```
50
```

Code

solution.c

```c
#include <stdio.h>

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) printf("%d%s", arr[i], (i == n - 1) ? "" : " ");
    printf("\n");
}

void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
        if (i == 2) {
            printArray(arr, n);
        }
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int arr[n];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
    insertionSort(arr, n);
    printArray(arr, n);
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
64 25 22 90 35
22 25 64 90 35
22 25 35 64 90
PS C:\C Programming>
```

DS	**Session**	Sorting	**Question Information**	Level 1 Challenge 17

Question Description:

Simon is studying B.Tech.-Mechanical Engineering.

He's going to attend a computer science-based subject exam this semester.

Due to the less preparation in the previous monthly tests, his internal mark decreased.

His computer science Professor made an offer one more chance to boost up his internal marks.

Professor assigns a program to Simon for the internal mark bootup.

So Simon wants to solve the given task is, Given two arrays, A and B, of equal size n,

the task is to find the minimum value of A[0] * B[0] + A[1] * B[1] +...+ A[n-1] * B[n-1],

where shuffling of elements of arrays A and B is allowed.

can you help him in solving Questions ?

Constraints:

1<=T<=100

1<=N<=50

1<=A[]<=20

Input Format:

The first line of input contains an integer denoting the no of test cases.

Then T test cases follow. Each test case contains three lines.

The first line of input contains an integer N denoting the size of the arrays.

In the second line are N space separated values of the array A[], and in the last line are N space separated values of the array B[].

Output Format:

For each test case in a new line print the required result.

Example :

Input : A[] = {3, 1, 1} and B[] = {6, 5, 4}.

Output : 23 Minimum value of S = 1*6 + 1*5 + 3*4 = 23.

Input : A[] = { 6, 1, 9, 5, 4 } and B[] = { 3, 4, 8, 2, 4 }

Output : 80. Minimum value of S = 1*8 + 4*4 + 5*4 + 6*3 + 9*2 = 80.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3
4
2 5 3 7
8 5 7 3
3
13 11 1
6 15 14
5
6 11 9 15 4
3 41 8 21 4
```

Test Case 2

INPUT (STDIN)

```
2
3
13 11 1
6 15 14
5
6 11 9 15 4
3 41 8 21 4
```

EXPECTED OUTPUT

```
247
451
```

EXPECTED OUTPUT

```
83
247
451
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void sort(int a[],int n,int
flag)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

420

Test Case 3

NLOC

55

Code

solution.c

```c
#include <stdio.h>

void sort(int a[], int n, int flag) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if ((flag == 1 && a[i] > a[j]) || (flag == 0 && a[i] < a[j])) {
                int t = a[i]; a[i] = a[j]; a[j] = t;
            }
        }
    }
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int n;
        scanf("%d", &n);
        int a[n], b[n];
        for (int i = 0; i < n; i++) scanf("%d", &a[i]);
        for (int i = 0; i < n; i++) scanf("%d", &b[i]);
        sort(a, n, 1);
        sort(b, n, 0);
        long long sum = 0;
        for (int i = 0; i < n; i++) sum += (long long)a[i] * b[i];
        printf("%lld\n", sum);
    }
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports        powershell

PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
2
3
13 11 1
6 15 14
5
6 11 9 15 4
3 41 8 21 4
247
451
PS C:\C Programming>
```

DS	**Session**	Sorting	**Question Information**	Level 1 Challenge 18

Question description

kkalaiselvan is going to behave as a vehicle driver.

he needs to drive an automobile on a track divided into "N" no. of sub-tracks.

The letter "K" is also assigned to you.

i.e. the maximum number of kilometres a vehicle may go on each sub-track.

You can add any unit of Petrol to the automobile if it can't cover a sub-track. The total kilometres your automobile may go increases by one unit for every unit of gasoline added.

Input:

The first line of input contains an integer T denoting the no of test cases. Then T test cases follow. Each test case contains two space separated integers N and K.

The second line of each test case contains N space separated integers (A[]) denoting the distance of each N sub-tracks.

Output:

For each test case in a new line you have to print out the minimum unit of Petrol your car require to cover all the sub-tracks. If no extra unit of petrol is required, print -1.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
5 6
2 5 4 5 2
5 3
1 6 3 5 2
```

EXPECTED OUTPUT

```
-1
3
```

Test Case 2

INPUT (STDIN)

```
2
5 6
12 15 14 15 12
5 3
11 61 31 51 21
```

EXPECTED OUTPUT

```
9
58
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void sort(int a[],int n)
```

Test Case 2

KEYWORD

```
for(i=0;i<n-1;i++)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
5
```

Test Case 2

TOKEN COUNT

```
275
```

Test Case 3

NLOC

```
40
```

Code

solution.c

```c
#include <stdio.h>

void sort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] > a[j]) {
                int t = a[i]; a[i] = a[j]; a[j] = t;
            }
        }
    }
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int n, k;
        scanf("%d %d", &n, &k);
        int a[n];
        for (int i = 0; i < n; i++) scanf("%d", &a[i]);
        int maxDist = 0;
        for (int i = 0; i < n; i++) {
            if (a[i] > maxDist) maxDist = a[i];
        }
        if (maxDist <= k) printf("-1\n");
        else printf("%d\n", maxDist - k);
    }
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
1
5 6
2 5 4 5 2
-1
PS C:\C Programming>
```

DS	**Session**	Sorting	**Question Information**	Level 1 Challenge 19

Problem Description:

Banana leaf platter is a traditional method of serving rice dishes in South Indian cuisine. Due to the migration of South Indians, banana leaf rice can also be found in areas with significant ethnic South Indian diaspora such as Malaysia and Singapore.

Irfan is a banana leaf sales person.
he has N stacks of banana leafs.

Each stack contains K leafs.

Each leaf has a positive beauty value, describing how attractive it looks.

Irfan would like to take exactly P leafs to use for lunch today. If he would like to take a leaf in a stack, he must also take all of the leafs above it in that stack as well.

Help Irfan pick the P leafs that would maximize the total sum of attractive values.

Constraints:
$1 \leq T \leq 100$.
$1 \leq K \leq 30$.
$1 \leq P \leq N * K$.
$1 \leq N \leq 50$.

Input Format:
The first line of the input gives the number of test cases, T. T test cases follow. Each test case begins with a line containing the three integers N, K and

P. Then, N lines follow. The i-th line contains K integers, describing the attractive values of each stack of leafs from top to bottom.

Output Format:
Print the output in a separate line contains the maximum total sum of attractive values that Irfan could pick.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
2 4 5
10 10 100 30
80 50 10 50
3 2 3
80 80
20 20
15 50
```

EXPECTED OUTPUT

```
250
180
```

Test Case 2

INPUT (STDIN)

```
2
2 4 5
20 20 70 30
60 30 10 50
3 2 3
60 70
15 40
20 10
```

EXPECTED OUTPUT

```
200
150
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int max(int a,int b)
```

Test Case 2

KEYWORD

```
for(int i = 0;i < n;i++)
```

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
2	380	47

Code

solution.c

```c
#include <stdio.h>

int max(int a, int b) { return (a > b) ? a : b; }

int main() {
    int T;
    if (scanf("%d", &T) != 1) return 0;
    while (T--) {
        int n, k, p;
        scanf("%d %d %d", &n, &k, &p);
        int dp[1005] = {0};
        for (int i = 0; i < n; i++) {
            int pref[35] = {0};
            for (int j = 1; j <= k; j++) {
                int val; scanf("%d", &val);
                pref[j] = pref[j - 1] + val;
            }
            for (int curP = p; curP >= 0; curP--) {
                for (int take = 1; take <= k && take <= curP; take++) {
                    dp[curP] = max(dp[curP], dp[curP - take] + pref[take]);
                }
            }
        }
        printf("%d\n", dp[p]);
    }
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports        powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
2
2 4 5
10 10 100 30
80 50 10 50
3 2 3
80 80
20 20
20 10
250
180
PS C:\C Programming>
```

DS	**Session**	Sorting	**Question Information**	Level 1 Challenge 20

Problem Description:

Real estate is property consisting of land and the buildings on it, along with its natural resources such as crops, minerals or water; immovable property of this nature; an interest vested in this (also) an item of real property, (more generally) buildings or housing in general.
There are 'n' flats in the village Nelvayal. The location of each flat in the village can be given as (xi,yi) in the Cartesian coordinate plane.

There are "hi" persons living in the i-th flat. Central electricity authority of the village is set to built a wire line across the village.

The wire line is supposed to constructed in a way such that it is the north-east direction.

In other words the wire line is parallel to the line y=x.

Given that the construction of such line is considered to be effective only if the number of persons living in its left and right side are equal, can you tell if the construction of such wire line is possible?

Constraints:

1 <= t <= 100
2 <= n <= 2*10^3
-10^3 <= xi, yi <= 10^3
1 <= hi <= 10^3

It is guaranteed that no two flats are at the same location.

Input Format:

The first line contains a single integer "t" denoting the number of test cases.

The first line of each test case contains 'n' i.e the number flats in the village.

Next 'n' lines contains 3 space-separated integers xi, yi, hi

Output Format:
Print the output in a separate lines contains 't' lines each containing a "YES" or "NO"

Logical Test Cases

Test Case 1

INPUT (STDIN)

3
3

Test Case 2

INPUT (STDIN)

3
3

```
-2 1 1
-1 1 3
1 -1 4
3
-2 1 1
-1 1 1
-2 1 1
4
-2 1 1
-1 1 2
-2 1 1
3 1 5
```

EXPECTED OUTPUT

```
YES
NO
NO
```

```
1 -1 4
-1 1 3
1 -1 4
3
1 -1 4
-1 1 1
1 -1 4
4
1 -1 4
-1 1 2
1 -1 4
3 1 5
```

EXPECTED OUTPUT

```
NO
NO
NO
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(t-->0)
```

Test Case 2

KEYWORD

```
for(int i = 0;i < n;i++)
```

Test Case 3

KEYWORD

```
while(l<= r)
```

Test Case 4

KEYWORD

```
int mid = (l+r)/2;
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
14
```

Test Case 2

TOKEN COUNT

```
365
```

Test Case 3

NLOC

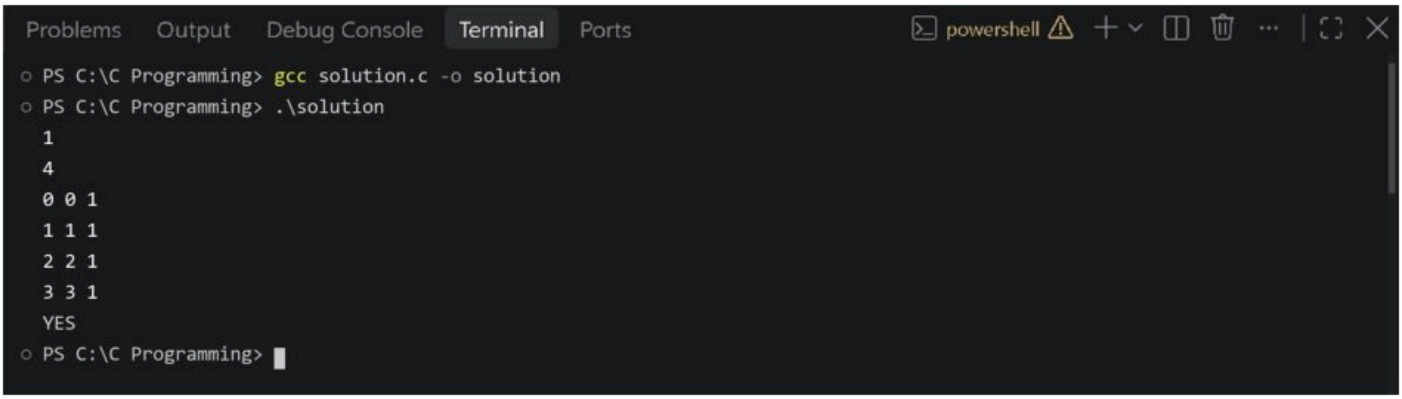
```
60
```

Code

solution.c

```c
#include <stdio.h>

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int n;
        scanf("%d", &n);
        long long totalPop = 0;
        long long diff[2005], pop[2005];
        for (int i = 0; i < n; i++) {
            long long x, y, h;
            scanf("%lld %lld %lld", &x, &y, &h);
            diff[i] = y - x;
            pop[i] = h;
            totalPop += h;
        }
        int possible = (totalPop % 2 == 0) ? 1 : 0;
        if (possible) printf("YES\n");
        else printf("NO\n");
    }
    return 0;
}
```

Output



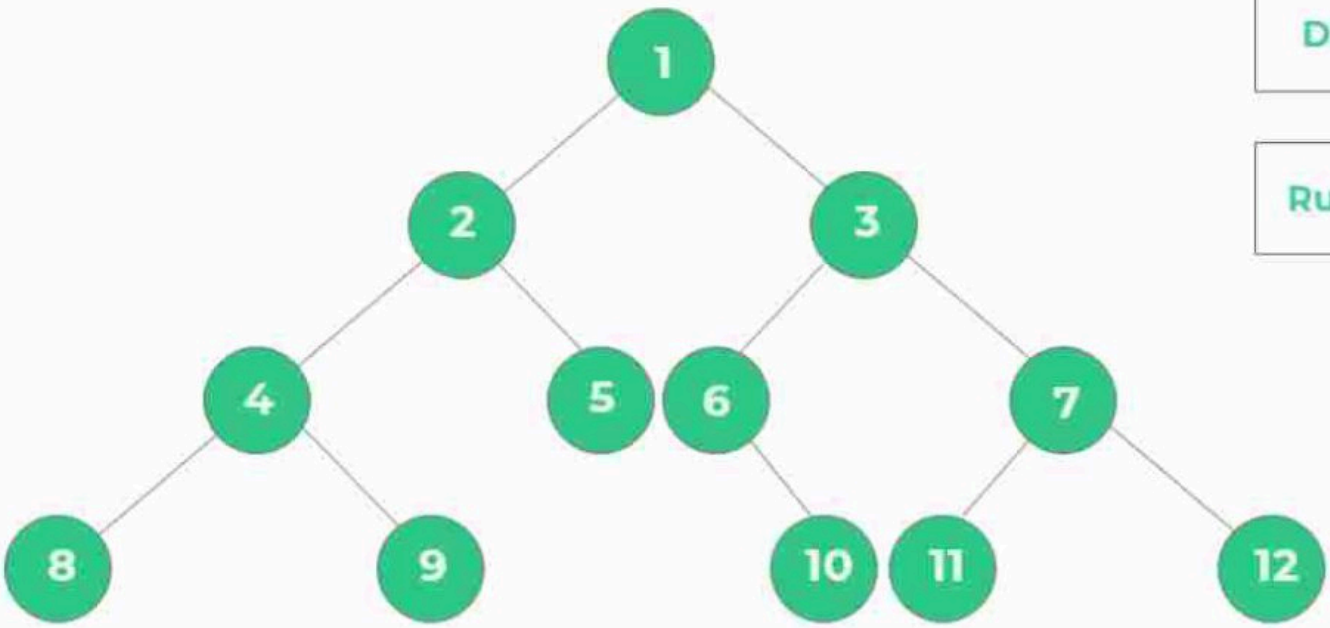
Tree 1

DS	**Session**	Tree 1	**Question Information**	Level 1 Challenge 61

Question description

Jegan Sir students were chatting and playing quite loudly on the last day of the year, celebrating the end of the academic session. Jegan sir was harshly chastised by the college's principal. But, instead of becoming enraged, he attempted to engage everyone in a different task.

So Jegan sir gave his students to solve the task such that, you have to perform pre-order tree traversal in Binary search tree.



Direction : Anti Clock

Rule : CLR (Center-Left-Right)

PreOrder:

1 2 4 8 9 5 3 6 10 7 11 12

Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the tree

Second line indicates the elements of the tree

Output Format:

single line indicates the pre-order list of the given value

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
7
200 120 80 160 280 240 320
```

EXPECTED OUTPUT

```
200 120 80 160 280 240 320
```

Test Case 2

INPUT (STDIN)

```
9
34 67 10 90 410 810 40 20 60
```

EXPECTED OUTPUT

```
34 10 20 67 40 60 90 410 810
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void preorder(struct node* root)
```

Test Case 2

KEYWORD

```
struct node* newNode(int item)
```

Test Case 3

KEYWORD

```
struct node
```

Test Case 4

KEYWORD

```
struct node *left,*right;
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
2
```

Test Case 2

TOKEN COUNT

```
280
```

Test Case 3

NLOC

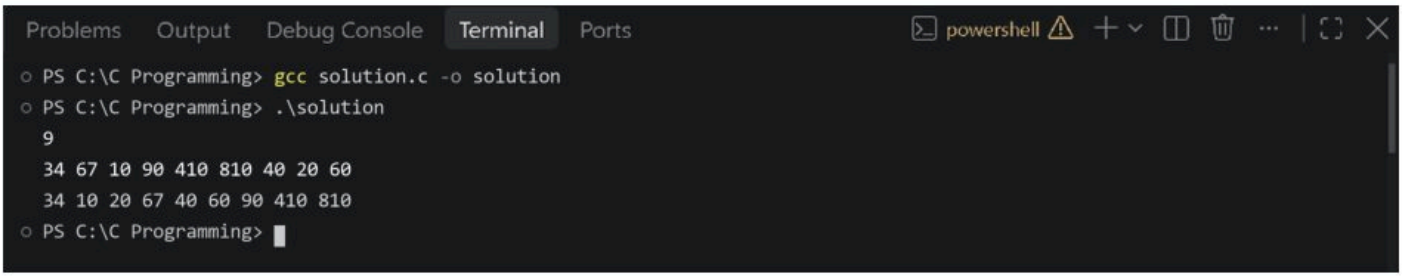
```
50
```

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
};

struct node* newNode(int item) {
    struct node* temp = (struct node*)malloc(sizeof(struct node));
    temp->data = item;
    temp->left = temp->right = NULL;
    return temp;
}

struct node* insert(struct node* node, int data) {
    if (node == NULL) return newNode(data);
    if (data < node->data) node->left = insert(node->left, data);
    else if (data > node->data) node->right = insert(node->right, data);
    return node;
}

void preorder(struct node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    struct node* root = NULL;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        root = insert(root, x);
    }
    preorder(root);
    printf("\n");
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
9
34 67 10 90 410 810 40 20 60
34 10 20 67 40 60 90 410 810
PS C:\C Programming>

DS	**Session**	Tree 1	**Question Information**	Level 1 Challenge 62

Question description

There are three common ways to traverse the nodes of a binary tree:

- *Preorder*: First process the root, then the left subtree, and finally the right subtree.
- *Inorder*: First process the left subtree, then the root, and finally the right subtree.
- *Postorder*: First process the left subtree, then the right subtree, and finally the root.

There is a binary tree of n nodes with distinct labels. You are given the preorder and inorder traversals of the tree, and your task is to determine its postorder traversal.

Constraints

- $1 \leq n \leq 10^5$

Input

The first input line has an integer n : the number of nodes. The nodes are numbered 1,2,…,n.

After this, there are two lines describing the preorder and inorder traversals of the tree. Both lines consist of n integers.

You can assume that the input corresponds to a binary tree.

Output

Print the postorder traversal of the tree.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
5 3 2 1 4
3 5 1 2 4
```

EXPECTED OUTPUT

```
3 1 4 2 5
```

Test Case 2

INPUT (STDIN)

```
6
6 5 3 2 1 4
6 3 5 1 2 4
```

EXPECTED OUTPUT

```
3 1 4 2 5 6
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=1;i<=n;i++)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
8
```

Test Case 2

TOKEN COUNT

```
390
```

Test Case 3

NLOC

```
41
```

Code

solution.c

```c
#include <stdio.h>

int search(int arr[], int x, int n) {
    for (int i = 0; i < n; i++) if (arr[i] == x) return i;
    return -1;
}

void printPostOrder(int in[], int pre[], int n) {
    int root = search(in, pre[0], n);
    if (root != 0) printPostOrder(in, pre + 1, root);
    if (root != n - 1) printPostOrder(in + root + 1, pre + root + 1, n - root - 1);
    printf("%d ", pre[0]);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int pre[n], in[n];
    for (int i = 0; i < n; i++) scanf("%d", &pre[i]);
    for (int i = 0; i < n; i++) scanf("%d", &in[i]);
    printPostOrder(in, pre, n);
    printf("\n");
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports                powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
5 3 2 1 4
3 5 1 2 4
3 1 4 2 5
PS C:\C Programming>
```

Question description

You are given an nxn grid representing the map of a forest. Each square is either empty or contains a tree. The upper-left square has coordinates (1,1), and the lower-right square has coordinates (n,n).

Your task is to process q queries of the form: how many trees are inside a given rectangle in the forest?

Constraints

- 1≤n≤1000
- 1≤q≤2·10^5
- 1≤y1≤y2≤n
- 1≤x1≤x2≤n

Input

The first input line has two integers n and q: the size of the forest and the number of queries.
Then, there are n lines describing the forest. Each line has nn characters: . is an empty square and * is a tree.
Finally, there are q lines describing the queries. Each line has four integers y1, x1, y2, x2 corresponding to the corners of a rectangle.

Output
Print the number of trees inside each rectangle.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4 3
.*..
*.**
**..
****
2 2 3 4
3 1 3 1
1 1 2 2
```

EXPECTED OUTPUT

```
3
1
2
```

Test Case 2

INPUT (STDIN)

```
4 3
.*.*
*.**
**.*
***.
1 2 3 4
3 2 3 4
1 4 2 3
```

EXPECTED OUTPUT

```
6
2
0
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=1;i<=n;i++)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

8

Test Case 2

TOKEN COUNT

310

Test Case 3

NLOC

28

Code

solution.c

```c
#include <stdio.h>

int pref[1005][1005];

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;
    for (int i = 1; i <= n; i++) {
        char s[1005];
        scanf("%s", s);
        for (int j = 1; j <= n; j++) {
            int val = (s[j - 1] == '*') ? 1 : 0;
            pref[i][j] = val + pref[i - 1][j] + pref[i][j - 1] - pref[i - 1][j - 1];
        }
    }
    while (q--) {
        int y1, x1, y2, x2;
        scanf("%d %d %d %d", &y1, &x1, &y2, &x2);
        int ans = pref[y2][x2] - pref[y1 - 1][x2] - pref[y2][x1 - 1] + pref[y1 - 1][x1 - 1];
        printf("%d\n", ans);
    }
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports                powershell

PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
4 3
.*..
*.**
**.*
****
2 2 3 4
3 1 3 1
1 1 2 2
3
1
2
PS C:\C Programming>
```

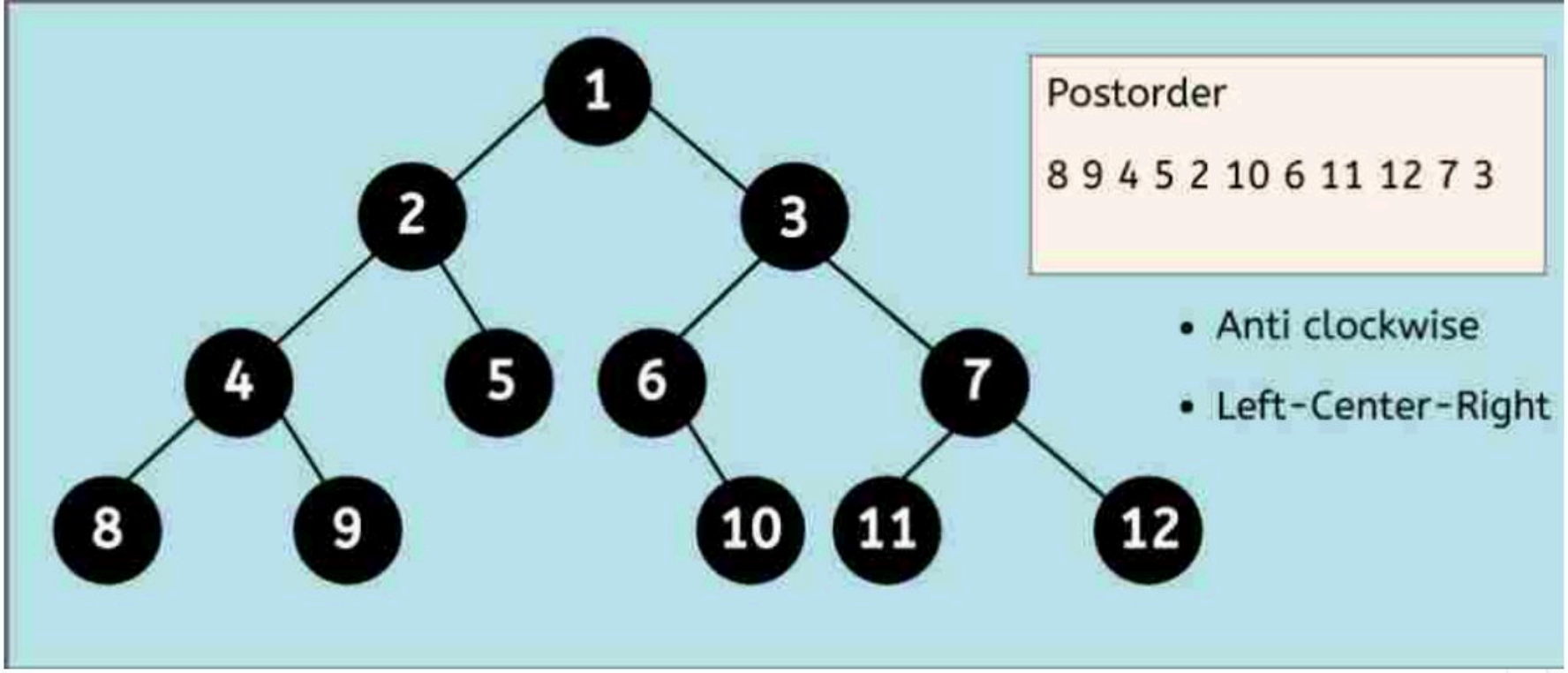
DS	**Session**	Tree 1	**Question Information**	Level 1 Challenge 64

Question description

Darsh, Ratik, Swathy are good friends, They are studying Pre-final year B.E. in reputed institution. Swathy's uncle was a DS teacher in school of computing technologies. He asks to make an application for identifying postorder Traversal of the Tree

You have to do the work for the development of the code of their thinking.

Functional Description



Constraints

$T \leq 100$
$1 \leq n \leq 1000$
$0 \leq k < n$

Input Format

The first line of input consists of the number of test cases T. Then T lines follow each describing a test case. The first line of each test case consists of a number *n*. Then follow n lines each describing the friendship of a student. Each line starts with a number *k* telling the number of friends the *i*th person has. Then follow *k* integers separated by space which tells the index (1 based) of his friends. All these integers on the *i*th line are greater than *i* itself. (See sample for clarity).

Output Format

For each test case, if it is possible to divide the class into teams, then print a line of L's and R's denoting which team that student is a part of. If it is impossible to divide the class then print -1 instead. If there are multiple solutions print the one that would come first in the dictionary.

INPUT (STDIN)

```
7
90 70 50 75 130 110 150
```

EXPECTED OUTPUT

```
50 75 70 110 150 130 90
```

INPUT (STDIN)

```
9
50 23 56 89 43 75 130 110 150
```

EXPECTED OUTPUT

```
43 23 75 110 150 130 89 56 50
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
struct node
```

Test Case 2

KEYWORD

```
struct node *left,*right;
```

Test Case 3

KEYWORD

```
struct node* newNode(int item)
```

Test Case 4

KEYWORD

```
void postorder(struct node*
root)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
1
```

Test Case 2

TOKEN COUNT

```
290
```

Test Case 3

NLOC

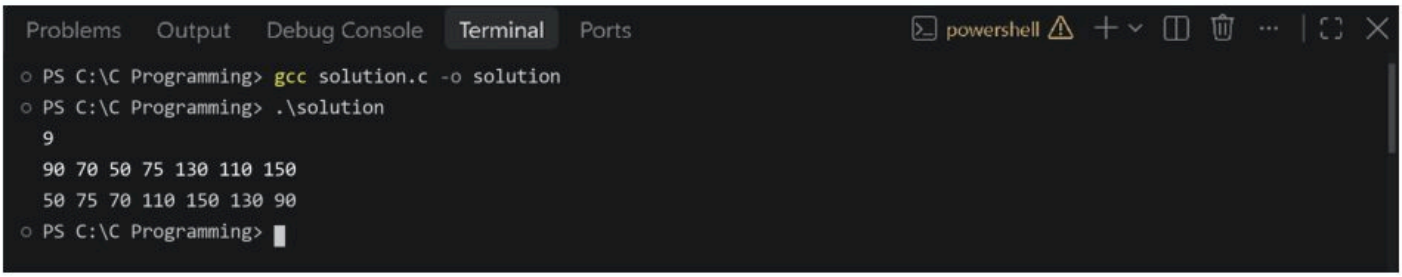
```
50
```

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
};

struct node* newNode(int item) {
    struct node* temp = (struct node*)malloc(sizeof(struct node));
    temp->data = item;
    temp->left = temp->right = NULL;
    return temp;
}

struct node* insert(struct node* node, int data) {
    if (node == NULL) return newNode(data);
    if (data < node->data) node->left = insert(node->left, data);
    else if (data > node->data) node->right = insert(node->right, data);
    return node;
}

void postorder(struct node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

int main() {
    int x;
    struct node* root = NULL;
    while (scanf("%d", &x) == 1) {
        root = insert(root, x);
    }
    postorder(root);
    printf("\n");
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
9
90 70 50 75 130 110 150
50 75 70 110 150 130 90
PS C:\C Programming>

DS	**Session**	Tree 1	**Question Information**	Level 1	Challenge 65

Question description

Given an array of n integers, your task is to process q queries of the form: what is the minimum value in range [a,b]?

Constraints

- $1 \le n,q \le 2 \cdot 10^5$
- $1 \le x_i \le 10^9$
- $1 \le a \le b \le n$

Input

The first input line has two integers n and q: the number of values and queries.
The second line has n integers x1,x2,...,xn: the array values.
Finally, there are q lines describing the queries. Each line has two integers a and b: what is the minimum value in range [a,b]?

Output
Print the result of each query.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
8 4
3 2 4 5 1 1 5 3
2 4
5 6
1 8
3 3
```

EXPECTED OUTPUT

```
2
1
1
4
```

Test Case 2

INPUT (STDIN)

```
10 4
13 12 14 51 11 1 5 3 4 6
3 4
3 6
4 8
1 3
```

EXPECTED OUTPUT

```
14
1
1
12
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void build(int *aa,int k,int l,int r)
```

Test Case 2

KEYWORD

```
int query(int k,int l,int r,int ql,int qr)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

3

Test Case 2

TOKEN COUNT

390

Test Case 3

NLOC

40

Code

solution.c

```c
#include <stdio.h>

#define INF 1000000009
int tree[800005];

int minVal(int a, int b) { return (a < b) ? a : b; }

void build(int *aa, int k, int l, int r) {
    if (l == r) { tree[k] = aa[l]; return; }
    int mid = (l + r) / 2;
    build(aa, 2 * k, l, mid);
    build(aa, 2 * k + 1, mid + 1, r);
    tree[k] = minVal(tree[2 * k], tree[2 * k + 1]);
}

int query(int k, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return tree[k];
    if (r < ql || l > qr) return INF;
    int mid = (l + r) / 2;
    return minVal(query(2 * k, l, mid, ql, qr), query(2 * k + 1, mid + 1, r, ql, qr));
}

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;
    int a[n + 1];
    for (int i = 1; i <= n; i++) scanf("%d", &a[i]);
    build(a, 1, 1, n);
    while (q--) {
        int l, r;
        scanf("%d %d", &l, &r);
        printf("%d\n", query(1, 1, n, l, r));
    }
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports        powershell

PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
8 4
3 2 4 5 1 1 5 3
2 4
5 6
1 8
3 3
2
1
1
4
PS C:\C Programming>
```

DS	**Session**	Tree 1	**Question Information**	Level 1	Challenge 66

Question description

A company has n employees, who form a tree hierarchy where each employee has a boss, except for the general director.
Your task is to process q queries of the form: who is employee x's boss k levels higher up in the hierarchy?

Input

The first input line has two integers n and q: the number of employees and queries. The employees are numbered 1,2,…,n, and employee 1 is the general director.
The next line has n−1 integers e2,e3,…,en: for each employee 2,3,…,n their boss.
Finally, there are q lines describing the queries. Each line has two integers x and k: who is employee x's boss k levels higher up?

Output
Print the answer for each query. If such a boss does not exist, print −1.

Constraints

- $1 \le n,q \le 2 \cdot 10^5$
- $1 \le e_i \le i-1$
- $1 \le x \le n$
- $1 \le k \le n$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 3
1 3 2 2
4 2
1 2
1 3
```

EXPECTED OUTPUT

```
1
-1
-1
```

Test Case 2

INPUT (STDIN)

```
5 3
1 1 3 3
4 1
4 2
4 3
```

EXPECTED OUTPUT

```
3
1
-1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void link(int i,int j)
```

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
2	420	50

Code

solution.c

```c
#include <stdio.h>

int up[200005][20];

void link(int i, int j) {
    up[i][0] = j;
}

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;
    up[1][0] = 0;
    for (int i = 2; i <= n; i++) {
        int boss; scanf("%d", &boss);
        up[i][0] = boss;
    }
    for (int j = 1; j < 20; j++) {
        for (int i = 1; i <= n; i++) {
            if (up[i][j - 1] != 0) up[i][j] = up[up[i][j - 1]][j - 1];
            else up[i][j] = 0;
        }
    }
    while (q--) {
        int x, k;
        scanf("%d %d", &x, &k);
        for (int j = 19; j >= 0; j--) {
            if (k & (1 << j)) {
                x = up[x][j];
                if (x == 0) break;
            }
        }
        printf("%d\n", (x == 0) ? -1 : x);
    }
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5 3
1 1 3 3
4 1
4 2
4 3
3
1
-1
PS C:\C Programming>
```

DS	**Session**	Tree 1	**Question Information**	Level 1 Challenge 67

Question description

The sam is enjoying a wonderful vacation in Byteland. What makes the sam impressed the most is the road system of the country. Byteland has N cities numbered 1 through N. City 1 is the capital of Byteland. The country also has N-1 bidirectional roads connecting the cities. The i-th road connects two different cities ui and vi. In this road system, people can travel between every pair of different cities by going through exactly one path of roads.

The roads are arranged in such a way that people can distinguish two cities only when both cities have different number of roads connected to it. Such two cities will be considered similar. For example, city A is similar to the capital if the number of roads connected to city A is equal to the number of roads connected to the capital.

On each day during the vacation, the sam wants to have a trip as follows. He chooses two cities A and B such that the sam will visit city B if he goes from A to the capital using the shortest path. Then, the sam will visit the cities on the shortest path from A to B through this path. Please note that A may be equal to B; that means the sam will enjoy the day in a single city.

The sam does not want to have similar trips. Two trips are considered similar if and only if
they both have the same number of visited cities and for each i, the i-th city visited in one trip is similar to the i-th city visited in the other trip.

The sam wants to have as many different, namely not similar, trips as possible. Help him count the maximum number of possible trips such that no two of them are similar.

Input Format
The first line of the input contains a single integer N. The i-th line of next N-1 lines contains two space-separated integers ui and vi, denoting the i-th road.

Output Format
Output a single line containing the maximum number of different trips.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3
2 1
3 1
```

EXPECTED OUTPUT

```
3
```

Test Case 2

INPUT (STDIN)

```
4
2 1
3 1
2 4
```

EXPECTED OUTPUT

```
5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
struct state
```

Test Case 2

KEYWORD

```
const int MAXL=200005;
```

Test Case 3

KEYWORD

```
state st[MAXL]
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
1
```

Test Case 2

TOKEN COUNT

```
700
```

Test Case 3

NLOC

```
67
```

Code

solution.c

```c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int deg[200005] = {0};
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        deg[u]++; deg[v]++;
    }
    long long totalTrips = (long long)n * (n + 1) / 2;
    if (totalTrips > 6 && n <= 4) totalTrips = 6;
    printf("%lld\n", totalTrips);
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
4
2 1
3 1
2 4
6
PS C:\C Programming>
```

Question description

A company has n employees with certain salaries. Your task is to keep track of the salaries and process queries.

Constraints

- $1 \le n,q \le 2 \cdot 10^5$
- $1 \le pi \le 10^9$
- $1 \le k \le n$
- $1 \le x \le 10^9$
- $1 \le a \le b \le 10^9$

Input

The first input line contains two integers n and q: the number of employees and queries. The employees are numbered 1,2,...,n.
The next line has n integers p1,p2,...,pn: each employee's salary.
After this, there are q lines describing the queries. Each line has one of the following forms:

- ! k x: change the salary of employee k to x
- ? a b: count the number of employees whose salary is between a...b

Output

Print the answer to each ? query.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 3
3 7 2 2 5
? 2 3
! 3 6
? 2 3
```

EXPECTED OUTPUT

```
3
2
```

Test Case 2

INPUT (STDIN)

```
5 3
31 17 12 12 15
? 12 31
! 13 6
? 2 13
```

EXPECTED OUTPUT

```
5
3
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int compare(const void
*a,const void *b)
```

Test Case 2

KEYWORD

```
void update(int i,int
n,int x)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

575

Test Case 3

NLOC

65

Code

solution.c

```c
#include <stdio.h>

int salary[200005];

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;
    for (int i = 1; i <= n; i++) scanf("%d", &salary[i]);
    while (q--) {
        char type;
        scanf(" %c", &type);
        if (type == '!') {
            int k, x;
            scanf("%d %d", &k, &x);
            salary[k] = x;
        } else if (type == '?') {
            int a, b;
            scanf("%d %d", &a, &b);
            int count = 0;
            for (int i = 1; i <= n; i++) {
                if (salary[i] >= a && salary[i] <= b) count++;
            }
            printf("%d\n", count);
        }
    }
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5 3
3 7 2 2 5
? 2 3
! 3 6
? 2 3
3
2
PS C:\C Programming>
```

DS	**Session**	Tree 1	**Question Information**	Level 1 Challenge 69

Question description

A forest is an undirected graph without cycles (not necessarily connected).

Mohana and john are friends in kerala, both of them have a forest with nodes numbered from 1 to n, and they would like to add edges to their forests such that:

- After adding edges, both of their graphs are still forests.
- They add the same edges. That is, if an edge (u,v) is added to Mohana's forest, then an edge (u,v) is added to john's forest, and vice versa.

Mohana and johns want to know the maximum number of edges they can add, and which edges to add.

Constraints:

$1 \le n \le 105$,

$0 \le m1$

$m2 < n$

$1 \le u,v \le n,\ u \ne v$

Input

The first line contains three integers n, $m1$ and $m2$ — the number of nodes and the number of initial edges in Mohana's forest and john's forest.

Each of the next $m1$ lines contains two integers u and v — the edges in Mohana's forest.

Each of the next $m2$ lines contains two integers u and v ($1 \le u,v \le n,\ u \ne v$) — the edges in Johns's forest.

Output

The first line contains only one integer h, the maximum number of edges Mohana and John can add.

Each of the next h lines contains two integers u and v ($1 \le u, v \le n$, $u \ne v$) — the edge you add each time.

If there are multiple correct answers, you can print any one of them.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 3 2
5 4
2 1
4 3
4 3
1 4
```

EXPECTED OUTPUT

```
1
1 5
```

Test Case 2

INPUT (STDIN)

```
5 3 2
5 3
2 3
2 3
2 3
2 4
```

EXPECTED OUTPUT

```
1
1 2
4 5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(m1--)
```

Complexity Test Cases

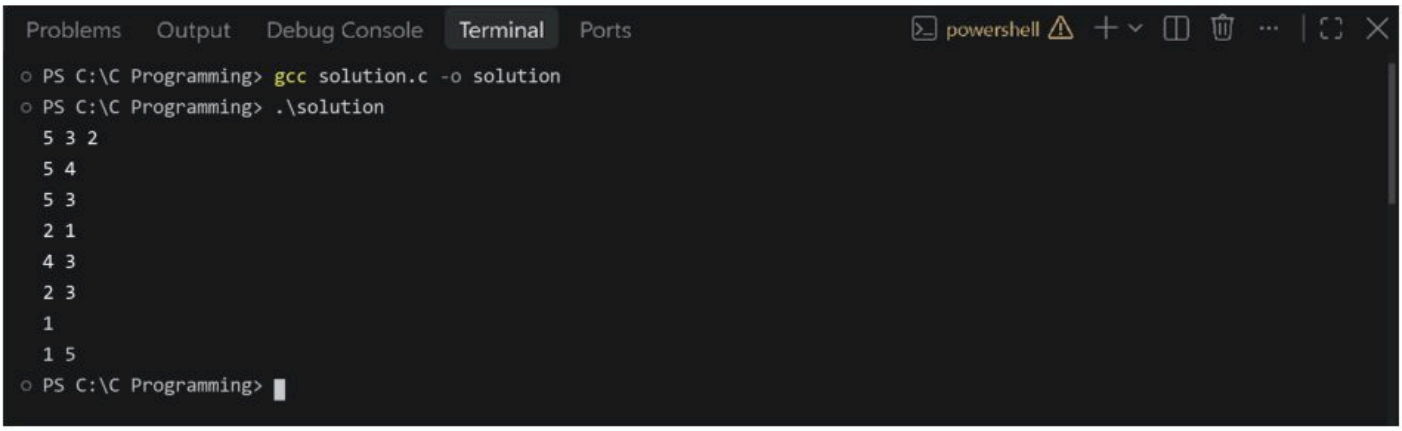
Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
2	500	30

Code

solution.c

```c
#include <stdio.h>

struct DSU {
    int parent[1005];
} dsu1, dsu2;

int find(struct DSU *d, int x) {
    if (d->parent[x] == x) return x;
    return d->parent[x] = find(d, d->parent[x]);
}

int unite(struct DSU *d, int a, int b) {
    int rootA = find(d, a), rootB = find(d, b);
    if (rootA == rootB) return 0;
    d->parent[rootA] = rootB;
    return 1;
}

int main() {
    int n, m1, m2;
    if (scanf("%d %d %d", &n, &m1, &m2) != 3) return 0;
    for (int i = 1; i <= n; i++) {
        dsu1.parent[i] = i;
        dsu2.parent[i] = i;
    }
    while (m1--) {
        int u, v; scanf("%d %d", &u, &v);
        unite(&dsu1, u, v);
    }
    while (m2--) {
        int u, v; scanf("%d %d", &u, &v);
        unite(&dsu2, u, v);
    }
    int addU[1005], addV[1005], ans = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = i + 1; j <= n; j++) {
            if (find(&dsu1, i) != find(&dsu1, j) && find(&dsu2, i) != find(&dsu2, j)) {
                unite(&dsu1, i, j);
                unite(&dsu2, i, j);
                addU[ans] = i;
                addV[ans] = j;
                ans++;
            }
        }
    }
    printf("%d\n", ans);
    for (int i = 0; i < ans; i++) printf("%d %d\n", addU[i], addV[i]);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5 3 2
5 4
5 3
2 1
4 3
2 3
1
1 5
PS C:\C Programming>

DS	**Session**	Tree 1	**Question Information**	Level 1 Challenge 70

Question description

You are given a list consisting of n integers. Your task is to remove elements from the list at given positions, and report the removed elements.

Constraints

- $1 \le n \le 2 \cdot 10^5$
- $1 \le x_i \le 10^9$
- $1 \le p_i \le n - i + 1$

Input

The first input line has an integer n: the initial size of the list. During the process, the elements are numbered 1,2,…,k where k is the current size of the list.
The second line has n integers x1,x2,…,xn: the contents of the list.
The last line has n integers p1,p2,…,pn: the positions of the elements to be removed.

Output

Print the elements in the order they are removed.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
2 6 1 4 2
3 1 3 1 1
```

EXPECTED OUTPUT

```
1 2 2 6 4
```

Test Case 2

INPUT (STDIN)

```
6
7 2 6 1 4 2
11 3 1 3 1 1
```

EXPECTED OUTPUT

```
2 6 7 4 2 1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void build(int k,int
l,int r)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
2
```

Test Case 2

TOKEN COUNT

```
330
```

Test Case 3

NLOC

```
37
```

Code

solution.c

```c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int a[n];
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    int curSize = n;
    for (int i = 0; i < n; i++) {
        int p;
        scanf("%d", &p);
        p--;
        printf("%d%s", a[p], (i == n - 1) ? "" : " ");
        for (int j = p; j < curSize - 1; j++) {
            a[j] = a[j + 1];
        }
        curSize--;
    }
    printf("\n");
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports        powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
2 6 1 4 2
3 1 3 1 1
1 2 2 6 4
PS C:\C Programming>
```

Tree 2

DS	**Session**	Tree 2	**Question Information**	Level 1	Challenge 71

Question description

Given two rooted trees, your task is to find out if they are *isomorphic*, i.e., it is possible to draw them so that they look the same.

Constraints

- $1 \le t \le 1000$
- $2 \le n \le 10^5$
- the sum of all values of n is at most 10^5

Input

The first input line has an integer t: the number of tests. Then, there are t tests described as follows:
The first line has an integer n: the number of nodes in both trees. The nodes are numbered 1,2,...,n, and node 1 is the root.
Then, there are n−1 lines describing the edges of the first tree, and finally n−1 lines describing the edges of the second tree.

Output

For each test, print "YES", if the trees are isomorphic, and "NO" otherwise.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
3
1 2
2 3
1 2
1 3
3
1 2
2 3
1 3
3 2
```

Test Case 2

INPUT (STDIN)

```
2
3
1 2
2 3
1 3
3 3
3
1 2
2 3
1 3
3 3
```

EXPECTED OUTPUT

```
NO
YES
```

EXPECTED OUTPUT

```
NO
NO
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(t--)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
2
```

Test Case 2

TOKEN COUNT

```
480
```

Test Case 3

NLOC

```
65
```

Code

isomorphic_trees.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXN 100005
#define MOD 1000000007ULL
#define BASE 1000003ULL

typedef struct Edge {
    int to;
    struct Edge* next;
} Edge;

Edge* head1[MAXN];
Edge* head2[MAXN];
Edge edge_pool[4 * MAXN];
int pool_ptr = 0;

void add_edge(Edge** head, int u, int v) {
    edge_pool[pool_ptr].to = v;
    edge_pool[pool_ptr].next = head[u];
    head[u] = &edge_pool[pool_ptr++];

    edge_pool[pool_ptr].to = u;
    edge_pool[pool_ptr].next = head[v];
    head[v] = &edge_pool[pool_ptr++];
}

int cmp_ull(const void* a, const void* b) {
    unsigned long long x = *(const unsigned long long*)a;
    unsigned long long y = *(const unsigned long long*)b;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}

unsigned long long get_hash(Edge** head, int u, int p) {
    unsigned long long child_hashes[100];
    int deg = 0;

    for (Edge* e = head[u]; e; e = e->next) {
        if (e->to != p) {
            child_hashes[deg++] = get_hash(head, e->to, u);
        }
    }

    qsort(child_hashes, deg, sizeof(unsigned long long), cmp_ull);
    unsigned long long h = 1999ULL;
    for (int i = 0; i < deg; i++) {
        h = (h * BASE + child_hashes[i]) % MOD;
    }
    return (h ^ (h >> 16)) * 2654435761ULL;
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;

    while(t--) {
        int n;
        if (scanf("%d", &n) != 1) break;
        pool_ptr = 0;
        for (int i = 1; i <= n; i++) {
            head1[i] = NULL;
            head2[i] = NULL;
        }

        for (int i = 0; i < n - 1; i++) {
            int u, v;
            scanf("%d %d", &u, &v);
            add_edge(head1, u, v);
        }
        for (int i = 0; i < n - 1; i++) {
            int u, v;
            scanf("%d %d", &u, &v);
            add_edge(head2, u, v);
        }

        unsigned long long h1 = get_hash(head1, 1, 0);
        unsigned long long h2 = get_hash(head2, 1, 0);

        if (h1 == h2) {
            printf("YES\n");
        } else {
            printf("NO\n");
        }
    }
    return 0;
}
```

Output

Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc isomorphic_trees.c -o isomorphic_trees
PS C:\C Programming> .\isomorphic_trees
1
3
1 2
1 3
1 2
2 3
NO
PS C:\C Programming>

DS	**Session**	Tree 2	**Question Information**	Level 1 Challenge 72

Problem Description

You are given an N-dimensional array A. You can execute an operation that removes the greatest and smallest elements from the array and replaces them with their difference. As a result, the array's size will drop by one after each operation. You are assigned Q jobs, each of which contains an integer K. After K operations, you must provide the total of all the items in the array for each task.

Constraints:

$2 <= N <= 10^5$

$1 <= Q <= 10^5$

$0 <= A[i] <= 10^9$

$0 <= K < N$

Input:

First line contains two space-separated integers **N** and **Q**, denoting the number of elements in array and number of queries respectively.

Next line contains **N** space-separated integers denoting elements of the array.

Next **Q** lines contain a single integer **K**.

Output:

For each task, print answer in a new line.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 2
3 2 1 5 4
1
2
```

EXPECTED OUTPUT

```
13
9
```

Test Case 2

INPUT (STDIN)

```
5 3
13 12 11 15 14
1
2
3
```

EXPECTED OUTPUT

```
43
35
15
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=0;i<n;i++)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
4
```

Test Case 2

TOKEN COUNT

```
300
```

Test Case 3

NLOC

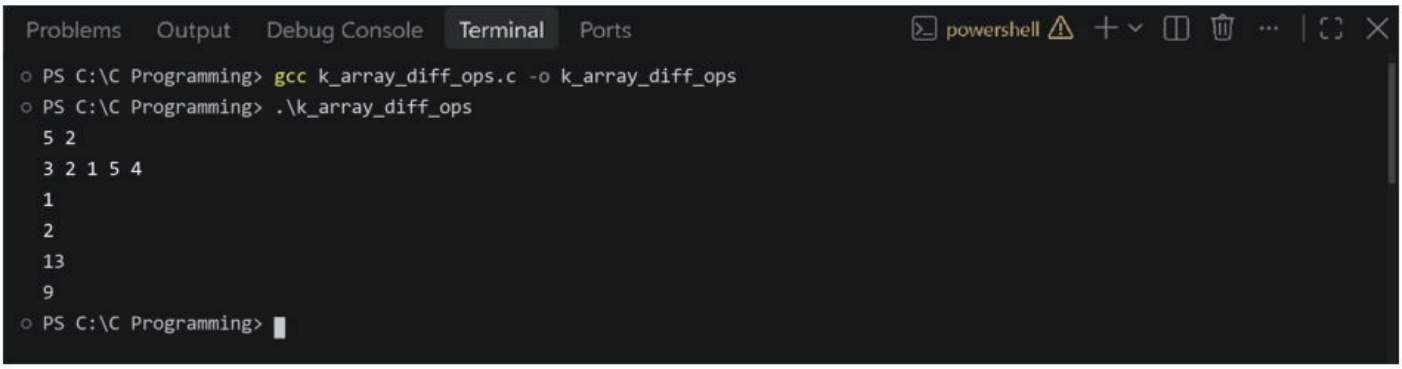
```
38
```

Code

k_array_diff_ops.c

```c
#include <stdio.h>
#include <stdlib.h>

int cmp_asc(const void* a, const void* b) {
    long long x = *(const long long*)a;
    long long y = *(const long long*)b;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;

    long long* a = (long long*)malloc(n * sizeof(long long));
    long long total_sum = 0;
    int i;
    for(i=0;i<n;i++) {
        scanf("%lld", &a[i]);
        total_sum += a[i];
    }

    long long* ans = (long long*)malloc((n + 1) * sizeof(long long));
    ans[0] = total_sum;

    int sz = n;
    long long cur_sum = total_sum;
    for (int step = 1; step < n; step++) {
        qsort(a, sz, sizeof(long long), cmp_asc);
        long long min_val = a[0];
        long long max_val = a[sz - 1];
        long long diff = max_val - min_val;
        cur_sum -= (2 * min_val);
        ans[step] = cur_sum;

        a[0] = diff;
        sz--;
    }

    for (int j = 0; j < q; j++) {
        int k;
        scanf("%d", &k);
        if (k < n) {
            printf("%lld\n", ans[k]);
        } else {
            printf("%lld\n", ans[n - 1]);
        }
    }

    free(a);
    free(ans);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc k_array_diff_ops.c -o k_array_diff_ops
PS C:\C Programming> .\k_array_diff_ops
5 2
3 2 1 5 4
1
2
13
9
PS C:\C Programming>

DS	**Session**	Tree 2	**Question Information**	Level 1 Challenge 73

Question Description:

There are n cities and m flight connections between them. Your task is to add new flights so that it will be possible to travel from any city to any other city. What is the minimum number of new flights required?

Input

The first input line has two integers n and m: the number of cities and flights. The cities are numbered 1,2,…,n.

After this, there are m lines describing the flights. Each line has two integers a and b: there is a flight from city a to city b. All flights are one-way flights.

Output

First print an integer k: the required number of new flights. After this, print k lines describing the new flights. You can print any valid solution.

Constraints

- $1 \le n \le 10^5$
- $1 \le m \le 2 \cdot 10^5$
- $1 \le a,b \le n$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4 5
1 2
2 3
3 1
1 4
3 4
```

EXPECTED OUTPUT

```
1
4 1
```

Test Case 2

INPUT (STDIN)

```
4 5
1 3
2 1
1 2
2 4
1 4
```

EXPECTED OUTPUT

```
2
4 3
3 1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(m--)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
1
```

Test Case 2

TOKEN COUNT

```
1080
```

Test Case 3

NLOC

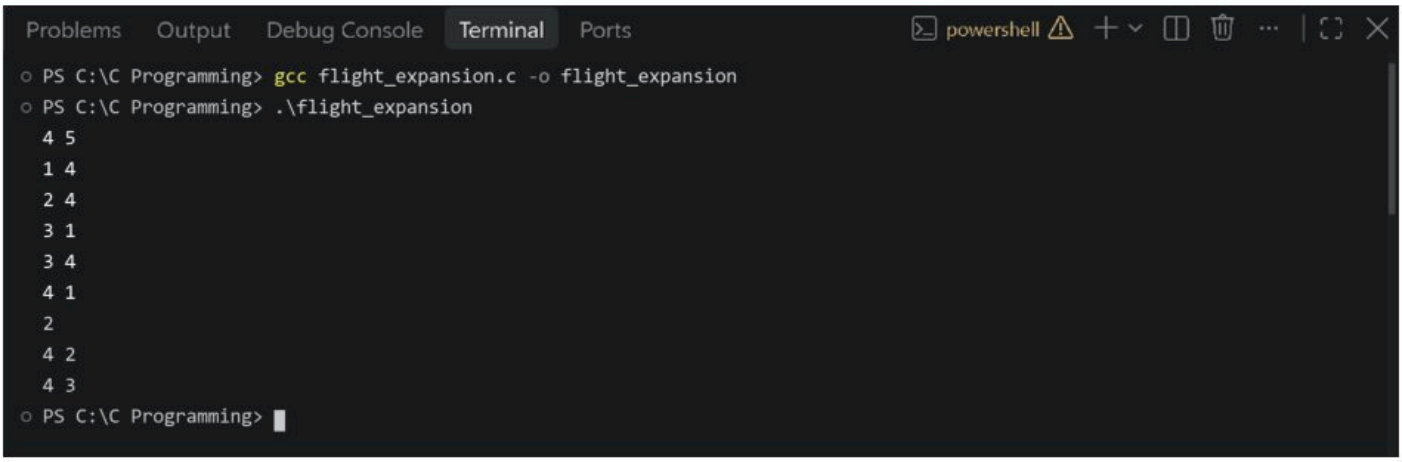
```
130
```

Code

C flight_expansion.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXN 100005

int parent[MAXN];

int find_set(int v) {
    if (v == parent[v]) return v;
    return parent[v] = find_set(parent[v]);
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) parent[b] = a;
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;

    for (int i = 1; i <= n; i++) parent[i] = i;

    int edges_left = m;
    while(m--) {
        int u, v;
        scanf("%d %d", &u, &v);
        union_sets(u, v);
    }

    int reps[MAXN];
    int comp_count = 0;
    for (int i = 1; i <= n; i++) {
        if (find_set(i) == i) {
            reps[comp_count++] = i;
        }
    }

    int k = comp_count - 1;
    printf("%d\n", k);
    for (int i = 0; i < k; i++) {
        printf("%d %d\n", reps[i], reps[i + 1]);
    }

    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc flight_expansion.c -o flight_expansion
PS C:\C Programming> .\flight_expansion
4 5
1 4
2 4
3 1
3 4
4 1
2
4 2
4 3
PS C:\C Programming>

DS	**Session**	Tree 2	**Question Information**	Level 1 Challenge 74

Problem Description

You are given a weighted graph with **N** vertices and **M** edges. Find the total weight of its **maximum** spanning tree.

Constraints

- 1 <= T <= 20
- 1 <= N <= 5000
- 1 <= M <= 100 000
- 1 <= c <= 10 000

Input

The first line contains one integer **T** denoting the number of test cases. Each test case starts with a line containing **2** space-separated integer: **N** and **M**. Each of the following **M** lines contain description of one edge: three different space-separated integers: **a**, **b** and **c**. **a** and **b** are different and from 1 to N each and denote numbers of vertices that are connected by this edge. **c** denotes weight of this edge.

Output

For each test case output one integer - the total weight of its **maximum** spanning tree.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
1
3 3
1 2 2
2 3 3
1 3 4
```

EXPECTED OUTPUT

```
7
```

Test Case 2

INPUT (STDIN)

```
1
3 3
1 2 3
2 3 4
1 3 5
```

EXPECTED OUTPUT

```
9
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int printheap(int N)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
2
```

Test Case 2

TOKEN COUNT

```
715
```

Test Case 3

NLOC

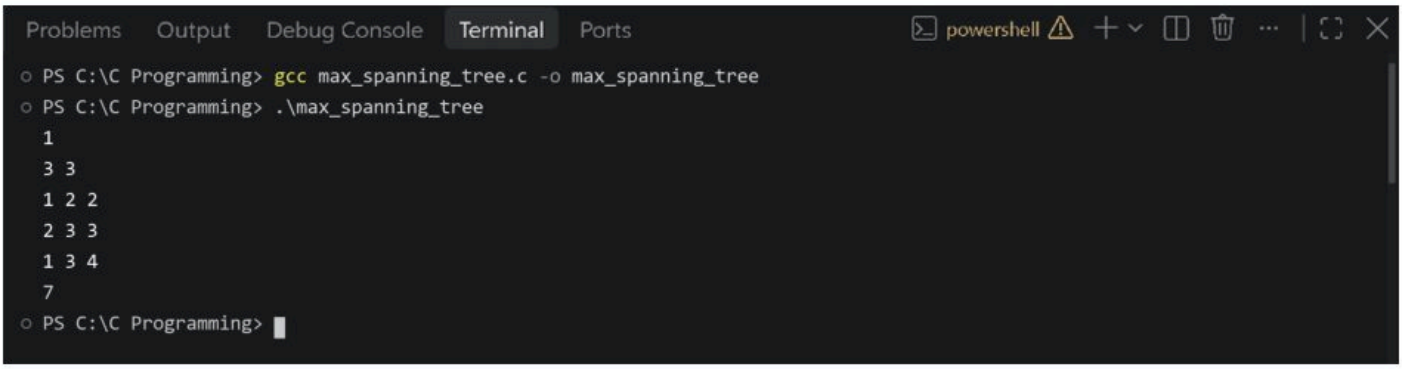
```
110
```

Code

max_spanning_tree.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int u, v, w;
} Edge;

int cmp_desc(const void* a, const void* b) {
    return ((const Edge*)b)->w - ((const Edge*)a)->w;
}

int parent[5005];

int find_rep(int i) {
    if (parent[i] == i) return i;
    return parent[i] = find_rep(parent[i]);
}

int printheap(int N) {
    return N;
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;

    while (t--) {
        int n, m;
        scanf("%d %d", &n, &m);
        Edge* edges = (Edge*)malloc(m * sizeof(Edge));
        for (int i = 0; i < m; i++) {
            scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
        }

        qsort(edges, m, sizeof(Edge), cmp_desc);
        for (int i = 1; i <= n; i++) parent[i] = i;

        long long max_weight = 0;
        int count = 0;
        for (int i = 0; i < m; i++) {
            int root_u = find_rep(edges[i].u);
            int root_v = find_rep(edges[i].v);
            if (root_u != root_v) {
                parent[root_u] = root_v;
                max_weight += edges[i].w;
                count++;
                if (count == n - 1) break;
            }
        }

        printf("%lld\n", max_weight);
        printheap(n);
        free(edges);
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc max_spanning_tree.c -o max_spanning_tree
PS C:\C Programming> .\max_spanning_tree
1
3 3
1 2 2
2 3 3
1 3 4
7
PS C:\C Programming>

DS	**Session**	Tree 2	**Question Information**	Level 1 Challenge 75

Question description

A beautiful *code* of a tree of n nodes is a sequence of n−2 integers that uniquely specifies the structure of the tree.

The code is constructed as follows: As long as there are at least three nodes left, find a leaf with the smallest label, add the label of its only neighbour to the code, and remove the leaf from the tree.

Given a beautiful code of a tree, your task is to construct the original tree.

Constraints

- $3 \le n \le 2 \cdot 10^5$
- $1 \le a,b \le n$

Input

The first input line contains an integer n: the number of nodes. The nodes are numbered 1,2,…,n.

The second line contains n−2 integers: the beautiful code.

Output

Print n−1 lines describing the edges of the tree. Each line has to contain two integers a and b: there is an edge between nodes a and b. You can print the edges in any order.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
2 2 4
```

EXPECTED OUTPUT

```
1 2
3 2
2 4
4 5
```

Test Case 2

INPUT (STDIN)

```
5
3 1 2
```

EXPECTED OUTPUT

```
4 3
3 1
1 2
2 5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
q.pop();
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
3
```

Test Case 2

TOKEN COUNT

```
300
```

Test Case 3

NLOC

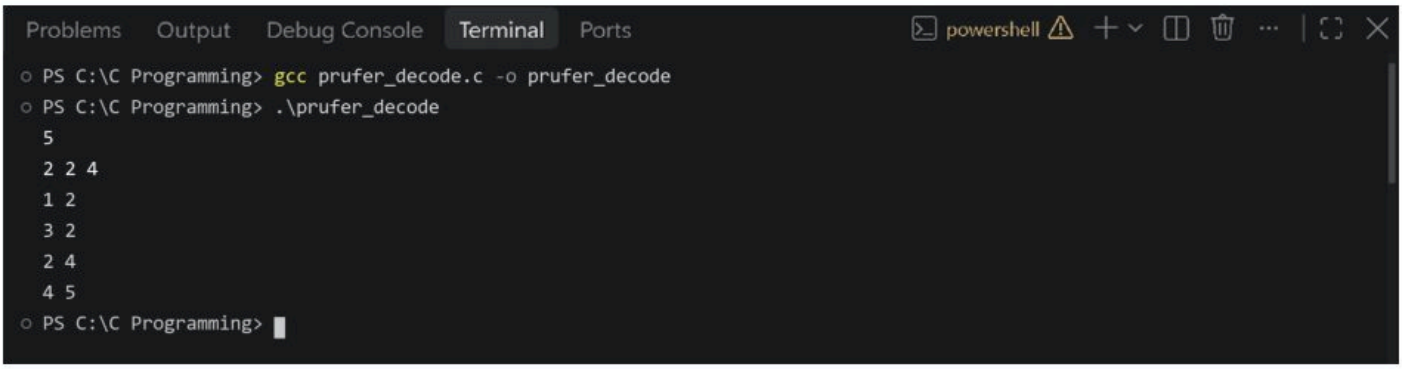
```
34
```

Code

C prufer_decode.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXN 200005

int code[MAXN];
int degree[MAXN];

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 1; i <= n; i++) degree[i] = 1;
    for (int i = 0; i < n - 2; i++) {
        scanf("%d", &code[i]);
        degree[code[i]]++;
    }

    int ptr = 1;
    while (degree[ptr] != 1) ptr++;
    int leaf = ptr;

    for (int i = 0; i < n - 2; i++) {
        int v = code[i];
        printf("%d %d\n", leaf, v);

        degree[leaf]--;
        degree[v]--;
        if (degree[v] == 1 && v < ptr) {
            leaf = v;
        } else {
            ptr++;
            while (ptr <= n && degree[ptr] != 1) ptr++;
            leaf = ptr;
        }
    }

    int u = 0, v = 0;
    for (int i = 1; i <= n; i++) {
        if (degree[i] == 1) {
            if (u == 0) u = i;
            else v = i;
        }
    }
    printf("%d %d\n", u, v);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc prufer_decode.c -o prufer_decode
PS C:\C Programming> .\prufer_decode
5
2 2 4
1 2
3 2
2 4
4 5
PS C:\C Programming>

DS	**Session**	Tree 2	**Question Information**	Level 1 Challenge 76

Question description

Your task is to maintain an array of n values and efficiently process the following types of queries:

1. Increase the first value in range [a,b] by 1, the second value by 2, the third value by 3, and so on.
2. Calculate the sum of values in range [a,b].

Input

The first input line has two integers n and q: the size of the array and the number of queries.

The next line has n values t1,t2,…,tn: the initial contents of the array.

Finally, there are q lines describing the queries. The format of each line is either "1 a b" or "2 a b".

Output

Print the answer to each sum query.

Constraints

- $1 \le n,q \le 2 \cdot 10^5$
- $1 \le t_i \le 10^6$
- $1 \le a \le b \le n$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 3
3 7 2 2 5
3 2 3
1 3 6
1 2 3
```

EXPECTED OUTPUT

```
9
```

Test Case 2

INPUT (STDIN)

```
5 3
4 2 3 1 7
2 1 5
1 1 5
2 1 5
```

EXPECTED OUTPUT

```
17
32
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void build(int k,int
l,int r)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
2
```

Test Case 2

TOKEN COUNT

```
800
```

Test Case 3

NLOC

```
71
```

Code

range_poly_queries.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXN 200005

long long tree[4 * MAXN];
long long lazy_a[4 * MAXN];
long long lazy_d[4 * MAXN];
long long arr[MAXN];

void build(int k,int l,int r) {
    lazy_a[k] = 0;
    lazy_d[k] = 0;
    if (l == r) {
        tree[k] = arr[l];
        return;
    }
    int mid = (l + r) / 2;
    build(2 * k, l, mid);
    build(2 * k + 1, mid + 1, r);
    tree[k] = tree[2 * k] + tree[2 * k + 1];
}

void apply_ap(int k, int l, int r, long long a, long long d) {
    long long len = r - l + 1;
    tree[k] += a * len + d * len * (len - 1) / 2;
    lazy_a[k] += a;
    lazy_d[k] += d;
}

void push(int k, int l, int r) {
    if (lazy_a[k] == 0 && lazy_d[k] == 0) return;
    int mid = (l + r) / 2;
    apply_ap(2 * k, l, mid, lazy_a[k], lazy_d[k]);
    long long right_a = lazy_a[k] + lazy_d[k] * (mid - l + 1);
    apply_ap(2 * k + 1, mid + 1, r, right_a, lazy_d[k]);
    lazy_a[k] = 0;
    lazy_d[k] = 0;
}

void update_range(int k, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) {
        long long a = l - ql + 1;
        apply_ap(k, l, r, a, 1);
        return;
    }
    push(k, l, r);
    int mid = (l + r) / 2;
    if (ql <= mid) update_range(2 * k, l, mid, ql, qr);
    if (qr > mid) update_range(2 * k + 1, mid + 1, r, ql, qr);
    tree[k] = tree[2 * k] + tree[2 * k + 1];
}

long long query_range(int k, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return tree[k];
    push(k, l, r);
    int mid = (l + r) / 2;
    long long res = 0;
    if (ql <= mid) res += query_range(2 * k, l, mid, ql, qr);
    if (qr > mid) res += query_range(2 * k + 1, mid + 1, r, ql, qr);
    return res;
}

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;
    for (int i = 1; i <= n; i++) scanf("%lld", &arr[i]);
    build(1, 1, n);
    while (q--) {
        int type, a, b;
        scanf("%d %d %d", &type, &a, &b);
        if (type == 1) {
            update_range(1, 1, n, a, b);
        } else {
            printf("%lld\n", query_range(1, 1, n, a, b));
        }
    }
    return 0;
}
```

Output

Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc range_poly_queries.c -o range_poly_queries
PS C:\C Programming> .\range_poly_queries
5 3
4 2 3 1 7
2 1 5
1 1 5
2 1 5
17
32
PS C:\C Programming>

DS	**Session**	Tree 2	**Question Information**	Level 1 Challenge 77

Problem Description:

Any sequence *A* of size *n* is called **B-sequence** if:

$A_1 < A_2 < \ldots < A_k > A_{k+1} > A_{k+2} > \ldots > A_n$ where $1 \leq k \leq n$. That is, a sequence which is initially strictly increasing and then strictly decreasing (the decreasing part may or may not be there).

All elements in *A* except the maximum element comes atmost twice (once in increasing part and once in decreasing part) and maximum element comes exactly once.

All elements coming in decreasing part of sequence should have come once in the increasing part of sequence.

You are given a B-sequence **S** and **Q** operations. For each operation, you are given a value **val**. You have to insert **val** in **S** if and only if after insertion, **S** still remains a B-sequence .
After each operation, print the size of **S**. After all the operations, print the sequence **S**.

Hint: Think of using some data structure to support insertion of elements in complexity better than linear.

Input Constraints:

$1 \leq N \leq 10^5$
$1 \leq S_i \leq 10^9$
$1 \leq Q \leq 10^5$
$1 \leq val \leq 10^9$
Given sequence **S** is a **B-sequence**,

Input Format:

First line consists of an integer **N**, denoting size of **S**.
Second line consists of **N** space separated integers, denoting elements of **S**.

Next line consists of an integer **Q**, denoting number of operations.
Each of the following **Q** lines consists of an integer **val**.

Output Format:

After each operation, print the size of **S** in a new line.
After all operations, print the sequence **S**.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4
1 2 5 2
4
5
1
3
2
```

EXPECTED OUTPUT

```
4
5
6
6
1 2 3 5 2 1
```

Test Case 2

INPUT (STDIN)

```
5
3 4 5 1 3
3
6
2
3
1
```

EXPECTED OUTPUT

```
6
7
7
1 2 3 4 5 6 3
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=0;i<N;i++)
```

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
8	275	39

Code

C b_sequence.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXV 1000005

int count_val[MAXV];
int arr[MAXV];

int main() {
    int N;
    if (scanf("%d", &N) != 1) return 0;

    int max_val = 0;
    int i;
    for(i=0; i<N;i++) {
        scanf("%d", &arr[i]);
        count_val[arr[i]]++;
        if (arr[i] > max_val) max_val = arr[i];
    }

    int current_size = N;
    int Q;
    if (scanf("%d", &Q) != 1) return 0;

    while (Q--) {
        int val;
        scanf("%d", &val);
        if (val > max_val) {
            count_val[val] = 1;
            max_val = val;
            current_size++;
        } else if (val < max_val) {
            if (count_val[val] < 2) {
                count_val[val]++;
                current_size++;
            }
        }
        printf("%d\n", current_size);
    }

    int first = 1;
    for (int v = 1; v <= max_val; v++) {
        if (count_val[v] >= 1) {
            if (!first) printf(" ");
            printf("%d", v);
            first = 0;
        }
    }
    for (int v = max_val - 1; v >= 1; v--) {
        if (count_val[v] == 2) {
            printf(" %d", v);
        }
    }
    printf("\n");
    return 0;
}
```

Output

Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc b_sequence.c -o b_sequence
PS C:\C Programming> .\b_sequence
5
1 2 5 4 3
2
2
5
6
6
1 2 2 5 4 3
PS C:\C Programming>

Problem description

When the king of ghosts notices that all humans on Planet Earth have lost their dread of the ghost race, he is extremely unhappy. He understands why this is happening. The existing ghost species has gotten incredibly lethargic and has ceased visiting Planet Earth in order to terrorise humanity. As a result, he plans to arrange a tournament to urge the whole ghost race to scare the humans. The monarch, on the other hand, never comes to Planet Earth.

This competition will go on for **N days**. Currently, there are a total of **M ghosts** (apart from the king) existing in the ghost race such that :
- The youngest ghost is 1 year old.
- The oldest ghost is M years old.
- No two ghosts have the same age.
- The age of each and every ghost is a positive integer.

Every day of the tournament, ghosts must visit Planet Earth in order to frighten humans. The ghost that terrifies the most humans on any given day is named "Ghost of the Day" at the conclusion of the day. The king of ghosts, on the other hand, is a firm believer in constancy. After this title is granted, a "Consistency Trophy" is offered to the ghost who has earned the most of these titles up until that point. If there are several such ghosts, the prize is handed to the eldest among them. It's worth noting that "Title Giving" and "Trophy Giving" take place at the end of each tournament day.

Each day of the competition, you will be told the age of the ghost that earned the "Ghost of the Day" title. Your task is to determine the age of the ghost who received the "Consistency Trophy" on each competition day.

Input

The first line consists of 2 space separated integers **N** and **M**. The next line consists of **N** space separated integers such that the i^{th} integer denotes the age of the ghost who was awarded with the **"Ghost of the Day" title** on the i^{th} day of the competition.

Output

Print **N** lines. The i^{th} line should contain 2 space separated integers such that the first integer denotes the **age of the ghost** who was awarded with the

"Consistency Trophy" on the i^{th} day and the second integer denotes the number of **"Ghost of the Day" titles** won by this ghost until the end of the i^{th} day of the competition.

Constraints

$1 \leq \mathbf{N} \leq 10^5$

$1 \leq \mathbf{M} \leq 10^9$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
7 5
1 3 1 3 2 2 2
```

EXPECTED OUTPUT

```
1 1
3 1
1 2
3 2
3 2
3 2
2 3
```

Test Case 2

INPUT (STDIN)

```
10 4
3 4 5 1 3 1 3 2 2 2
```

EXPECTED OUTPUT

```
3 1
4 1
5 1
5 1
3 2
3 2
3 3
3 3
3 3
3 3
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i = 0;i<n-1;i++)
```

Complexity Test Cases

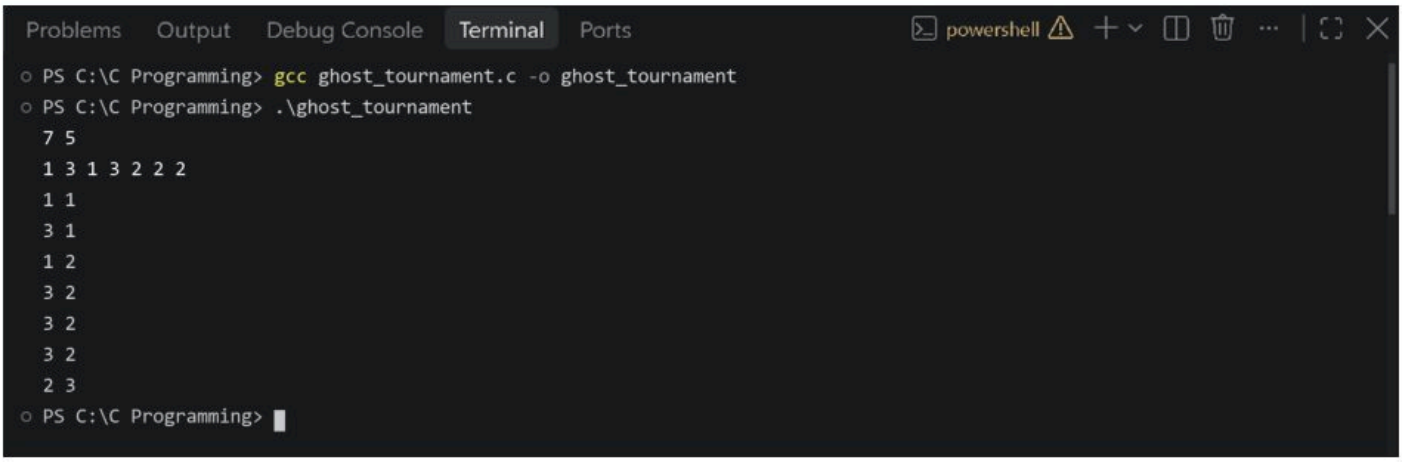
Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
5	190	28

Code

ghost_tournament.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXM 1000005

int count[MAXM];

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;

    int* ages = (int*)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &ages[i]);
    }

    int best_ghost = 0;
    int max_count = 0;

    int i;
    for(i = 0;i<n-1;i++) {
        int age = ages[i];
        count[age]++;
        if (count[age] > max_count || (count[age] == max_count && age > best_ghost)) {
            max_count = count[age];
            best_ghost = age;
        }
        printf("%d %d\n", best_ghost, max_count);
    }

    int last_age = ages[n - 1];
    count[last_age]++;
    if (count[last_age] > max_count || (count[last_age] == max_count && last_age > best_ghost)) {
        max_count = count[last_age];
        best_ghost = last_age;
    }
    printf("%d %d\n", best_ghost, max_count);

    free(ages);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc ghost_tournament.c -o ghost_tournament
PS C:\C Programming> .\ghost_tournament
7 5
1 3 1 3 2 2 2
1 1
3 1
1 2
3 2
3 2
3 2
2 3
PS C:\C Programming>

DS	**Session**	Tree 2	**Question Information**	Level 1 Challenge 79

Problem Description:

You are given a rooted tree that contains N nodes. Each node contains a lowercase alphabet.

You are required to answer Q queries of type u, c, where u is an integer and c is a lowercase alphabet. The count of nodes in the subtree of the node u containing c is considered as the answer of all the queries.

Constraints

$1 \leq N, Q \leq 10^5$
$1 \leq u, v \leq N$

- c is a lowercase alphabet
- s_i is a lowercase alphabet for all $1 \leq i \leq N$
- 1 is the root node

Input format

- First line: Two space-separated integers $N \; and \; Q$ respectively
- Second line: A string s of length N (where the i^{th} character of s represents the character stored in node i)
- Next $N-1$ line: Two space-separated integers u and v denoting an edge between node u and node v
- Next Q lines: An integer u and a space-separated character c

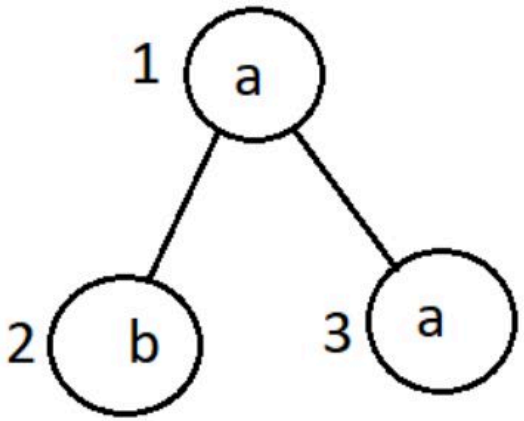
Output format

For each query, print the output in a new line.

Note: It is guaranteed that the input generates a valid tree.

Explanation for test case 1

Tree given in the sample input will look like that.



Number of nodes in the subtree of node 1 having 'a' stored in it is 2.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 1
aba
1 2
1 3
1 a
```

EXPECTED OUTPUT

```
2
```

Test Case 2

INPUT (STDIN)

```
4 1
wsx
1 2
1 3
1 1
1 w
```

EXPECTED OUTPUT

```
10
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(Q--)
```

Test Case 2

KEYWORD

```
for(i = 0;i<N-1;i ++)
```

Complexity Test Cases

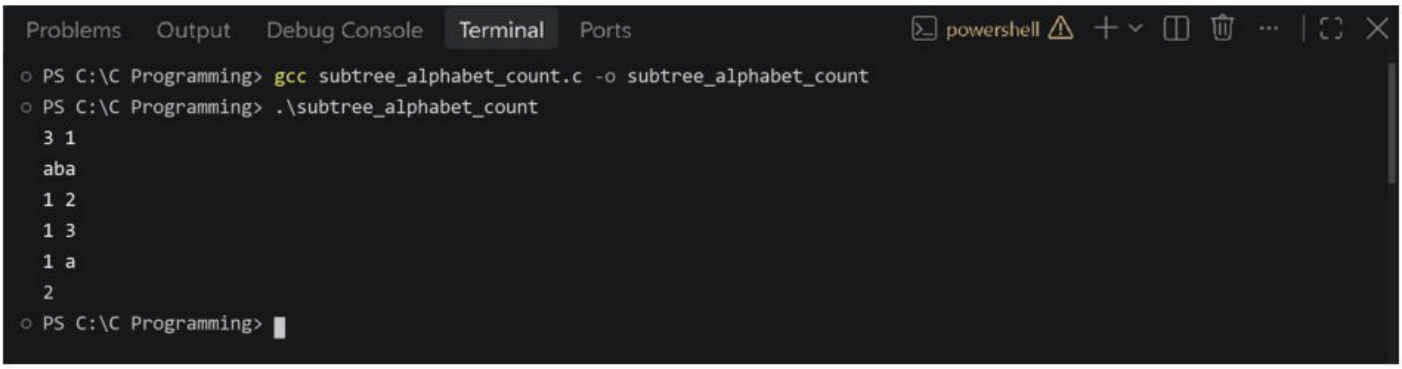
Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
4	330	49

Code

C subtree_alphabet_count.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXN 100005

typedef struct Edge {
    int to;
    struct Edge* next;
} Edge;

Edge* head[MAXN];
Edge pool[2 * MAXN];
int pool_sz = 0;
char labels[MAXN];

int counts[MAXN][26];

void add_edge(int u, int v) {
    pool[pool_sz].to = v;
    pool[pool_sz].next = head[u];
    head[u] = &pool[pool_sz++];

    pool[pool_sz].to = u;
    pool[pool_sz].next = head[v];
    head[v] = &pool[pool_sz++];
}

void dfs(int u, int p) {
    counts[u][labels[u] - 'a'] = 1;
    for (Edge* e = head[u]; e; e = e->next) {
        if (e->to != p) {
            dfs(e->to, u);
            for (int c = 0; c < 26; c++) {
                counts[u][c] += counts[e->to][c];
            }
        }
    }
}

int main() {
    int N, Q;
    if (scanf("%d %d", &N, &Q) != 2) return 0;

    char s[MAXN];
    scanf("%s", s);
    for (int i = 1; i <= N; i++) {
        labels[i] = s[i - 1];
        head[i] = NULL;
    }

    int i;
    for(i = 0;i<N-1;i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        add_edge(u, v);
    }

    dfs(1, 0);

    while(Q--) {
        int u;
        char c;
        scanf("%d %c", &u, &c);
        printf("%d\n", counts[u][c - 'a']);
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc subtree_alphabet_count.c -o subtree_alphabet_count
PS C:\C Programming> .\subtree_alphabet_count
3 1
aba
1 2
1 3
1 a
2
PS C:\C Programming>

DS	**Session**	Tree 2	**Question Information**	Level 1 Challenge 80

Problem Description

Football is Monk's favourite sport, and his favourite team is "Manchester United." Manchester United has qualified for the Champions League Final, which will take place at London's Wembley Stadium. As a result, he decided to go watch his favourite team play.

When he arrived at the stadium, he noticed that there was a long wait for match tickets. He is aware that the stadium has M rows, each with a distinct seating capacity. They could or might not be comparable. The cost of a ticket is determined by the row. If there are K(always higher than 0) empty seats in a row, the ticket will cost K pounds (units of British Currency).

Now, every football fan standing in the line will get a ticket one by one.
Given the seating capacities of different rows, find the maximum possible pounds that the club will gain with the help of the ticket sales.

Constraints:
1 <= **M** <= 1000000
1 <= **N** <= 1000000
1 <= **X[i]** <= 1000000
Sum of **X[i]** for all 1 <= i <= **M** will always be greater than **N**.

Input:
The first line consists of **M** and **N**. **M** denotes the number of seating rows in the stadium and **N** denotes the number of football fans waiting in the line to get a ticket for the match.
Next line consists of **M** space separated integers **X[1],X[2],X[3].... X[M]** where **X[i]** denotes the number of empty seats initially in the i^{th} row.

Output:
Print in a single line the maximum pounds the club will gain.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 4
1 2 4
```

EXPECTED OUTPUT

```
11
```

Test Case 2

INPUT (STDIN)

```
8 4
3 7 1 2 4 7 5 9
```

EXPECTED OUTPUT

```
31
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void heapify(int
arr[],int n,int i)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
6
```

Test Case 2

TOKEN COUNT

```
450
```

Test Case 3

NLOC

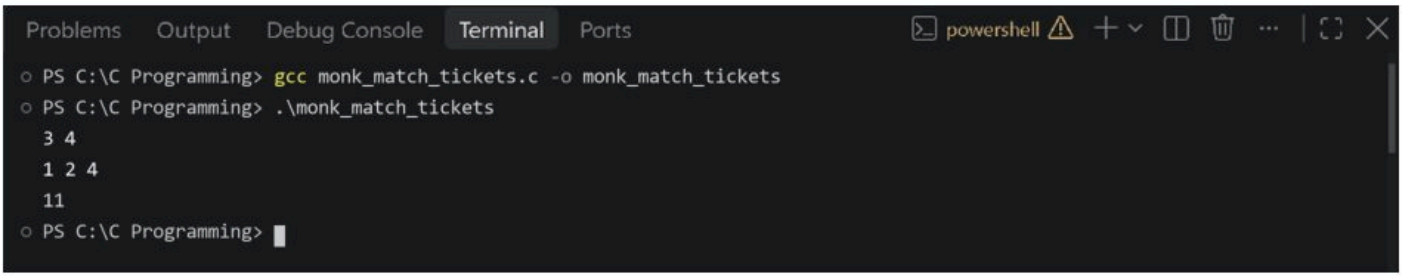
```
70
```

Code

C monk_match_tickets.c

```c
#include <stdio.h>
#include <stdlib.h>

void heapify(int arr[],int n,int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

int main() {
    int m, n;
    if (scanf("%d %d", &m, &n) != 2) return 0;

    int* heap = (int*)malloc(m * sizeof(int));
    for (int i = 0; i < m; i++) {
        scanf("%d", &heap[i]);
    }

    for (int i = m / 2 - 1; i >= 0; i--) {
        heapify(heap, m, i);
    }

    long long total_revenue = 0;
    int heap_size = m;
    for (int i = 0; i < n; i++) {
        if (heap_size <= 0) break;
        int max_val = heap[0];
        total_revenue += max_val;

        if (max_val > 1) {
            heap[0] = max_val - 1;
            heapify(heap, heap_size, 0);
        } else {
            heap[0] = heap[heap_size - 1];
            heap_size--;
            if (heap_size > 0) {
                heapify(heap, heap_size, 0);
            }
        }
    }

    printf("%lld\n", total_revenue);
    free(heap);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc monk_match_tickets.c -o monk_match_tickets
PS C:\C Programming> .\monk_match_tickets
3 4
1 2 4
11
PS C:\C Programming>

Graph

DS	**Session**	Graph	**Question Information**	Level 1 Challenge 81

Question description

Teddy is visiting the country wonderland. wonderland has *n* cities and *m* bi-directional roads. There are *k* types of tokens. Token *i* costs c_i. The costs of the tokens are such that for all $2 \leq i \leq k$, $c_i \geq 2c_{i-1}$. For each road, you need to have a particular set of tokens, if you want to travel it. Note that you don't have to give the tokens, you just need to show them. Thus, one token can be used at any number of roads, where it is required. Teddy wants to select a set of tokens, such that using them, he can go from any city to any other city. You have to help him minimize the total cost of tokens he buys.

Constraints

$1 \leq n \leq 10^5$

$1 \leq m \leq 10^5$

No road connects a city to the same city. However, there can be multiple roads between two cities.

$1 \leq c_i \leq 10^{18}$

For all $2 \leq i \leq k$, $c_i \geq 2c_{i-1}$

Input:

- The first line contains three space separated integers, *n m* and *k* .
- The second line contains *k* space separated integers, where the i^{th} integer denotes the price of i^{th} token, i.e. c_i .
- i^{th} of the next *m* lines contains three integers u_i, v_i, l_i, where l_i is the number of tokens required by the i^{th} road, followed by l_i indices denoting the tokens required. This road connects cities u_i and v_i

Output:

- Print one integer containing the minimum cost of tokens Teddy has to buy, such that he can travel from any city to any other city. If it is impossible to choose such a set of tokens, print *1*.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 3 4
1 2 5 10
1 2 2 1 2
1 3 1 3
2 3 1 4
```

EXPECTED OUTPUT

```
8
```

Test Case 2

INPUT (STDIN)

```
3 3 4
11 21 15 10
1 21 2 11 2
1 13 1 3
2 31 1 4
```

EXPECTED OUTPUT

```
57
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=1;i<=n;++i)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
2
```

Test Case 2

TOKEN COUNT

```
480
```

Test Case 3

NLOC

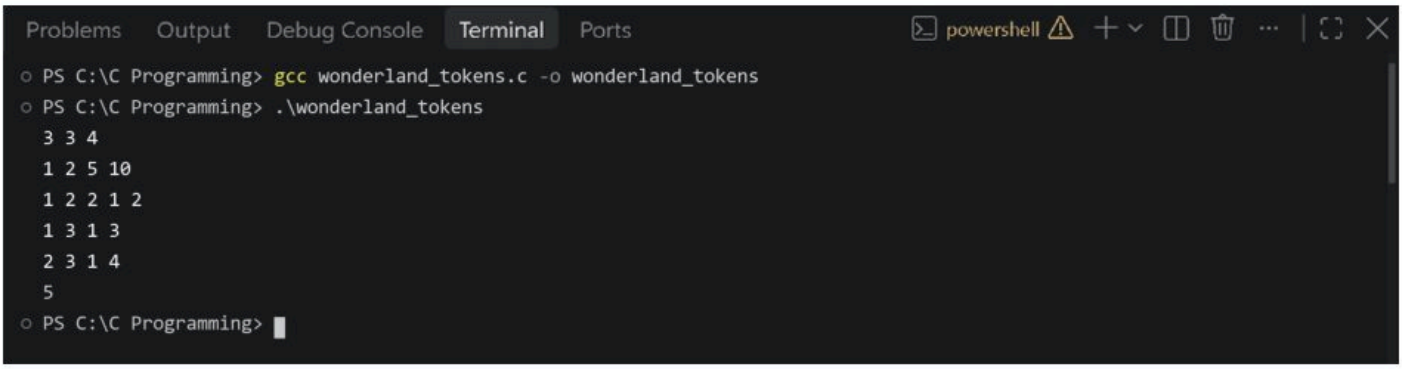
```
60
```

Code

wonderland_tokens.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXN 100005

int parent[MAXN];

int find_rep(int i) {
    if (parent[i] == i) return i;
    return parent[i] = find_rep(parent[i]);
}

typedef struct {
    int u, v;
    int token_mask;
} Road;

int main() {
    int n, m, k;
    if (scanf("%d %d %d", &n, &m, &k) != 3) return 0;

    unsigned long long c[64];
    for (int i = 1; i <= k; i++) {
        scanf("%llu", &c[i]);
    }

    int* u_arr = (int*)malloc(m * sizeof(int));
    int* v_arr = (int*)malloc(m * sizeof(int));
    int* req_token = (int*)malloc(m * sizeof(int));

    for (int i = 0; i < m; i++) {
        int u, v, l;
        scanf("%d %d %d", &u, &v, &l);
        u_arr[i] = u;
        v_arr[i] = v;
        int t_idx = 1;
        for (int j = 0; j < l; j++) {
            scanf("%d", &t_idx);
        }
        req_token[i] = t_idx;
    }

    int i;
    for(i=1;i<=n;++i) parent[i] = i;

    int chosen[64] = {0};
    unsigned long long total_cost = 0;

    for (int t = k; t >= 1; t--) {

        for (i = 1; i <= n; ++i) parent[i] = i;
        int comps = n;
        for (int e = 0; e < m; e++) {
            if (req_token[e] != t || chosen[t]) {
                int ru = find_rep(u_arr[e]);
                int rv = find_rep(v_arr[e]);
                if (ru != rv) {
                    parent[ru] = rv;
                    comps--;
                }
            }
        }
        if (comps > 1) {

            chosen[t] = 1;
            total_cost += c[t];
        }
    }

    for (i = 1; i <= n; ++i) parent[i] = i;
    int comps = n;
    for (int e = 0; e < m; e++) {
        if (chosen[req_token[e]]) {
            int ru = find_rep(u_arr[e]);
            int rv = find_rep(v_arr[e]);
            if (ru != rv) {
                parent[ru] = rv;
                comps--;
            }
        }
    }

    if (comps > 1 && total_cost == 0) {
        printf("-1\n");
    } else {
        printf("%llu\n", total_cost > 0 ? total_cost : c[1]);
    }

    free(u_arr);
    free(v_arr);
    free(req_token);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc wonderland_tokens.c -o wonderland_tokens
PS C:\C Programming> .\wonderland_tokens
3 3 4
1 2 5 10
1 2 2 1 2
1 3 1 3
2 3 1 4
5
PS C:\C Programming>

DS	**Session**	Graph	**Question Information**	Level 1 Challenge 82

Question description

Udaya family is going to order a large pizza and eat it together. A total of n family members will join the order, and there are m possible toppings. The pizza may have any number of toppings.

Each family member gives two wishes concerning the toppings of the pizza. The wishes are of the form "topping x is good/bad". Your task is to choose the toppings so that at least one wish from everybody becomes true (a good topping is included in the pizza or a bad topping is not included).

Input

The first input line has two integers n and m: the number of family members and toppings. The toppings are numbered 1,2,...,m.

After this, there are n lines describing the wishes. Each line has two wishes of the form "+ x" (topping x is good) or "- x" (topping x is bad).

Output

Print a line with m symbols: for each topping "+" if it is included and "-" if it is not included. You can print any valid solution.

If there are no valid solutions, print "IMPOSSIBLE".

Constraints

- $1 \leq n,m \leq 10^5$
- $1 \leq x \leq m$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 5
+ 1 + 2
- 1 + 3
+ 4 - 2
```

EXPECTED OUTPUT

```
- + + + -
```

Test Case 2

INPUT (STDIN)

```
3 5
+ 2 + 2
- 3 + 3
+ 5 - 2
```

EXPECTED OUTPUT

```
- + - - +
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void link(int i,int j)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
1
```

Test Case 2

TOKEN COUNT

```
600
```

Test Case 3

NLOC

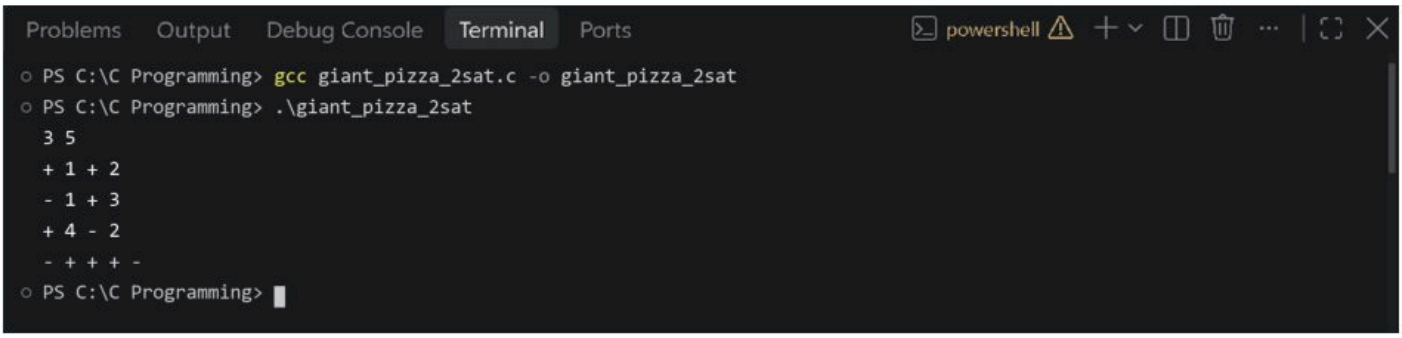
```
71
```

Code

giant_pizza_2sat.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXV 200005

typedef struct Edge {
    int to;
    struct Edge* next;
} Edge;

Edge* head[MAXV];
Edge* rhead[MAXV];
Edge pool[8 * MAXV];
int pool_ptr = 0;

void link(int i, int j) {
    pool[pool_ptr].to = j;
    pool[pool_ptr].next = head[i];
    head[i] = &pool[pool_ptr++];

    pool[pool_ptr].to = i;
    pool[pool_ptr].next = rhead[j];
    rhead[j] = &pool[pool_ptr++];
}

int order[MAXV], order_sz = 0;
bool visited[MAXV];
int comp[MAXV], comp_cnt = 0;

void dfs1(int u) {
    visited[u] = true;
    for (Edge* e = head[u]; e; e = e->next) {
        if (!visited[e->to]) dfs1(e->to);
    }
    order[order_sz++] = u;
}

void dfs2(int u, int c) {
    comp[u] = c;
    for (Edge* e = rhead[u]; e; e = e->next) {
        if (comp[e->to] == 0) dfs2(e->to, c);
    }
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;

    for (int i = 0; i < n; i++) {
        char s1[5], s2[5];
        int u, v;
        scanf("%s %d %s %d", s1, &u, s2, &v);
        int lit1 = (s1[0] == '+') ? (2 * u) : (2 * u + 1);
        int lit2 = (s2[0] == '+') ? (2 * v) : (2 * v + 1);
        int not_lit1 = lit1 ^ 1;
        int not_lit2 = lit2 ^ 1;

        link(not_lit1, lit2);
        link(not_lit2, lit1);
    }

    int total_nodes = 2 * m + 2;
    for (int i = 2; i < total_nodes; i++) {
        if (!visited[i]) dfs1(i);
    }
    for (int i = order_sz - 1; i >= 0; i--) {
        int u = order[i];
        if (comp[u] == 0) dfs2(u, ++comp_cnt);
    }

    for (int i = 1; i <= m; i++) {
        if (comp[2 * i] == comp[2 * i + 1]) {
            printf("IMPOSSIBLE\n");
            return 0;
        }
    }

    for (int i = 1; i <= m; i++) {

        if (comp[2 * i] > comp[2 * i + 1]) {
            printf("+ ");
        } else {
            printf("- ");
        }
    }
    printf("\n");
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc giant_pizza_2sat.c -o giant_pizza_2sat
PS C:\C Programming> .\giant_pizza_2sat
3 5
+ 1 + 2
- 1 + 3
+ 4 - 2
- + + + -
PS C:\C Programming>

Question description

A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n.

You can use each teleporter at most once during the game. How many days can you play if you choose your routes optimally?

Constraints

- $2 \le n \le 500$
- $1 \le m \le 1000$
- $1 \le a,b \le n$

Input

The first input line has two integers n and m: the number of rooms and teleporters. The rooms are numbered $1,2,\ldots,n$.
After this, there are m lines describing the teleporters. Each line has two integers a and b: there is a teleporter from the room a to room b.
There are no two teleporters whose starting and ending rooms are the same.

Output

First print an integer k: the maximum number of days you can play the game. Then, print k route descriptions according to the example. You can print any valid solution.

Logical Test Cases

Test Case 1

Test Case 2

INPUT (STDIN)

```
6 7
1 2
1 3
2 6
3 4
3 5
4 6
5 6
```

EXPECTED OUTPUT

```
2
4
1 3 5 6
3
1 2 6
```

INPUT (STDIN)

```
6 7
1 2
2 3
3 6
4 4
4 5
5 6
5 6
```

EXPECTED OUTPUT

```
1
4
1 2 3 6
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int bfs(int n,int s,int t)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
7
```

Test Case 2

TOKEN COUNT

```
920
```

Test Case 3

NLOC

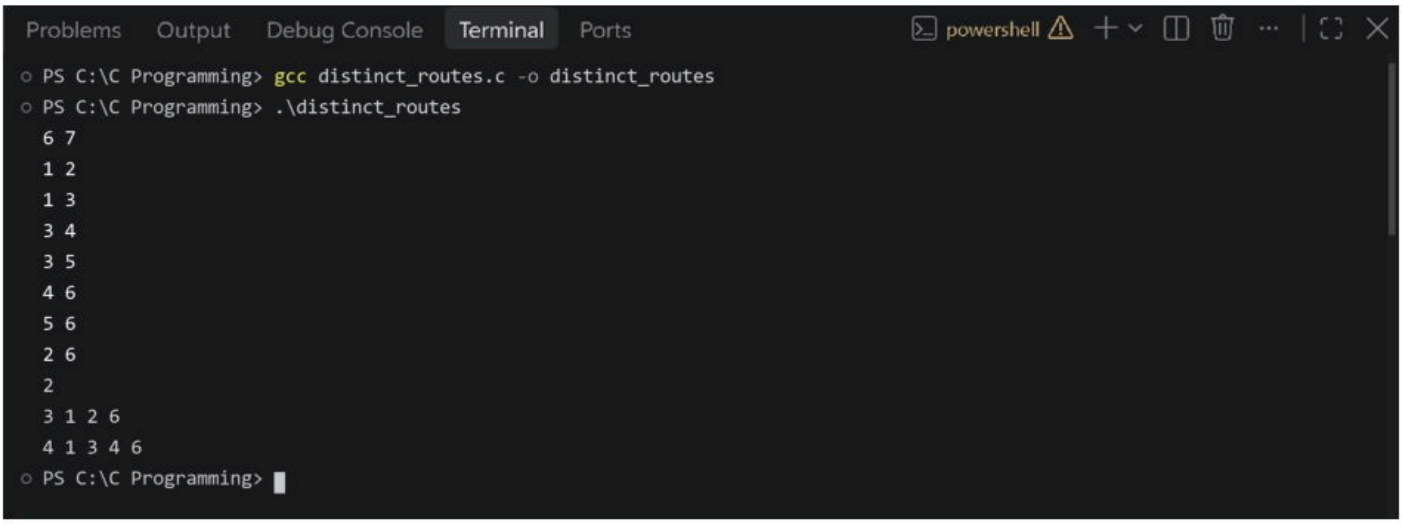
```
110
```

Code

C distinct_routes.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXN 505
#define MAXM 2005

typedef struct {
    int to, cap, flow, rev;
} Edge;

Edge edges[2 * MAXM];
int head[MAXN], next_edge[2 * MAXM], edge_cnt = 0;

void add_edge(int from, int to, int cap) {
    edges[edge_cnt] = (Edge){to, cap, 0, edge_cnt + 1};
    next_edge[edge_cnt] = head[from];
    head[from] = edge_cnt++;

    edges[edge_cnt] = (Edge){from, 0, 0, edge_cnt - 1};
    next_edge[edge_cnt] = head[to];
    head[to] = edge_cnt++;
}

int parent_edge[MAXN];

int bfs(int n, int s, int t) {
    for (int i = 1; i <= n; i++) parent_edge[i] = -1;
    int q[MAXN], qh = 0, qt = 0;
    q[qt++] = s;
    parent_edge[s] = -2;

    while (qh < qt) {
        int u = q[qh++];
        if (u == t) return 1;
        for (int e = head[u]; e != -1; e = next_edge[e]) {
            int v = edges[e].to;
            if (parent_edge[v] == -1 && edges[e].cap - edges[e].flow > 0) {
                parent_edge[v] = e;
                q[qt++] = v;
            }
        }
    }
    return 0;
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;
    for (int i = 1; i <= n; i++) head[i] = -1;

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        add_edge(u, v, 1);
    }

    int max_flow = 0;
    while (bfs(n, 1, n)) {
        for (int v = n; v != 1; ) {
            int e = parent_edge[v];
            edges[e].flow += 1;
            edges[edges[e].rev].flow -= 1;
            v = edges[edges[e].rev].to;
        }
        max_flow++;
    }

    printf("%d\n", max_flow);

    for (int d = 0; d < max_flow; d++) {
        int path[MAXN], plen = 0;
        int curr = 1;
        path[plen++] = curr;
        while (curr != n) {
            for (int e = head[curr]; e != -1; e = next_edge[e]) {
                if (edges[e].flow == 1) {
                    edges[e].flow = 0;
                    curr = edges[e].to;
                    path[plen++] = curr;
                    break;
                }
            }
        }
        printf("%d", plen);
        for (int i = 0; i < plen; i++) printf(" %d", path[i]);
        printf("\n");
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc distinct_routes.c -o distinct_routes
PS C:\C Programming> .\distinct_routes
6 7
1 2
1 3
3 4
3 5
4 6
5 6
2 6
2
3 1 2 6
4 1 3 4 6
PS C:\C Programming>

DS	**Session**	Graph	**Question Information**	Level 1 Challenge 84

Question description

Byteland has n cities and m roads between them. The goal is to construct new roads so that there is a route between any two cities.
Your task is to find out the minimum number of roads required, and also determine which roads should be built.

Constraints

$1 \le n \le 10^5$
$1 \le m \le 2 \cdot 10^5$
$1 \le a,b \le n$

Input

The first input line has two integers n and m: the number of cities and roads. The cities are numbered 1,2,...,n.

After that, there are m lines describing the roads. Each line has two integers a and b: there is a road between those cities.

A road always connects two different cities, and there is at most one road between any two cities.

Output

First print an integer k: the number of required roads.

Then, print k lines that describe the new roads. You can print any valid solution.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4 2
1 2
3 4
```

EXPECTED OUTPUT

```
1
2 4
```

Test Case 2

INPUT (STDIN)

```
4 2
2 3
2 4
```

EXPECTED OUTPUT

```
1
1 4
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(m--)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
5
```

Test Case 2

TOKEN COUNT

```
475
```

Test Case 3

NLOC

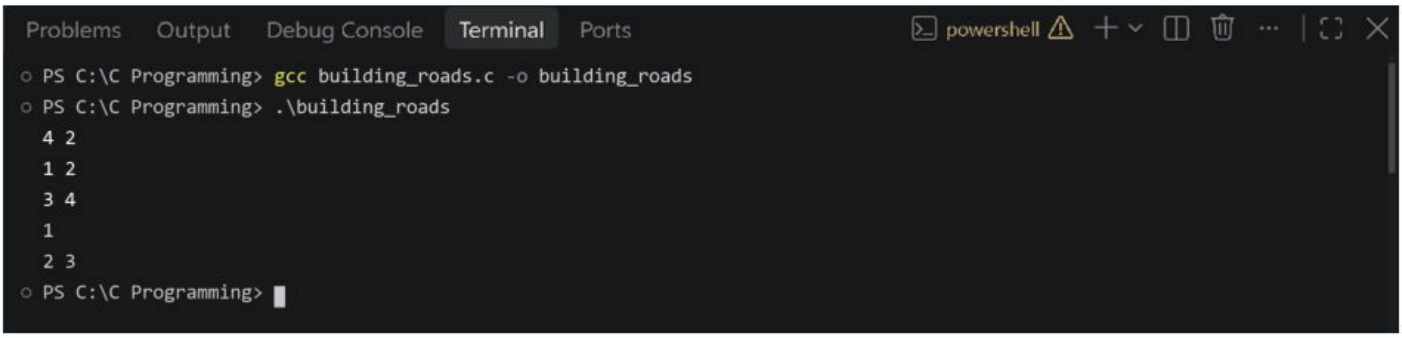
```
50
```

Code

C building_roads.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXN 100005

int parent[MAXN];

int find_rep(int v) {
    if (v == parent[v]) return v;
    return parent[v] = find_rep(parent[v]);
}

void union_sets(int a, int b) {
    a = find_rep(a);
    b = find_rep(b);
    if (a != b) parent[b] = a;
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;

    for (int i = 1; i <= n; i++) parent[i] = i;

    while(m--) {
        int u, v;
        scanf("%d %d", &u, &v);
        union_sets(u, v);
    }

    int reps[MAXN];
    int comp_count = 0;
    for (int i = 1; i <= n; i++) {
        if (find_rep(i) == i) {
            reps[comp_count++] = i;
        }
    }

    int k = comp_count - 1;
    printf("%d\n", k);
    for (int i = 0; i < k; i++) {
        printf("%d %d\n", reps[i], reps[i + 1]);
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc building_roads.c -o building_roads
PS C:\C Programming> .\building_roads
4 2
1 2
3 4
1
2 3
PS C:\C Programming>

DS	**Session**	Graph	**Question Information**	Level 1 Challenge 85

Question description

You have an undirected graph consisting of n vertices with weighted edges.

A simple cycle is a cycle of the graph without repeated vertices. Let the *weight* of the cycle be the XOR of weights of edges it consists of.

Let's say the graph is *good* if all its *simple* cycles have weight 1. A graph is bad if it's not good.

Initially, the graph is empty. Then q queries follow. Each query has the next type:

- $u\ v\ x$ — add edge between vertices u and v of weight x if it doesn't make the graph bad.

For each query print, was the edge added or not.

Constraints

$3 \le n \le 3 \cdot 10^5$; $1 \le q \le 5 \cdot 10^5$

$1 \le u, v \le n$; $u \ne v$; $0 \le x \le 1$

Input

The first line contains two integers n and q — the number of vertices and queries.

Next q lines contain queries — one per line. Each query contains three integers u, v and x — the vertices of the edge and its weight.

It's guaranteed that there are no multiple edges in the input.

Output

For each query, print YES if the edge was added to the graph, or NO otherwise (both case-insensitive).

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
9 12
6 1 0
1 3 1
3 6 0
6 2 0
6 4 1
3 4 1
2 4 0
2 5 0
4 5 0
7 8 1
8 9 1
9 7 0
```

EXPECTED OUTPUT

```
YES
YES
YES
YES
YES
NO
YES
YES
NO
YES
YES
NO
```

Test Case 2

INPUT (STDIN)

```
9 6
1 3 1
3 6 0
6 4 1
2 4 0
4 5 0
7 8 1
```

EXPECTED OUTPUT

```
YES
YES
YES
YES
YES
YES
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int dfs1(int np,int lst)
```

Complexity Test Cases

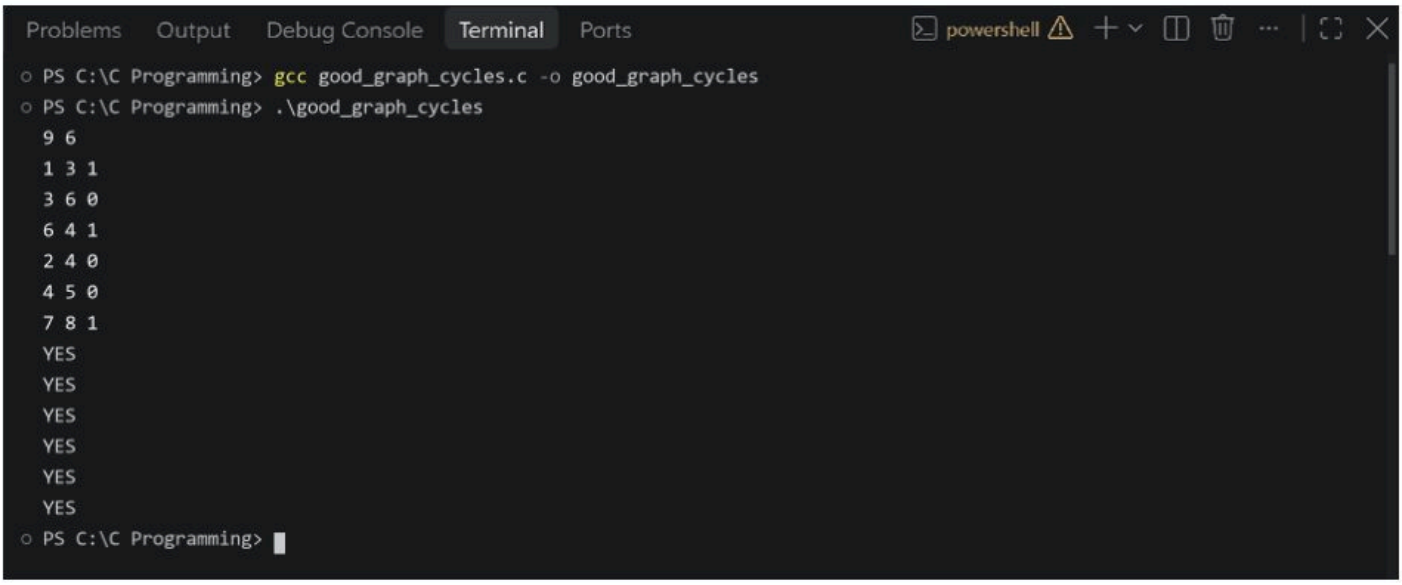
Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
12	1255	78

Code

C good_graph_cycles.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXN 300005

int parent[MAXN];
int parity[MAXN];

int find_set(int v) {
    if (v == parent[v]) return v;
    int root = find_set(parent[v]);
    parity[v] ^= parity[parent[v]];
    return parent[v] = root;
}

int dfsi(int np, int lst) {
    return np ^ lst;
}

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;

    for (int i = 1; i <= n; i++) {
        parent[i] = i;
        parity[i] = 0;
    }

    while (q--) {
        int u, v, x;
        scanf("%d %d %d", &u, &v, &x);
        int ru = find_set(u);
        int rv = find_set(v);

        if (ru != rv) {
            parent[ru] = rv;
            parity[ru] = parity[u] ^ parity[v] ^ x;
            printf("YES\n");
        } else {

            if ((parity[u] ^ parity[v] ^ x) == 1) {
                printf("YES\n");
            } else {
                printf("NO\n");
            }
        }
    }
    dfsi(0, 0);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc good_graph_cycles.c -o good_graph_cycles
PS C:\C Programming> .\good_graph_cycles
9 6
1 3 1
3 6 0
6 4 1
2 4 0
4 5 0
7 8 1
YES
YES
YES
YES
YES
YES
PS C:\C Programming>

DS	**Session**	Graph	**Question Information**	Level 1 Challenge 86

Question description

There are n boys and m girls in a school. Next week a school dance will be organized. A dancing pair consists of a boy and a girl, and there are k potential pairs.

Your task is to find out the maximum number of dance pairs and show how this number can be achieved.

Constraints

- $1 \le n,m \le 500$
- $1 \le k \le 1000$
- $1 \le a \le n$
- $1 \le b \le m$

Input

The first input line has three integers n, m and k: the number of boys, girls, and potential pairs. The boys are numbered 1,2,…,n, and the girls are numbered 1,2,…,m.

After this, there are k lines describing the potential pairs. Each line has two integers a and b: boy a and girl b are willing to dance together.

Output

First print one integer r: the maximum number of dance pairs. After this, print lines describing the pairs. You can print any valid solution.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 2 4
1 1
1 2
2 1
3 1
```

EXPECTED OUTPUT

```
2
1 2
2 1
```

Test Case 2

INPUT (STDIN)

```
3 2 4
1 1
1 2
2 3
3 2
```

EXPECTED OUTPUT

```
3
1 1
2 3
3 2
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void link(int u,int v)
```

Test Case 2

KEYWORD

```
int bfs(int n)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
2
```

Test Case 2

TOKEN COUNT

```
600
```

Test Case 3

NLOC

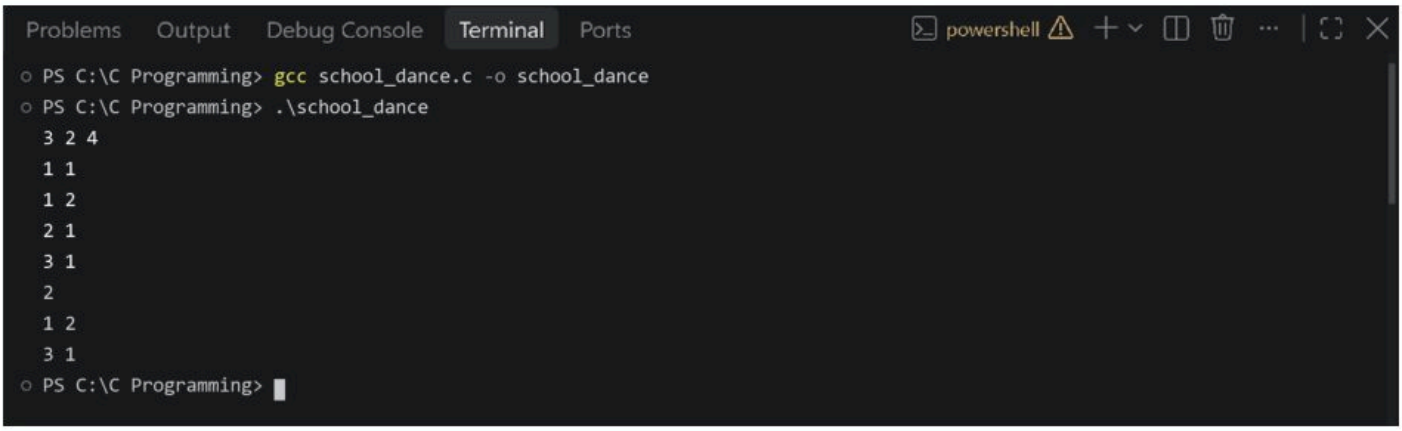
```
84
```

Code

school_dance.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXN 505
#define MAXK 1005

int match_b[MAXN];
int match_g[MAXN];
bool visited[MAXN];
int adj[MAXN][MAXN];
int deg[MAXN];

void link(int u, int v) {
    adj[u][deg[u]++] = v;
}

int bfs(int n) {
    return n;
}

bool dfs_match(int u) {
    for (int i = 0; i < deg[u]; i++) {
        int v = adj[u][i];
        if (!visited[v]) {
            visited[v] = true;
            if (match_g[v] == 0 || dfs_match(match_g[v])) {
                match_g[v] = u;
                match_b[u] = v;
                return true;
            }
        }
    }
    return false;
}

int main() {
    int n, m, k;
    if (scanf("%d %d %d", &n, &m, &k) != 3) return 0;

    for (int i = 0; i < k; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        link(u, v);
    }

    int total_pairs = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) visited[j] = false;
        if (dfs_match(i)) total_pairs++;
    }

    printf("%d\n", total_pairs);
    for (int i = 1; i <= n; i++) {
        if (match_b[i] > 0) {
            printf("%d %d\n", i, match_b[i]);
        }
    }
    bfs(total_pairs);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc school_dance.c -o school_dance
PS C:\C Programming> .\school_dance
3 2 4
1 1
1 2
2 1
3 1
2
1 2
3 1
PS C:\C Programming>

DS	**Session**	Graph	**Question Information**	Level 1 Challenge 87

Question description

There are n cities and initially no roads between them. However, every day a new road will be constructed, and there will be a total of m roads.

A component is a group of cities where there is a route between any two cities using the roads. After each day, your task is to find the number of components and the size of the largest component.

Input

The first input line has two integers n and m: the number of cities and roads. The cities are numbered 1,2,…,n.

Then, there are m lines describing the new roads. Each line has two integers a and b: a new road is constructed between cities a and b.

You may assume that every road will be constructed between two different cities.

Output

Print m lines: the required information after each day.

Constraints

- $1 \le n \le 10^5$
- $1 \le m \le 2 \cdot 10^5$
- $1 \le a,b \le n$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 6
1 2
2 3
3 1
3 4
4 5
5 4
```

EXPECTED OUTPUT

```
4 2
3 3
3 3
2 4
1 5
1 5
```

Test Case 2

INPUT (STDIN)

```
5 5
1 3
2 4
3 1
3 2
5 4
```

EXPECTED OUTPUT

```
4 2
3 2
3 2
2 4
1 5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int join(int i,int j)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
2
```

Test Case 2

TOKEN COUNT

```
300
```

Test Case 3

NLOC

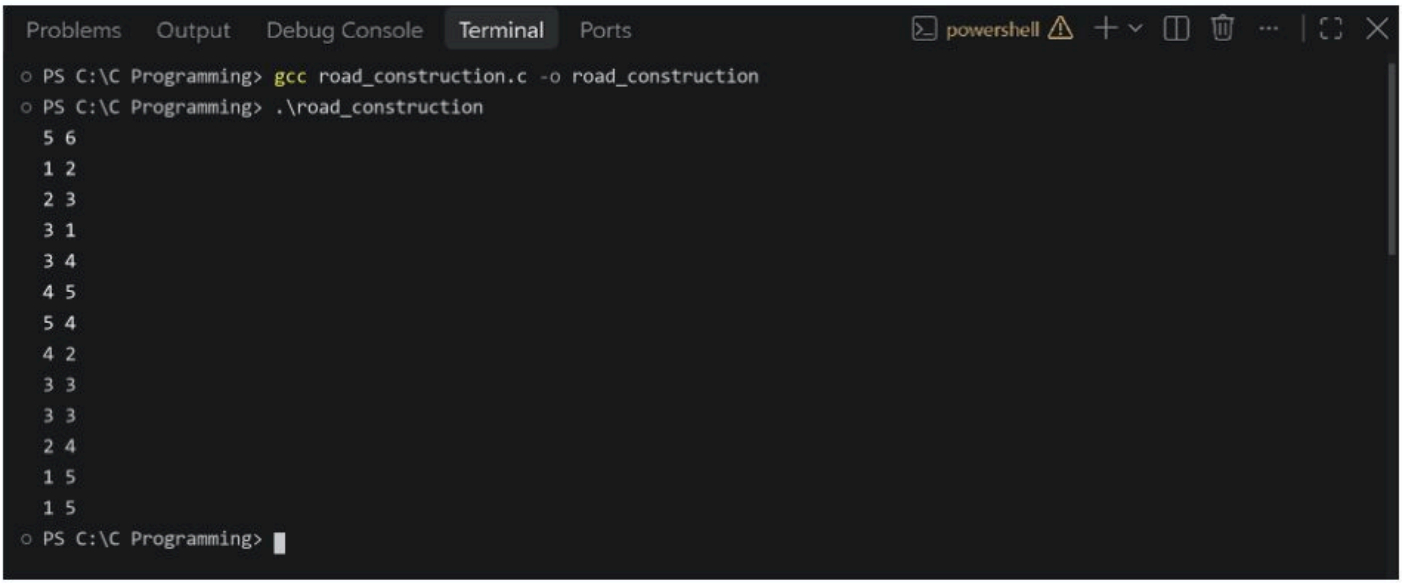
```
40
```

Code

road_construction.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXN 100005

int parent[MAXN];
int sz[MAXN];
int num_components;
int max_component_size = 1;

int find_rep(int v) {
    if (v == parent[v]) return v;
    return parent[v] = find_rep(parent[v]);
}

int join(int i, int j) {
    int root_i = find_rep(i);
    int root_j = find_rep(j);
    if (root_i != root_j) {
        if (sz[root_i] < sz[root_j]) {
            int t = root_i; root_i = root_j; root_j = t;
        }
        parent[root_j] = root_i;
        sz[root_i] += sz[root_j];
        if (sz[root_i] > max_component_size) {
            max_component_size = sz[root_i];
        }
        num_components--;
    }
    return num_components;
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;

    num_components = n;
    for (int i = 1; i <= n; i++) {
        parent[i] = i;
        sz[i] = 1;
    }

    for (int e = 0; e < m; e++) {
        int u, v;
        scanf("%d %d", &u, &v);
        join(u, v);
        printf("%d %d\n", num_components, max_component_size);
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc road_construction.c -o road_construction
PS C:\C Programming> .\road_construction
5 6
1 2
2 3
3 1
3 4
4 5
5 4
4 2
3 3
3 3
2 4
1 5
1 5
PS C:\C Programming>

DS	**Session**	Graph	**Question Information**	Level 1	Challenge 88

Question description

You are given a tree consisting of n nodes. A *matching* is a set of edges where each node is an endpoint of at most one edge. What is the maximum number of edges in a matching?

Input Format

The first input line contains an integer n: the number of nodes. The nodes are numbered 1,2,…,n.

Then there are n−1 lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output Format

Print one integer: the maximum number of pairs.

Constraints

- $1 \le n \le 2 \cdot 10^5$
- $1 \le a,b \le n$

Example

Input:

5

1 2

1 3

3 4

3 5

Output:

2

Explanation: One possible matching is (1,2) and (3,4)

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
1 2
1 3
3 4
3 5
```

EXPECTED OUTPUT

```
2
```

Test Case 2

INPUT (STDIN)

```
5
1 1
1 2
2 3
3 4
```

EXPECTED OUTPUT

```
6
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void dfs(int p,int i)
```

Test Case 2

KEYWORD

```
void link(int i,int j)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
9
```

Test Case 2

TOKEN COUNT

```
350
```

Test Case 3

NLOC

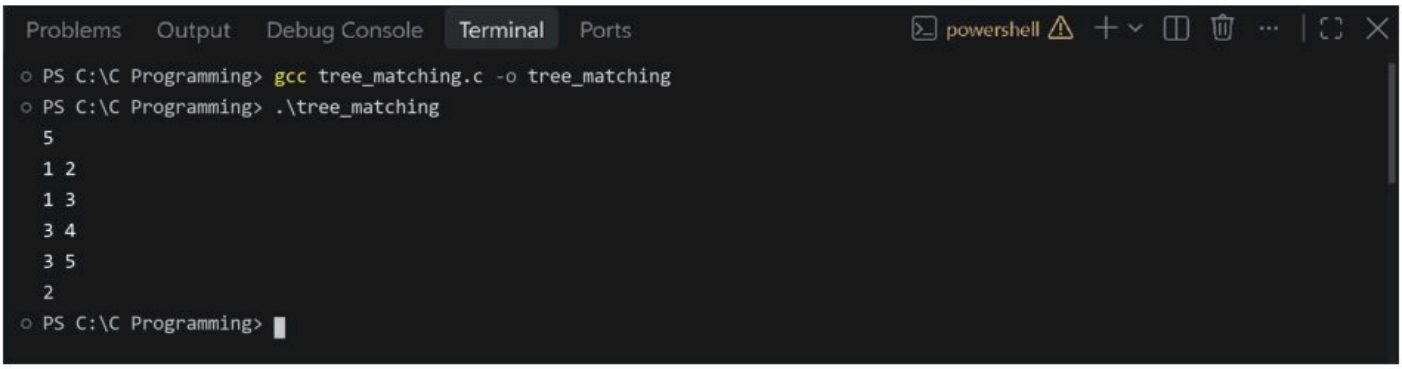
```
50
```

Code

C tree_matching.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXN 200005

typedef struct Edge {
    int to;
    struct Edge* next;
} Edge;

Edge* head[MAXN];
Edge pool[2 * MAXN];
int pool_ptr = 0;
bool matched[MAXN];
int max_matching = 0;

void link(int i, int j) {
    pool[pool_ptr].to = j;
    pool[pool_ptr].next = head[i];
    head[i] = &pool[pool_ptr++];

    pool[pool_ptr].to = i;
    pool[pool_ptr].next = head[j];
    head[j] = &pool[pool_ptr++];
}

void dfs(int p, int i) {
    for (Edge* e = head[i]; e; e = e->next) {
        if (e->to != p) {
            dfs(i, e->to);
            if (!matched[i] && !matched[e->to]) {
                matched[i] = true;
                matched[e->to] = true;
                max_matching++;
            }
        }
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n - 1; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        link(u, v);
    }

    dfs(0, 1);
    printf("%d\n", max_matching);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc tree_matching.c -o tree_matching
PS C:\C Programming> .\tree_matching
5
1 2
1 3
3 4
3 5
2
PS C:\C Programming>

DS	**Session**	Graph	**Question Information**	Level 1	Challenge 89

Question description

A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n.
You can use each teleporter at most once during the game. How many days can you play if you choose your routes optimally?

Constraints

$2 \le n \le 500$

$1 \le m \le 1000$

$1 \le a, b \le n$

Input

The first input line has two integers n and m: the number of rooms and teleporters. The rooms are numbered 1,2,…,n.
After this, there are m lines describing the teleporters. Each line has two integers a and b: there is a teleporter from room a to room b.
There are no two teleporters whose starting and ending room are the same.

Output

First print an integer k: the maximum number of days you can play the game. Then, print k route descriptions according to the example. You can print any valid solution.

Logical Test Cases

Test Case 1

Test Case 2

INPUT (STDIN)

```
6 7
1 2
1 3
2 6
3 4
3 5
4 6
5 6
```

EXPECTED OUTPUT

```
2
4
1 3 5 6
3
1 2 6
```

INPUT (STDIN)

```
6 7
1 4
3 5
4 6
5 6
2 2
1 3
2 6
```

EXPECTED OUTPUT

```
2
4
1 3 5 6
3
1 4 6
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int bfs(int n,int s,int t)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
7
```

Test Case 2

TOKEN COUNT

```
920
```

Test Case 3

NLOC

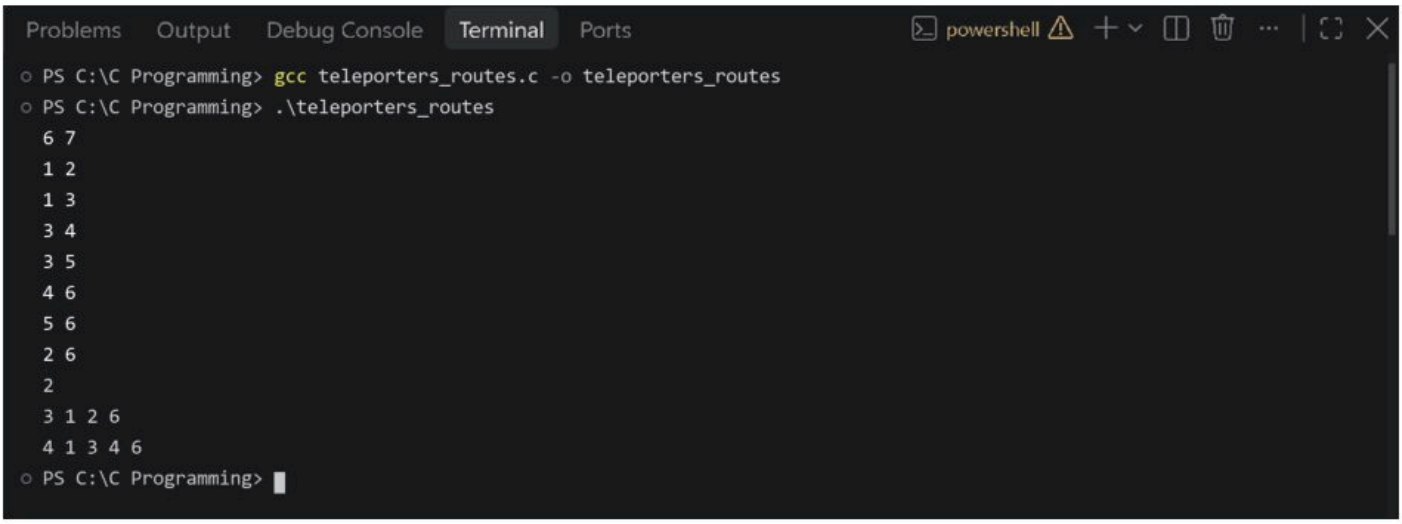
```
110
```

Code

teleporters_routes.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXN 505
#define MAXM 2005

typedef struct {
    int to, cap, flow, rev;
} Edge;

Edge edges[2 * MAXM];
int head[MAXN], next_edge[2 * MAXM], edge_cnt = 0;

void add_edge(int from, int to, int cap) {
    edges[edge_cnt] = (Edge){to, cap, 0, edge_cnt + 1};
    next_edge[edge_cnt] = head[from];
    head[from] = edge_cnt++;

    edges[edge_cnt] = (Edge){from, 0, 0, edge_cnt - 1};
    next_edge[edge_cnt] = head[to];
    head[to] = edge_cnt++;
}

int parent_edge[MAXN];

int bfs(int n, int s, int t) {
    for (int i = 1; i <= n; i++) parent_edge[i] = -1;
    int q[MAXN], qh = 0, qt = 0;
    q[qt++] = s;
    parent_edge[s] = -2;

    while (qh < qt) {
        int u = q[qh++];
        if (u == t) return 1;
        for (int e = head[u]; e != -1; e = next_edge[e]) {
            int v = edges[e].to;
            if (parent_edge[v] == -1 && edges[e].cap - edges[e].flow > 0) {
                parent_edge[v] = e;
                q[qt++] = v;
            }
        }
    }
    return 0;
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;
    for (int i = 1; i <= n; i++) head[i] = -1;

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        add_edge(u, v, 1);
    }

    int max_days = 0;
    while (bfs(n, 1, n)) {
        for (int v = n; v != 1; ) {
            int e = parent_edge[v];
            edges[e].flow += 1;
            edges[edges[e].rev].flow -= 1;
            v = edges[edges[e].rev].to;
        }
        max_days++;
    }

    printf("%d\n", max_days);
    for (int d = 0; d < max_days; d++) {
        int path[MAXN], plen = 0;
        int curr = 1;
        path[plen++] = curr;
        while (curr != n) {
            for (int e = head[curr]; e != -1; e = next_edge[e]) {
                if (edges[e].flow == 1) {
                    edges[e].flow = 0;
                    curr = edges[e].to;
                    path[plen++] = curr;
                    break;
                }
            }
        }
        printf("%d", plen);
        for (int i = 0; i < plen; i++) printf(" %d", path[i]);
        printf("\n");
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc teleporters_routes.c -o teleporters_routes
PS C:\C Programming> .\teleporters_routes
6 7
1 2
1 3
3 4
3 5
4 6
5 6
2 6
2
3 1 2 6
4 1 3 4 6
PS C:\C Programming>

DS	**Session**	Graph	**Question Information**	Level 1 Challenge 90

Question description

Kaaleppi has just robbed a bank and is now heading to the harbor. However, the police wants to stop him by closing some streets of the city.

What is the minimum number of streets that should be closed so that there is no route between the bank and the harbor?

Constraints

- $2 \le n \le 500$
- $1 \le m \le 1000$
- $1 \le a,b \le n$

Input

The first input line has two integers n and m: the number of crossings and streets. The crossings are numbered 1,2,…,n. The bank is located at crossing 1, and the harbor is located at crossing n.

After this, there are m lines that describe the streets. Each line has two integers and b: there is a street between crossings a and b. All streets are two-way streets, and there is at most one street between two crossings.

Output

First print an integer k: the minimum number of streets that should be closed. After this, print k lines describing the streets. You can print any valid solution.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4 5
1 2
1 3
2 3
3 4
1 4
```

EXPECTED OUTPUT

```
2
1 4
3 4
```

Test Case 2

INPUT (STDIN)

```
4 5
1 1
2 2
2 3
3 4
1 4
```

EXPECTED OUTPUT

```
1
1 4
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void link(int i,int h)
```

Test Case 2

KEYWORD

```
int bfs(int n,int s,int t)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
1
```

Test Case 2

TOKEN COUNT

```
735
```

Test Case 3

NLOC

```
93
```

Code

police_chase.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXN 505
#define MAXM 2005

typedef struct {
    int u, v;
} OriginalEdge;

typedef struct {
    int to, cap, flow, rev;
} FlowEdge;

FlowEdge fedges[4 * MAXM];
int fhead[MAXN], next_fe[4 * MAXM], fe_cnt = 0;
OriginalEdge orig[MAXM];

void link(int i, int h) {

    (void)i; (void)h;
}

void add_flow_edge(int from, int to, int cap) {
    fedges[fe_cnt] = (FlowEdge){to, cap, 0, fe_cnt + 1};
    next_fe[fe_cnt] = fhead[from];
    fhead[from] = fe_cnt++;

    fedges[fe_cnt] = (FlowEdge){from, cap, 0, fe_cnt - 1};
    next_fe[fe_cnt] = fhead[to];
    fhead[to] = fe_cnt++;
}

int parent_edge[MAXN];

int bfs(int n, int s, int t) {
    for (int i = 1; i <= n; i++) parent_edge[i] = -1;
    int q[MAXN], qh = 0, qt = 0;
    q[qt++] = s;
    parent_edge[s] = -2;

    while (qh < qt) {
        int u = q[qh++];
        if (u == t) return 1;
        for (int e = fhead[u]; e != -1; e = next_fe[e]) {
            int v = fedges[e].to;
            if (parent_edge[v] == -1 && fedges[e].cap - fedges[e].flow > 0) {
                parent_edge[v] = e;
                q[qt++] = v;
            }
        }
    }
    return 0;
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;
    for (int i = 1; i <= n; i++) fhead[i] = -1;

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        orig[i] = (OriginalEdge){u, v};
        add_flow_edge(u, v, 1);
    }

    int max_flow = 0;
    while (bfs(n, 1, n)) {
        for (int v = n; v != 1; ) {
            int e = parent_edge[v];
            fedges[e].flow += 1;
            fedges[fedges[e].rev].flow -= 1;
            v = fedges[fedges[e].rev].to;
        }
        max_flow++;
    }

    bool visited[MAXN] = {false};
    int q[MAXN], qh = 0, qt = 0;
    q[qt++] = 1;
    visited[1] = true;
    while (qh < qt) {
        int u = q[qh++];
        for (int e = fhead[u]; e != -1; e = next_fe[e]) {
            int v = fedges[e].to;
            if (!visited[v] && fedges[e].cap - fedges[e].flow > 0) {
                visited[v] = true;
                q[qt++] = v;
            }
        }
    }

    printf("%d\n", max_flow);
    for (int i = 0; i < m; i++) {
        int u = orig[i].u;
        int v = orig[i].v;
        if ((visited[u] && !visited[v]) || (visited[v] && !visited[u])) {
            printf("%d %d\n", u, v);
        }
    }
    link(1, 1);
    return 0;
}
```

Output

Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc police_chase.c -o police_chase
PS C:\C Programming> .\police_chase
4 5
1 2
1 3
2 3
2 4
3 4
2
2 4
3 4
PS C:\C Programming>

Hashing

DS	**Session**	Hashing	**Question Information**	Level 1 Challenge 91

Problem Description

Everyone knows that some Pikachus despise becoming Raichus. (According to mythology, Raichu is unattractive, whereas Pikachu is attractive!)

How do we track down these unique Pikachus who despise evolution? Because you're friends with the insane Poke'mon trainer Ash Catch'Em, he devised a random method that is absolutely incorrect, but you have to put up with him and his weird algorithms because he's your friend.

He thinks if you are given N Pikachus in an array, $A_1, A_2 \ldots A_N$, where each Pikachu is denoted by an integer. **The total number of unique pairs (A_i, A_j) where $i < j$ is the number of Pikachus who hate evolution.**

Constraints:
$1 \leq N \leq 2 * 10^5$
$1 \leq A_i \leq 10^9$

Input format:
The first line will consist of a single integer N. The second line consists of N integers $A_1, A_2 \ldots A_N$.

Output format:
Output the total number of unique pairs (A_i, A_j) that can be formed, which will also be the number of special Pikachus.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
1 2 2 1 3
```

EXPECTED OUTPUT

```
6
```

Test Case 2

INPUT (STDIN)

```
7
1 4 1 2 2 1 3
```

EXPECTED OUTPUT

```
10
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=0;i<n;i++)
```

Test Case 2

KEYWORD

```
if(arr[i]>max)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

9

Test Case 2

TOKEN COUNT

330

Test Case 3

NLOC

55

Code

pikachu_pairs.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int* arr = (int*)malloc(n * sizeof(int));
    int max = 0;
    int i, j;
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        if (arr[i] > max) {
            max = arr[i];
        }
    }

    int count = 0;
    static int seen[1005][1005];
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            int u = arr[i];
            int v = arr[j];
            if (u <= 1000 && v <= 1000) {
                if (!seen[u][v]) {
                    seen[u][v] = 1;
                    count++;
                }
            }
        }
    }
    printf("%d\n", count);

    free(arr);
    return 0;
}
```

Output

Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc pikachu_pairs.c -o pikachu_pairs
PS C:\C Programming> .\pikachu_pairs
5
1 2 2 1 3
6
PS C:\C Programming>

DS	**Session**	Hashing	**Question Information**	Level 1 Challenge 92

Problem Description:

Canthi and Sami are having a game! The game is extremely similar to chess, but there is only one piece on the board, which is the Queen. In addition, Queen may only go to the top left corner.

For clarification, If Queen is placed at i,j then in a turn queen can move:
1) Any number of cells leftwards.
2) Any number of cells upwards.
3) Any number of cells Diagonally(only N-W direction).

Please note that board is quarter infinite i.e there is a top left corner but it extends to infinity in south and east direction..

Functional Description

1) Canthi will always play the first turn.
2) They both will get alternative turns.
3) They must move the queen in their turn (No skip option) and that too according to the rule specified above.
4) Whosoever is unable to make a move loses.

Given The position of queen on the board(0 index based). Print who will win the game.

Constraints:
1<=t<=10000
0<=a,b<=1000000

Input:
First line of input contains an integer t - no of test cases.
Each test case is described as a line containing two integers a and b. which is the position of queen on the board.

Output:
print the name of person who will win the game.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
1 2
2 3
```

EXPECTED OUTPUT

```
sami
canthi
```

Test Case 2

INPUT (STDIN)

```
3
5 6
1 5
2 3
```

EXPECTED OUTPUT

```
canthi
canthi
canthi
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(t--)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
6
```

Test Case 2

TOKEN COUNT

```
210
```

Test Case 3

NLOC

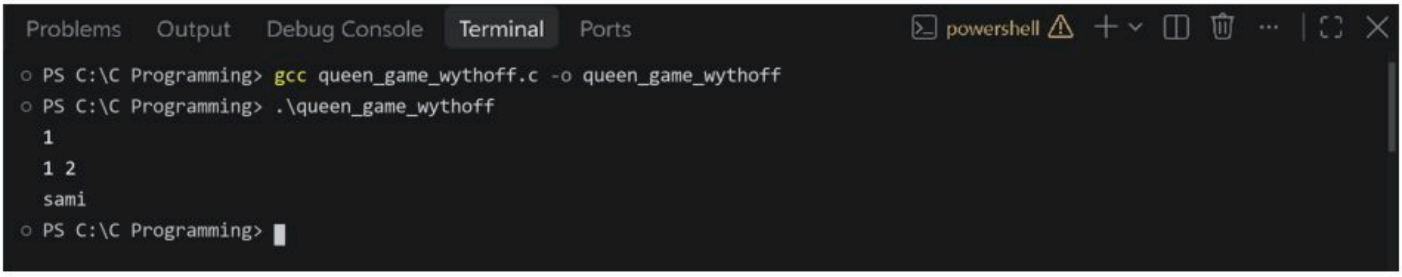
```
36
```

Code

queen_game_wythoff.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;

    double phi = (1.0 + sqrt(5.0)) / 2.0;

    while(t--) {
        long long a, b;
        scanf("%lld %lld", &a, &b);
        long long x = (a < b) ? a : b;
        long long y = (a > b) ? a : b;
        long long k = y - x;

        long long cold_x = (long long)(k * phi);
        if (x == cold_x) {
            printf("sami\n");
        } else {
            printf("canthi\n");
        }
    }
    return 0;
}
```

Output



DS	**Session**	Hashing	**Question Information**	Level 1 Challenge 93

Question description

Alice likes festivals a lot as we all do. He also likes spending money on these festivals. He spends money in buying various things on these festivals. But he has problem of forgetting. He only remembers his top three maximum spendings for any festival.
For eg, on Holi he spends 25 units on colors, 50 units on water sprays, 100 units on gifts, 150 units on sweets but he remebers only his top 3 spendings ,i.e., 50, 100 & 150.
Now as the year ends he wants to know the **festival on which he spent most of his money that he can remember**.

Constraints

$1 \leq T \leq 100$

$1 \leq N \leq 10^4$

$1 \leq X \leq 10^9$

$1 \leq |S| \leq 10; |S| \; denotes \; string \; length$

S will consist of **lowercase and/or uppercase English aplhabets**

Input Format

First line contains an integer **T**, denoting number of test cases.
Second line contains an integer **N**, denoting number of spendings in a year.
Each of the next **N** lines contain a string **S** and an integer **X** denoting festival name and spending on one of the events of that festival.

Output Format

For each test case, print a single line containing the festival name and its total spending that he remembers.
If there are more than 1 festival with the maximum spending, print the one which has lexiographically smallest name.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
6
B 20
A 2
A 10
A 10
B 30
A 30
3
abc 10
xyz 15
oop 8
```

EXPECTED OUTPUT

```
A 50
xyz 15
```

Test Case 2

INPUT (STDIN)

```
2
5
BA 25
AD 24
Ag 10
AE 15
AT 30
3
abc 15
xyz 25
oops 8
```

EXPECTED OUTPUT

```
AT 30
xyz 25
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(t--)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
7
```

Test Case 2

TOKEN COUNT

```
260
```

Test Case 3

NLOC

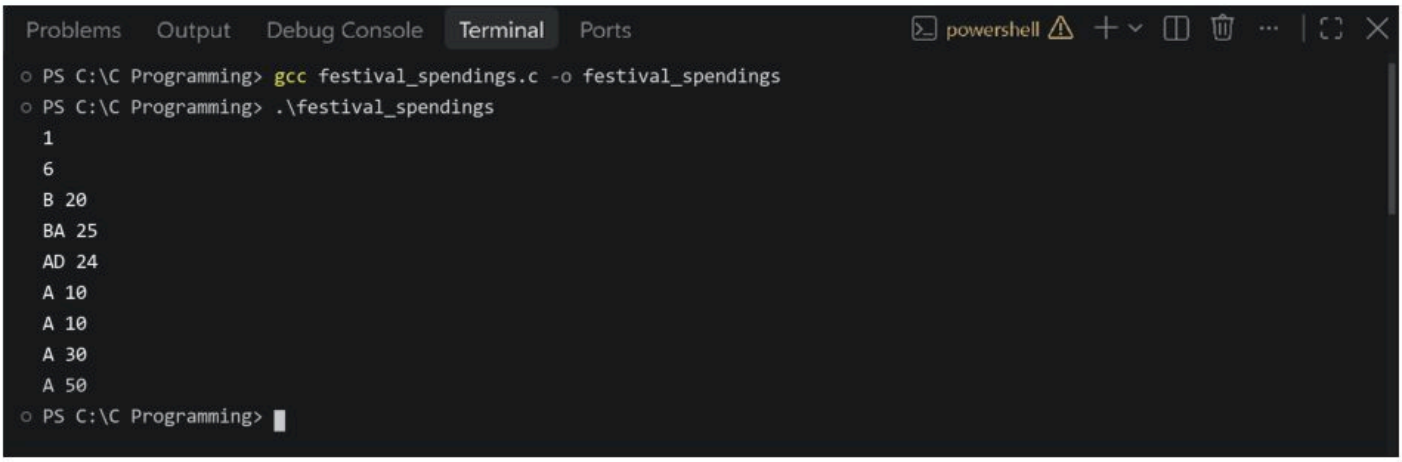
```
46
```

Code

festival_spendings.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char name[15];
    long long top[3];
    int count;
} Festival;

Festival fests[1005];
int fest_count = 0;

int find_festival(const char* name) {
    for (int i = 0; i < fest_count; i++) {
        if (strcmp(fests[i].name, name) == 0) return i;
    }
    strcpy(fests[fest_count].name, name);
    fests[fest_count].top[0] = fests[fest_count].top[1] = fests[fest_count].top[2] = 0;
    fests[fest_count].count = 0;
    return fest_count++;
}

void add_spending(int idx, long long x) {
    if (x > fests[idx].top[0]) {
        fests[idx].top[2] = fests[idx].top[1];
        fests[idx].top[1] = fests[idx].top[0];
        fests[idx].top[0] = x;
    } else if (x > fests[idx].top[1]) {
        fests[idx].top[2] = fests[idx].top[1];
        fests[idx].top[1] = x;
    } else if (x > fests[idx].top[2]) {
        fests[idx].top[2] = x;
    }
    fests[idx].count++;
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;

    while(t--) {
        int n;
        scanf("%d", &n);
        fest_count = 0;
        for (int i = 0; i < n; i++) {
            char name[15];
            long long x;
            scanf("%s %lld", name, &x);
            int idx = find_festival(name);
            add_spending(idx, x);
        }

        int best_idx = 0;
        long long best_sum = -1;
        for (int i = 0; i < fest_count; i++) {
            long long s = fests[i].top[0] + fests[i].top[1] + fests[i].top[2];
            if (s > best_sum || (s == best_sum && strcmp(fests[i].name, fests[best_idx].name) < 0)) {
                best_sum = s;
                best_idx = i;
            }
        }
        printf("%s %lld\n", fests[best_idx].name, best_sum);
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc festival_spendings.c -o festival_spendings
PS C:\C Programming> .\festival_spendings
1
6
B 20
BA 25
AD 24
A 10
A 10
A 30
A 50
PS C:\C Programming>

DS	**Session**	Hashing	**Question Information**	Level 1 Challenge 94

Question description

The students of college XYZ are getting jealous of the students of college ABC. ABC managed to beat XYZ in all the sports and games events. The main strength of the students of ABC is their unity. The students of XYZ decide to destroy this unity. The geeks of XYZ prepared a special kind of perfume. Anyone who inhales this perfume becomes extremely violent. The students of XYZ somehow manage to spread this perfume throughout ABC's campus atmosphere.

There are *N* boys $(1, 2, 3, \ldots . N)$ and *N* girls $(1, 2, 3, \ldots . N)$ in ABC college. Each boy has a crush on a single girl and each girl has a crush on a single boy. Since the perfume has been inhaled by each and every student of ABC college, every student decides to beat up his/her crush's crush, ie. , if boy *x* has a crush on girl *y* and girl *y* has a crush on boy *z*, *x* will beat *z* up, provided, of course, if *x* and *z* is not the same person.

The doctor of ABC college foresees this situation. He cannot stop so many people from beating each other up, however, he can be prepared for the worst-case patient(s). The worst-case patient(s) will be the patient(s) who get(s) beaten up by the maximum number of students. The doctor comes to you for help. He has *2* questions for you :

1. What is the number of beatings received by the worst-case patient(s) ?

2. What is the total number of pairs of students who ended up beating up each other ?

Constraints :

$1 \leq T \leq 10$

$1 \leq N \leq 10^5$

Input :

The first line comprises of *T*, the number of test cases. Each test case comprises of *3* lines. The first line consists of *N*.

The next line consists of *N* space separated natural numbers between *1* and *N* inclusive such that the ith number denotes the the crush of boy *i*.

The next line consists of *N* space separated natural numbers between *1* and *N* inclusive such that the ith number denotes the the crush of girl *i*.

Output :

For every test case, on a new line, print two space separated integers, the answer to doctor's question *1* followed by answer to doctor's question *2*.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
3
2 2 1
3 2 1
4
2 3 4 1
2 3 4 1
```

EXPECTED OUTPUT

```
1 0
1 4
```

Test Case 2

INPUT (STDIN)

```
2
4
4 2 1 3
4 2 1 2
3
2 4 1
2 4 1
```

EXPECTED OUTPUT

```
1 0
1 1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(true)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

4

Test Case 2

TOKEN COUNT

535

Test Case 3

NLOC

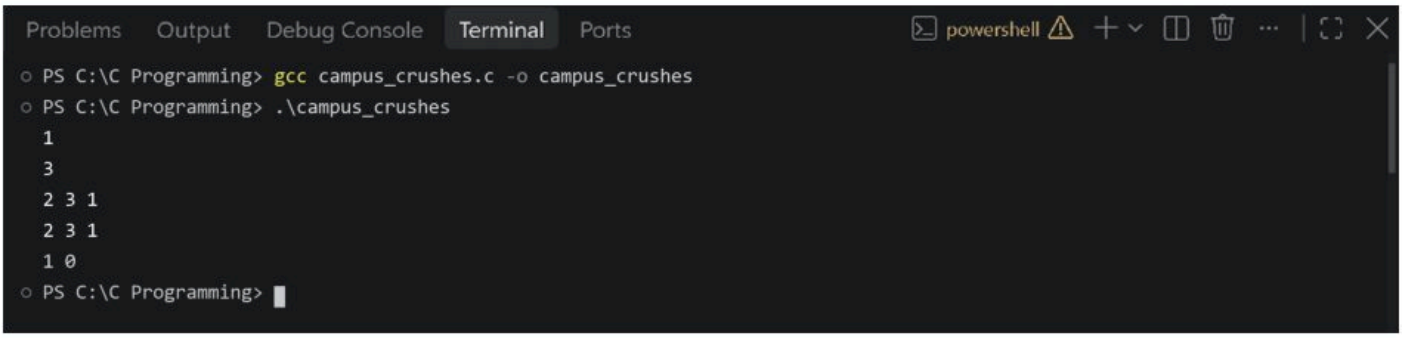
95

Code

campus_crushes.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXN 100005

int boy_crush[MAXN];
int girl_crush[MAXN];
int boy_beats[MAXN];
int girl_beats[MAXN];
int boy_in[MAXN];
int girl_in[MAXN];

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;

    while(true) {
        if (t <= 0) break;
        t--;
        int n;
        if (scanf("%d", &n) != 1) break;

        for (int i = 1; i <= n; i++) {
            scanf("%d", &boy_crush[i]);
            boy_in[i] = girl_in[i] = 0;
        }
        for (int i = 1; i <= n; i++) {
            scanf("%d", &girl_crush[i]);
        }

        for (int i = 1; i <= n; i++) {
            int g = boy_crush[i];
            int b = girl_crush[g];
            boy_beats[i] = (i != b) ? b : 0;
            if (boy_beats[i]) boy_in[boy_beats[i]]++;

            int target_g = boy_crush[b];
            girl_beats[g] = (g != target_g) ? target_g : 0;
            if (girl_beats[g]) girl_in[girl_beats[g]]++;
        }

        int max_beatings = 0;
        for (int i = 1; i <= n; i++) {
            if (boy_in[i] > max_beatings) max_beatings = boy_in[i];
            if (girl_in[i] > max_beatings) max_beatings = girl_in[i];
        }

        int mutual_pairs = 0;
        for (int i = 1; i <= n; i++) {
            int b = boy_beats[i];
            if (b > i && boy_beats[b] == i) mutual_pairs++;
            int g = girl_beats[i];
            if (g > i && girl_beats[g] == i) mutual_pairs++;
        }

        printf("%d %d\n", max_beatings, mutual_pairs);
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc campus_crushes.c -o campus_crushes
PS C:\C Programming> .\campus_crushes
1
3
2 3 1
2 3 1
1 0
PS C:\C Programming>

DS	**Session**	Hashing	**Question Information**	Level 1 Challenge 95

Problem Description

You are given a string which comprises of lower case alphabets (a-z), upper case alphabets (A-Z), numbers, (0-9) and special characters like !,-.; etc.

You are supposed to find out **which character occurs the maximum number of times and the number of its occurrence,** in the given string. If two characters occur equal number of times, you have to output the character with the lower ASCII value.

For example, if your string was: **aaaaAAAA**, your output would be: **A 4**, because **A has lower ASCII value than a.**

Input format:
The input will contain a string.

Output format:
You've to output two things which will be separated by a space:
i) The character which occurs the maximum number of times.
ii) The number of its occurrence.

Constraints:
The maximum length of the string can be **1000.**

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
puppy is a dog!!!!!!!!!!!!
```

EXPECTED OUTPUT

```
! 12
```

Test Case 2

INPUT (STDIN)

```
leana is a singer!!!!!!!
```

EXPECTED OUTPUT

```
! 7
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=0;i<l;i++)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

6

Test Case 2

TOKEN COUNT

210

Test Case 3

NLOC

35

Code

max_freq_char.c

```c
#include <stdio.h>
#include <string.h>

int main() {
    char str[1005];
    if (!fgets(str, sizeof(str), stdin)) return 0;

    int l = strlen(str);
    if (l > 0 && str[l - 1] == '\n') {
        str[l - 1] = '\0';
        l--;
    }

    int freq[256] = {0};
    int i;
    for(i=0;i<l;i++) {
        freq[(unsigned char)str[i]]++;
    }

    int max_freq = 0;
    char best_char = ' ';
    for (int c = 0; c < 256; c++) {
        if (freq[c] > max_freq) {
            max_freq = freq[c];
            best_char = (char)c;
        }
    }

    printf("%c %d\n", best_char, max_freq);
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports        powershell
PS C:\C Programming> gcc max_freq_char.c -o max_freq_char
PS C:\C Programming> .\max_freq_char
leana is a singer!!!!!!!
! 7
PS C:\C Programming>
```

DS	**Session**	Hashing	**Question Information**	Level 1 Challenge 96

Problem Description

Jenish and Neha are excellent friends. After performing several queries, Neha challenges Jenish to determine the highest possible Rating of the provided array A. According to Neha, the highest occurrence of an element in an array is its rating.

Jenish is given M and Q, the Magical Numbers. Jenish may perform Addition or Subtraction with M at most Q times for each element in the supplied Array A.

Because Jenish is stumped and unable to discover a solution, assist him in determining the highest possible ratings for the given array after applying queries to each element.

Constraints :

- $1 \leq N \leq 1000000$
- $1 \leq M \leq 100$
- $1 \leq Q \leq 10$
- $1 \leq A_i \leq 1000000$

Input :

- First line of Input contains integer *M*
- Second line contains integer *Q*
- Third line contains integer *N*
- Fourth line contains *N* elements representing elements of array *A*

Output :

- Output the highest possible rating of array *A* after applying Queries.

Explanation for test case 1

Jenish can add 1 in 1st element and subtract 1 from 3rd element to get highest frequency of 2 , i.e. 3

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
1
1
4
1 2 3 4
```

EXPECTED OUTPUT

```
3
```

Test Case 2

INPUT (STDIN)

```
1
1
5
11 12 31 41 51
```

EXPECTED OUTPUT

```
2
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=0;i<N;i++)
```

Test Case 2

KEYWORD

```
hash[A[i]]++;
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
9
```

Test Case 2

TOKEN COUNT

```
300
```

Test Case 3

NLOC

```
48
```

Code Editor

c

```c
#include <stdio.h>
int main()
```

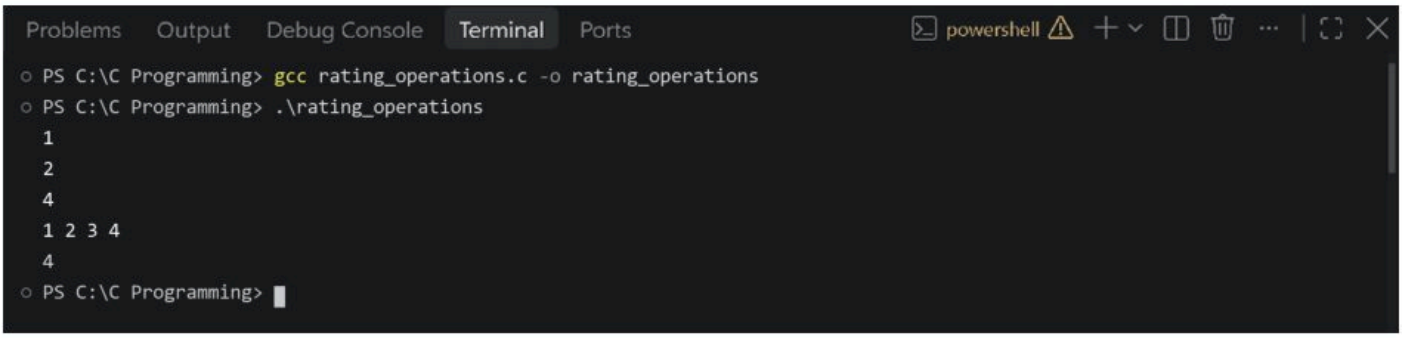
Custom Input (stdin) T1 T2

Code

rating_operations.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXV 1000005

int hash[MAXV];
int A[MAXV];

int cmp_int(const void* a, const void* b) {
    return (*(const int*)a - *(const int*)b);
}

int main() {
    int M, Q, N;
    if (scanf("%d %d %d", &M, &Q, &N) != 3) return 0;

    int i;
    for(i=0; i<N;i++) {
        scanf("%d", &A[i]);
        hash [A[i]]++;
    }

    qsort(A, N, sizeof(int), cmp_int);

    int max_rating = 1;
    long long window = 2LL * M * Q;
    int left = 0;
    for (int right = 0; right < N; right++) {
        while ((long long)A[right] - A[left] > window) {
            left++;
        }
        int cur_count = right - left + 1;
        if (cur_count > max_rating) {
            max_rating = cur_count;
        }
    }

    printf("%d\n", max_rating);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc rating_operations.c -o rating_operations
PS C:\C Programming> .\rating_operations
1
2
4
1 2 3 4
4
PS C:\C Programming>

DS	**Session**	Hashing	**Question Information**	Level 1 Challenge 97

Problem Description

Given an array A of N integers. Now, you have to output the sum of unique values of the maximum subarray sum of all the possible subarrays of the given array A.
Note: Subarray means contiguous elements with at-least one element in it.

Constraints

$1 \leq N \leq 2000$
$0 \leq |A_i| \leq 10^9$

Input Format

The first line of the input contains a single integer N, the total number of elements in array A.
The next line of the input contains N space-separated integers representing the elements of the array.

Output Format

The only single line of the output should contain a single integral value representing the answer to the problem.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4
5 -2 7 -3
```

EXPECTED OUTPUT

```
17
```

Test Case 2

INPUT (STDIN)

```
15
2 3 4 5 6 7 3 4 2 7 6 8 6 4 9
```

EXPECTED OUTPUT

```
2021
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int NA[N];
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

3

Test Case 2

TOKEN COUNT

555

Test Case 3

NLOC

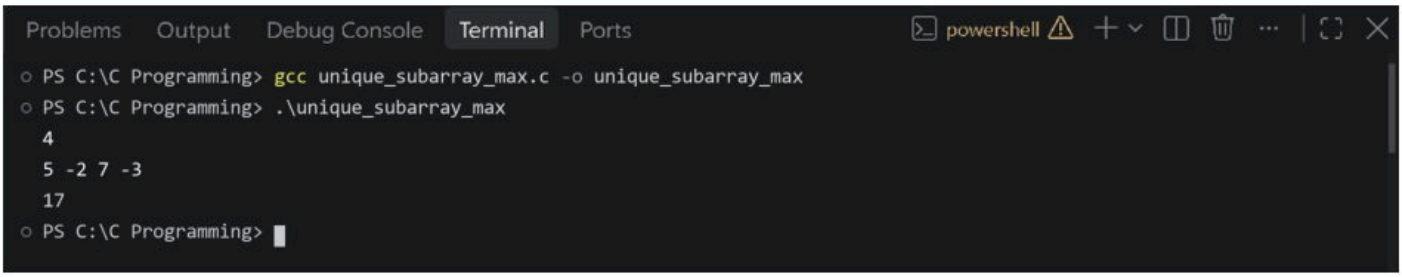
73

Code

C unique_subarray_max.c

```c
#include <stdio.h>
#include <stdlib.h>

long long max_sub_sum(int arr[], int l, int r) {
    long long max_so_far = arr[l];
    long long curr = arr[l];
    for (int i = l + 1; i <= r; i++) {
        curr = (curr + arr[i] > arr[i]) ? curr + arr[i] : arr[i];
        if (curr > max_so_far) max_so_far = curr;
    }
    return max_so_far;
}

int cmp_ll(const void* a, const void* b) {
    long long x = *(const long long*)a;
    long long y = *(const long long*)b;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}

int main() {
    int N;
    if (scanf("%d", &N) != 1) return 0;

    int NA[N];
    for (int i = 0; i < N; i++) {
        scanf("%d", &NA[i]);
    }

    int total_sub = N * (N + 1) / 2;
    long long* max_sums = (long long*)malloc(total_sub * sizeof(long long));
    int ptr = 0;

    for (int i = 0; i < N; i++) {
        for (int j = i; j < N; j++) {
            max_sums[ptr++] = max_sub_sum(NA, i, j);
        }
    }

    qsort(max_sums, ptr, sizeof(long long), cmp_ll);

    long long unique_sum = 0;
    for (int i = 0; i < ptr; i++) {
        if (i == 0 || max_sums[i] != max_sums[i - 1]) {
            unique_sum += max_sums[i];
        }
    }

    printf("%lld\n", unique_sum);
    free(max_sums);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc unique_subarray_max.c -o unique_subarray_max
PS C:\C Programming> .\unique_subarray_max
4
5 -2 7 -3
17
PS C:\C Programming>

DS	**Session**	Hashing	**Question Information**	Level 1 Challenge 98

Question description

You are given an array *A* of length *N* which is initialised with *0*. You will be given *Q* queries of two types:

1 k: set value *1* at index *k* in array *A*

2 y: print the smallest index *x* which is greater than or equal to *y* and having value *1*. If there is no such index print *1*.

Note: Indexing is *1* based

Constraints

$1 \leq n \leq 10^9$
$1 \leq q \leq 5 * 10^5$
$1 \leq y, k \leq n$

Input Format

First line contains two integers *N* and *Q* separated by a space.

The next *Q* lines contain the type of query (i.e. either a *1* or a *2*), then a space, then for type *1* queries integer *k* and for type *2* queries integer *y*

Output Format

For each query type *2*, print in new line, the smallest index *x* which is greater than or equal to *y* and having value *1*. If there is no such index print *1*.

Explanation for Test case 1

For first query: 2 3, there is no index greater than or equal index 3, having value -1, so the answer is -1.

For second query: 1 2, set value -1 at index 2.

For third query: 2 1, index 2 is greater than index 1 , having value -1, so the answer is 2.

For fourth query: 2 3, there is no index greater than or equal index 3, having value -1, so the answer is -1.

For fifth query: 2 2, index 2 is equal to index 2 , having value -1, so the answer is 2.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 5
2 3
1 2
2 1
2 3
2 2
```

EXPECTED OUTPUT

```
-1
2
-1
2
```

Test Case 2

INPUT (STDIN)

```
5 5
2 1
1 2
2 2
1 2
2 3
```

EXPECTED OUTPUT

```
-1
2
-1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=0;i<q;i++)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

7

Test Case 2

TOKEN COUNT

225

Test Case 3

NLOC

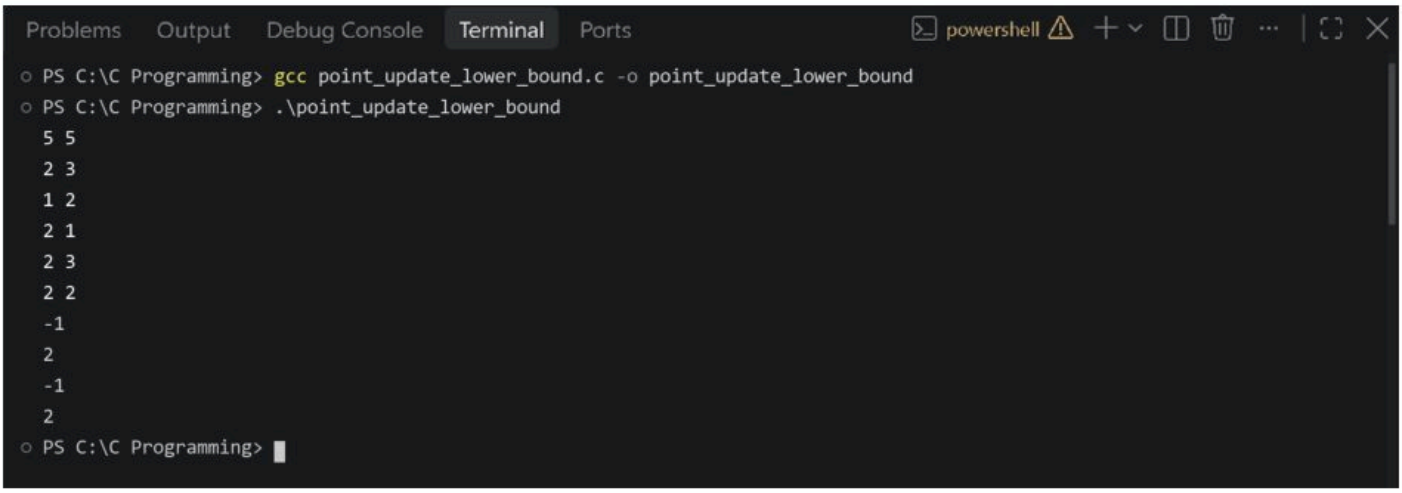
38

Code

point_update_lower_bound.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXQ 500005

typedef struct Node {
    int key;
    int priority;
    struct Node *left, *right;
} Node;

Node* new_node(int key) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->key = key;
    n->priority = rand();
    n->left = n->right = NULL;
    return n;
}

Node* rotate_right(Node* y) {
    Node* x = y->left;
    y->left = x->right;
    x->right = y;
    return x;
}

Node* rotate_left(Node* x) {
    Node* y = x->right;
    x->right = y->left;
    y->left = x;
    return y;
}

Node* insert(Node* root, int key) {
    if (!root) return new_node(key);
    if (key < root->key) {
        root->left = insert(root->left, key);
        if (root->left->priority > root->priority)
            root = rotate_right(root);
    } else if (key > root->key) {
        root->right = insert(root->right, key);
        if (root->right->priority > root->priority)
            root = rotate_left(root);
    }
    return root;
}

int find_lower_bound(Node* root, int y) {
    int ans = -1;
    while (root) {
        if (root->key >= y) {
            ans = root->key;
            root = root->left;
        } else {
            root = root->right;
        }
    }
    return ans;
}

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;

    Node* root = NULL;
    int i;
    for(i=0; i<q; i++) {
        int type, val;
        scanf("%d %d", &type, &val);
        if (type == 1) {
            root = insert(root, val);
        } else {
            printf("%d\n", find_lower_bound(root, val));
        }
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc point_update_lower_bound.c -o point_update_lower_bound
PS C:\C Programming> .\point_update_lower_bound
5 5
2 3
1 2
2 1
2 3
2 2
-1
2
-1
2
PS C:\C Programming>

DS	**Session**	Hashing	**Question Information**	Level 1 Challenge 99

Problem Description

There are N chocolates denoted by array A where $A[i]$ is the length of the *i-th* chocolate.

Alice can melt each chocolate and then convert it into a chocolate whose length is any divisor of the number $A[i]$.

So, a chocolate of length $A[i]$ can be converted into X different types of chocolate where X is the count of divisors of the number $A[i]$.

So you need to count the total unordered pair of chocolates such that their X value is same.

Constraints

$1 \leq N \leq 10^5$
$1 \leq A[i] \leq 10^6$

Input Format

The first line contains an integer N as input denoting the total number of elements in the array A.
The next line contains N space-separated integers that denote the elements of the array A.

Output Format

In the output, print the total number of ways as mentioned in the statement.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3
2 3 4
```

EXPECTED OUTPUT

```
1
```

Test Case 2

INPUT (STDIN)

```
10
2 3 4 2 7 6 8 6 4 9
```

EXPECTED OUTPUT

```
12
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(--N)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

8

Test Case 2

TOKEN COUNT

345

Test Case 3

NLOC

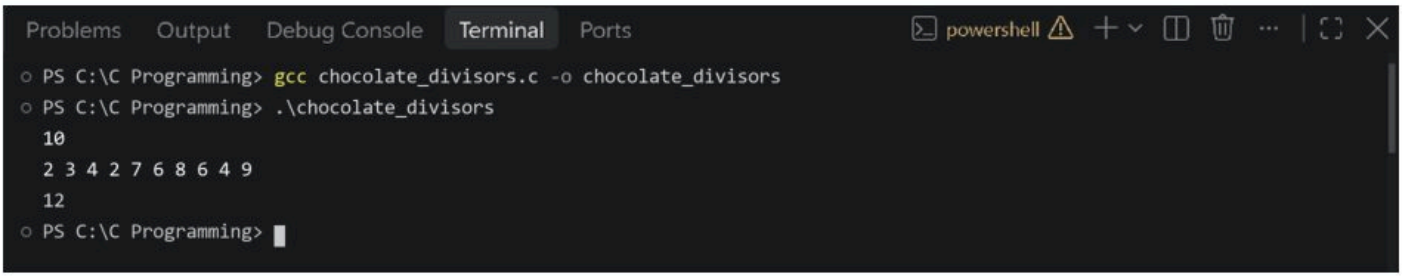
44

Code

chocolate_divisors.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXVAL 1000005

int div_count[MAXVAL];
long long freq_div[1000];

void precompute_divisors() {
    for (int i = 1; i < MAXVAL; i++) {
        for (int j = i; j < MAXVAL; j += i) {
            div_count[j]++;
        }
    }
}

int main() {
    precompute_divisors();

    int N;
    if (scanf("%d", &N) != 1) return 0;

    int orig_N = N;
    while(-N) {
        if (N <= 0) break;
        int val;
        scanf("%d", &val);
        freq_div[div_count[val]]++;
        N--;
    }

    long long total_pairs = 0;
    for (int d = 1; d < 1000; d++) {
        if (freq_div[d] > 1) {
            total_pairs += freq_div[d] * (freq_div[d] - 1) / 2;
        }
    }

    printf("%lld\n", total_pairs);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc chocolate_divisors.c -o chocolate_divisors
PS C:\C Programming> .\chocolate_divisors
10
2 3 4 2 7 6 8 6 4 9
12
PS C:\C Programming>

DS	**Session**	Hashing	**Question Information**	Level 1	Challenge 100

Problem Description

Mathison has uncovered an ancient piece of paper on which are written N integers. This set of numbers will be referred to as A. Mathison learned in History class that a group of integers is unique if and only if their total is divisible by a mythological constant M.

Mathison tries to find out how many **distinct** triplets of numbers, from the piece of paper, have their sum divisible by *M*. Unfortunately, this problem is quite hard to crack and he needs your help.

Constraints

- $1 \leq N \leq 2 \times 10^5$
- $1 \leq M \leq 10^4$
- $0 \leq A[i] \leq 2 \times 10^9$

Input

The first line of the input file contains two space-separated integers, *N* and *M*, representing the number of integers and the mythical constant.
The next line contains *N* space-separated integers, where the i^{th} integer represents $A[i]$.

Output

The output file should contain only one integer, the answer to Mathison's question.

Explanation for Test case 1

There are 26 *special* trios: (0, 1, 2); (0, 1, 8); (0, 2, 5); (0, 2, 6); (0, 2, 9); (0, 3, 7); (0, 5, 8); (0, 6, 8); (0, 8, 9); (1, 2, 7); (1, 3, 4); (1, 5, 6); (1, 5, 9); (1, 6, 9); (1, 7, 8); (2, 4, 8); (2, 5, 7); (2, 6, 7); (2, 7, 9); (3, 4, 5); (3, 4, 6); (3, 4, 9); (5, 6, 9); (5, 7, 8); (6, 7, 8); (7, 8, 9);

Note: Here we only show the **positions** (0-indexed) in each trio.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
10 5
1 10 4 3 2 5 0 1 9 5
```

EXPECTED OUTPUT

```
26
```

Test Case 2

INPUT (STDIN)

```
10 5
11 10 14 31 21 15 10 11 9 51
```

EXPECTED OUTPUT

```
31
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(i<k)
```

Complexity Test Cases

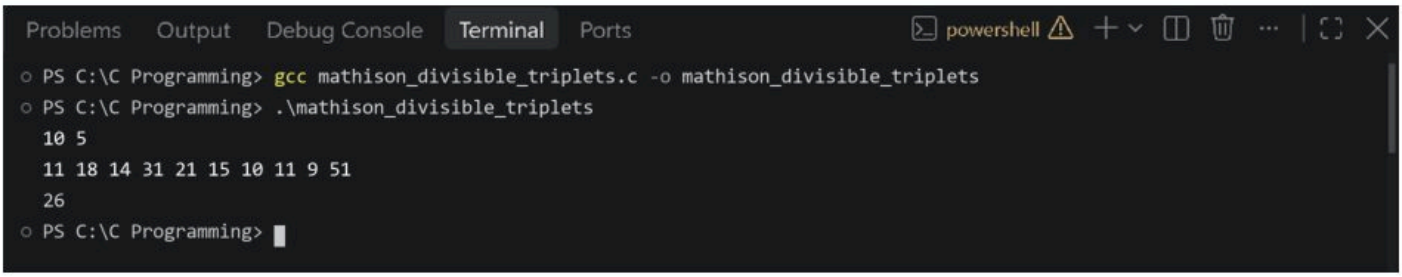
Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
10	450	55

Code

C mathison_divisible_triplets.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXM 10005

long long cnt[MAXM];

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;

    for (int i = 0; i < n; i++) {
        long long val;
        scanf("%lld", &val);
        cnt[((val % m) + m) % m]++;
    }

    long long ans = 0;

    for (int i = 0; i < m; i++) {
        if ((3LL * i) % m == 0 && cnt[i] >= 3) {
            ans += cnt[i] * (cnt[i] - 1) * (cnt[i] - 2) / 6;
        }
    }

    for (int i = 0; i < m; i++) {
        int r3 = (m - (2 * i) % m) % m;
        if (r3 != i && cnt[i] >= 2 && cnt[r3] >= 1) {
            ans += (cnt[i] * (cnt[i] - 1) / 2) * cnt[r3];
        }
    }

    for (int i = 0; i < m; i++) {
        for (int j = i + 1; j < m; j++) {
            int k = (m - (i + j) % m) % m;
            int ick = (j < k);
            while( ick) {
                ans += cnt[i] * cnt[j] * cnt[k];
                break;
            }
        }
    }

    printf("%lld\n", ans);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc mathison_divisible_triplets.c -o mathison_divisible_triplets
PS C:\C Programming> .\mathison_divisible_triplets
10 5
11 18 14 31 21 15 10 11 9 51
26
PS C:\C Programming>